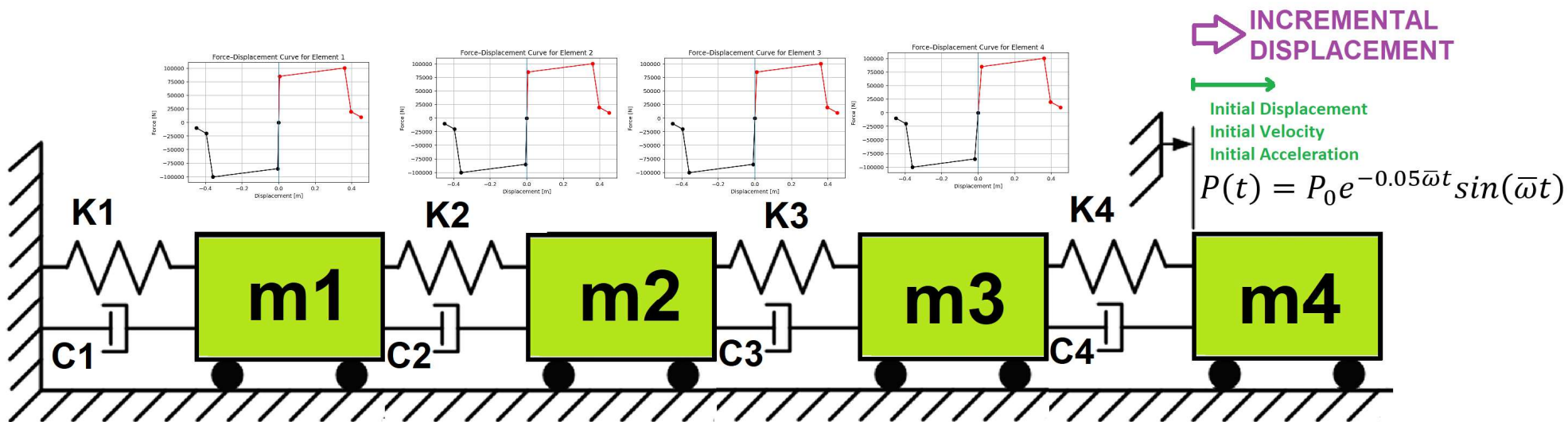


>> IN THE NAME OF ALLAH, THE MOST GRACIOUS, THE MOST MERCIFUL <<

COMPREHENSIVE NONLINEAR SEISMIC ASSESSMENT OF A MULTI-DEGREE- FREEDOM (MDOF) STRUCTURE: AN OPENSEES FRAMEWORK FOR STATIC PUSHOVER, CYCLIC DEGRADATION, STATIC TIME-HISTORY AND DYNAMIC TIME-HISTORY ANALYSIS

THIS PROGRAM IS WRITTEN BY SALAR DELAVAR GHASHGHAEI (QASHQAI)



$$\text{Structural Ductility Damage Index} = \frac{\Delta_d - \Delta_y}{\Delta_u - \Delta_y}$$

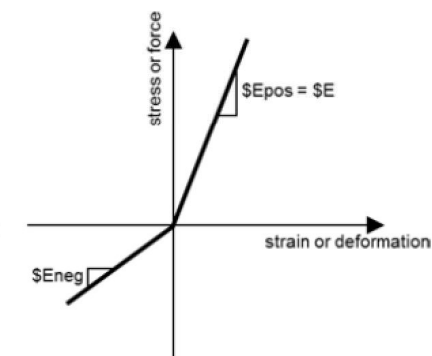
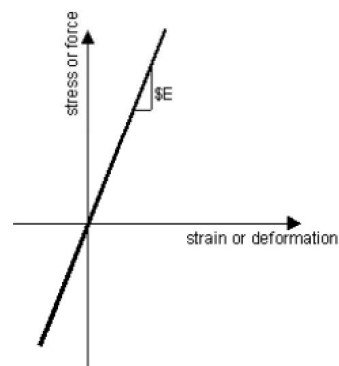
Δ_d = Lateral Displacement from Dynamic Analysis

Δ_y = Lateral Yield Displacement from Pushover Analysis

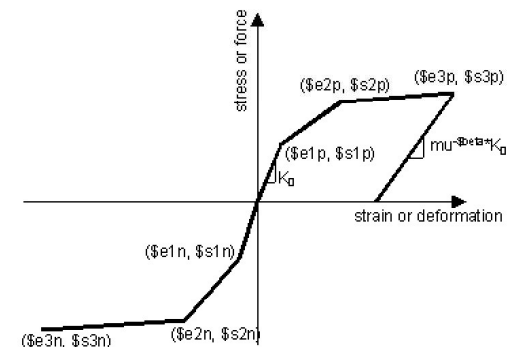
Δ_u = Lateral Ultimate Displacement from Pushover Analysis

$$P(t) = P_0 e^{-0.05\bar{\omega}t} \sin(\bar{\omega}t)$$

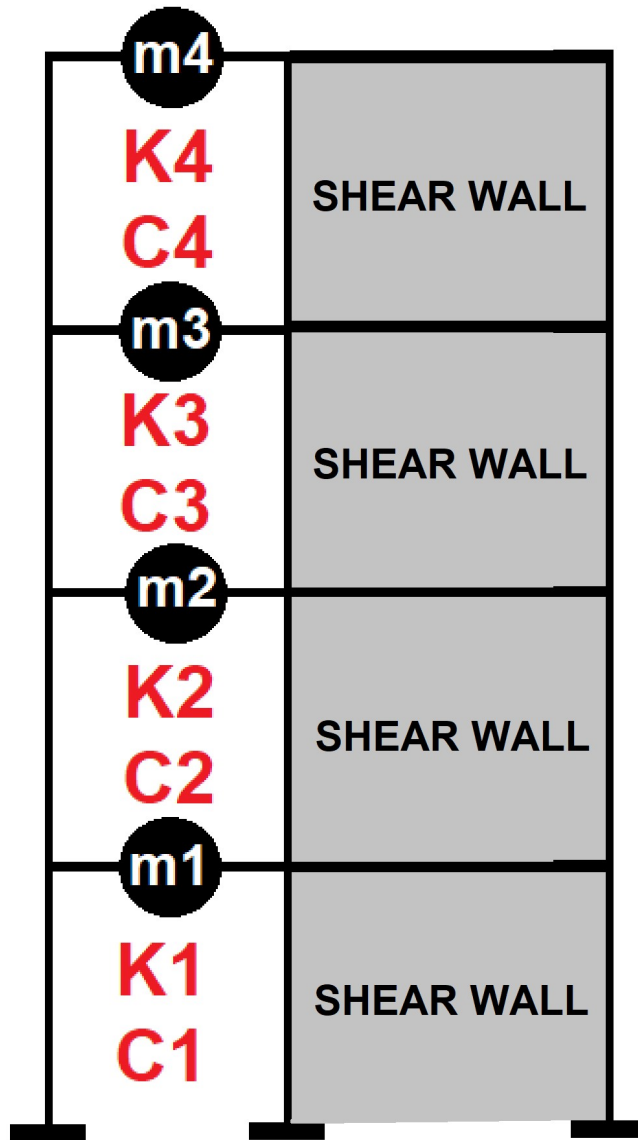
$$m\ddot{u} + c\dot{u} + ku = P(t)$$



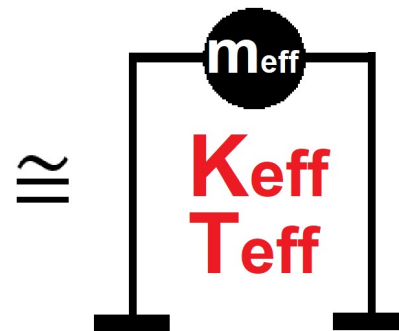
ELASTIC MATERIAL



INELASTIC MATERIAL

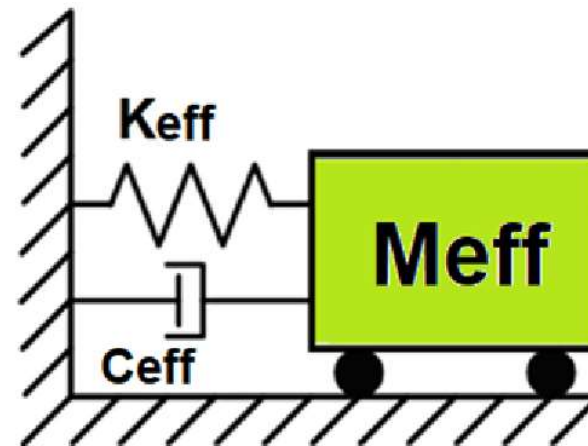


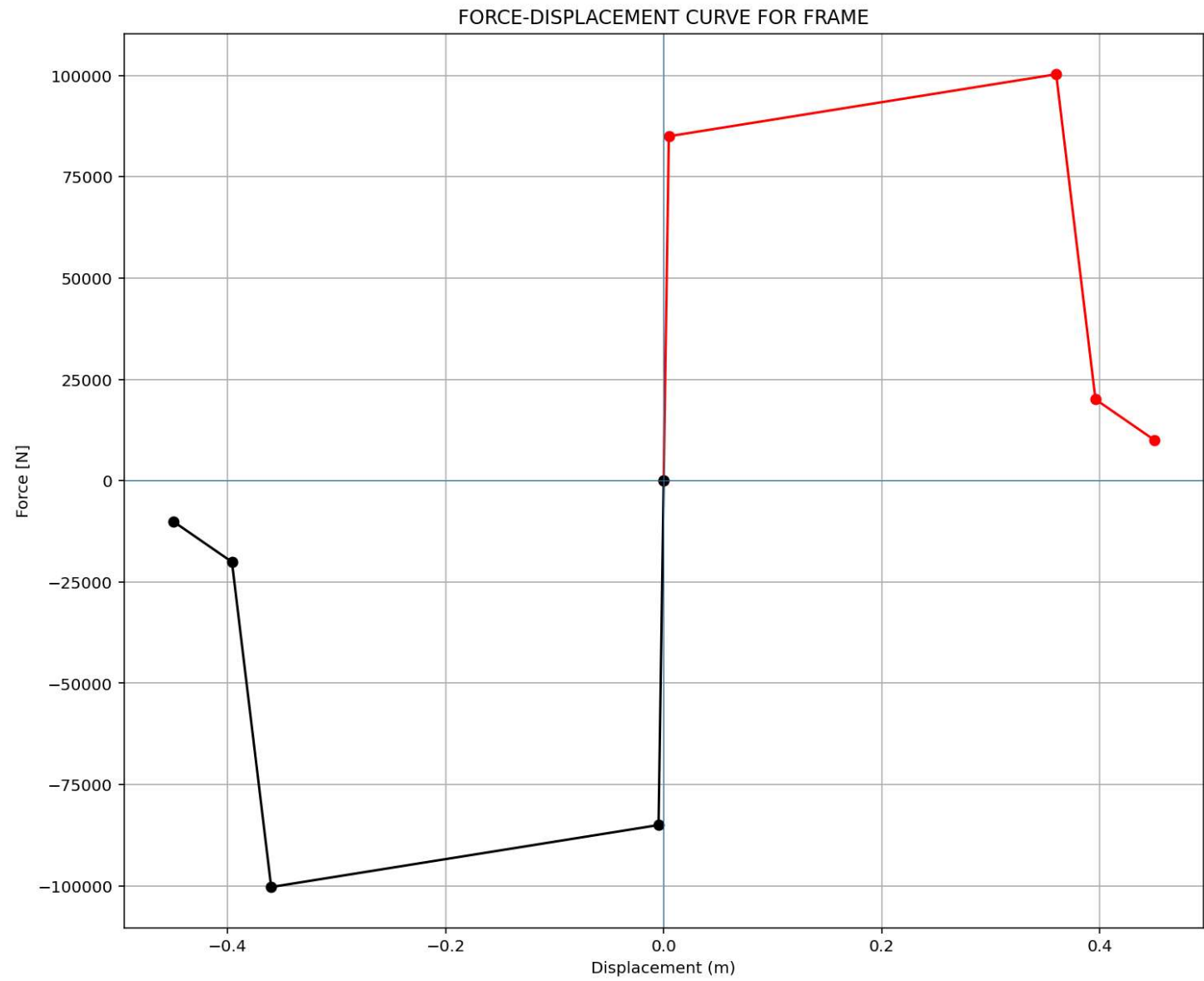
Displacement-based seismic analysis transformation,
converting a multi-degree-of-freedom (MDOF) system
into an equivalent single-degree-of-freedom (SDOF)
system for seismic assessment



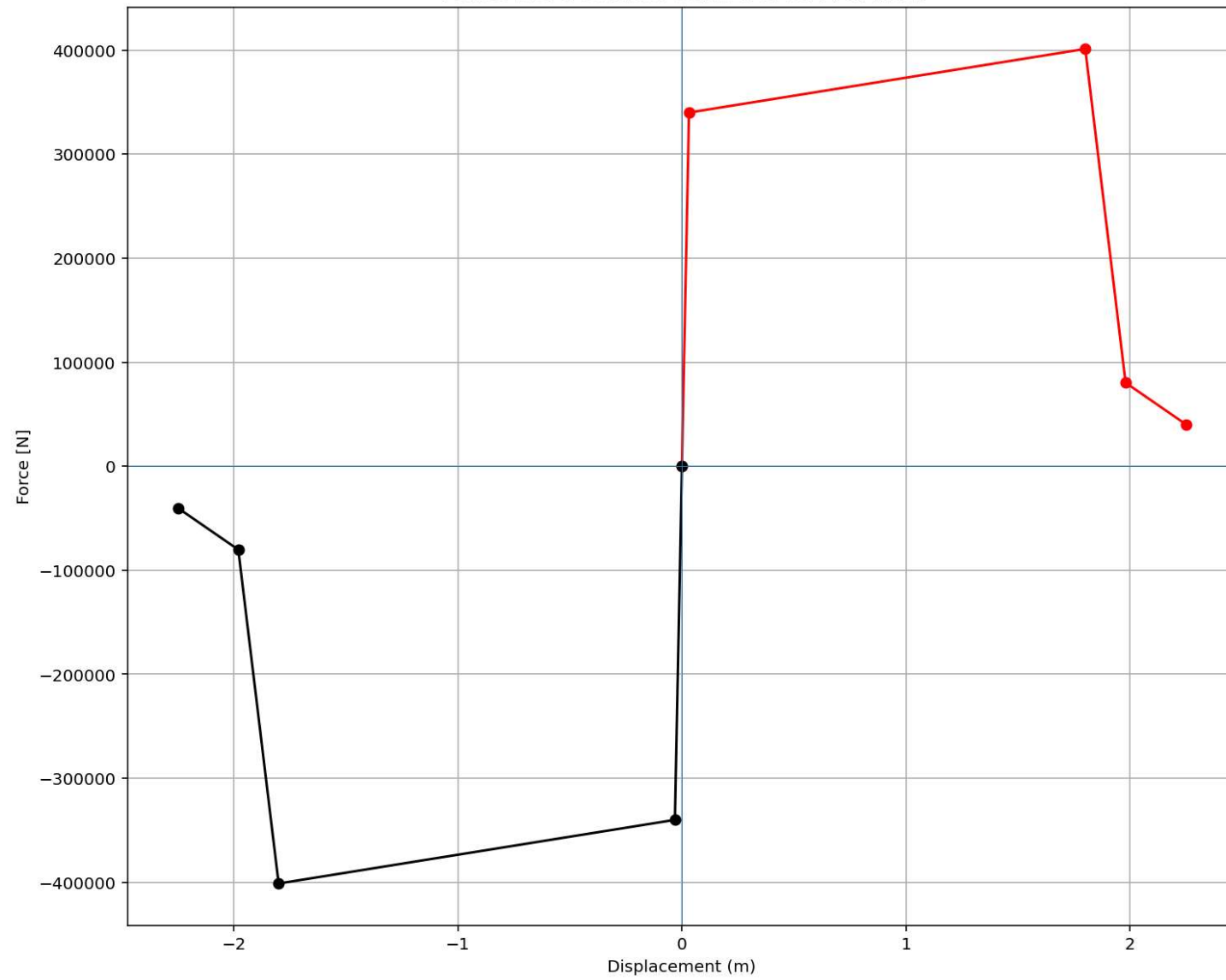
$$\Delta_d = \frac{\sum_{i=1}^n m_i \Delta_i^2}{\sum_{i=1}^n m_i \Delta_i} \quad m_{eff} = \frac{\sum_{i=1}^n m_i \Delta_i}{\Delta_d}$$

$$k_{eff} = \frac{\sum_{i=1}^n k_i \Delta_i}{\Delta_d} \quad T_{eff} = \frac{2\pi}{\sqrt{\frac{k_{eff}}{m_{eff}}}}$$





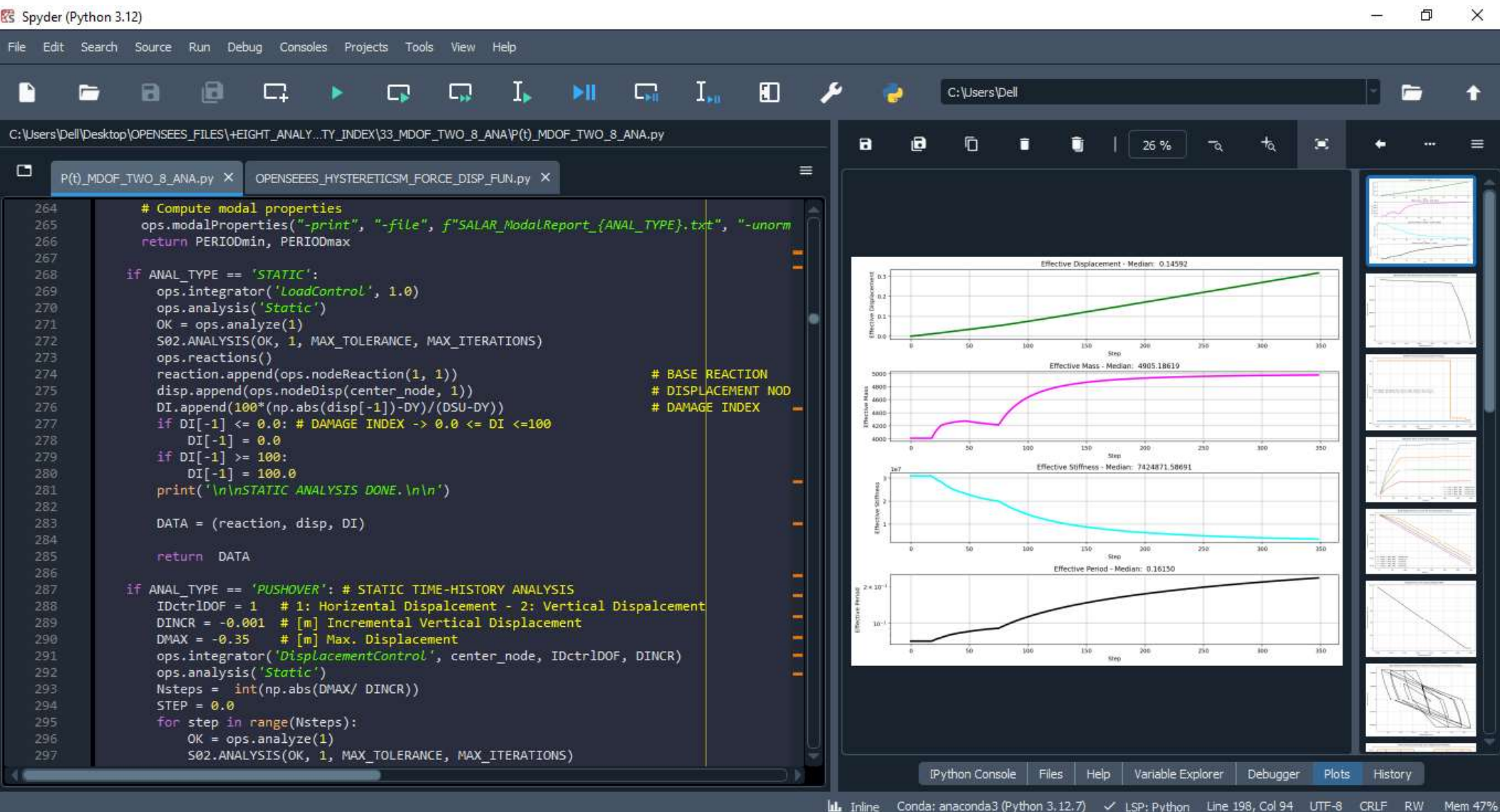
FORCE-DISPLACEMENT CURVE FOR SHEAR-WALL



Number of people	Time taken (minutes)
1	100
2	50
3	33
4	25
5	20
6	17
7	15
8	14
9	13
10	12



STATIC ANALYSIS



PUSHOVER ANALYSIS

Spyder (Python 3.12)

File Edit Search Source Run Debug Consoles Projects Tools View Help

C:\Users\Dell\Desktop\OPENSEES_FILES\+EIGHT_ANALY...TY_INDEX\33_MDOF_TWO_8_ANA\P(t)_MDOF_TWO_8_ANA.py

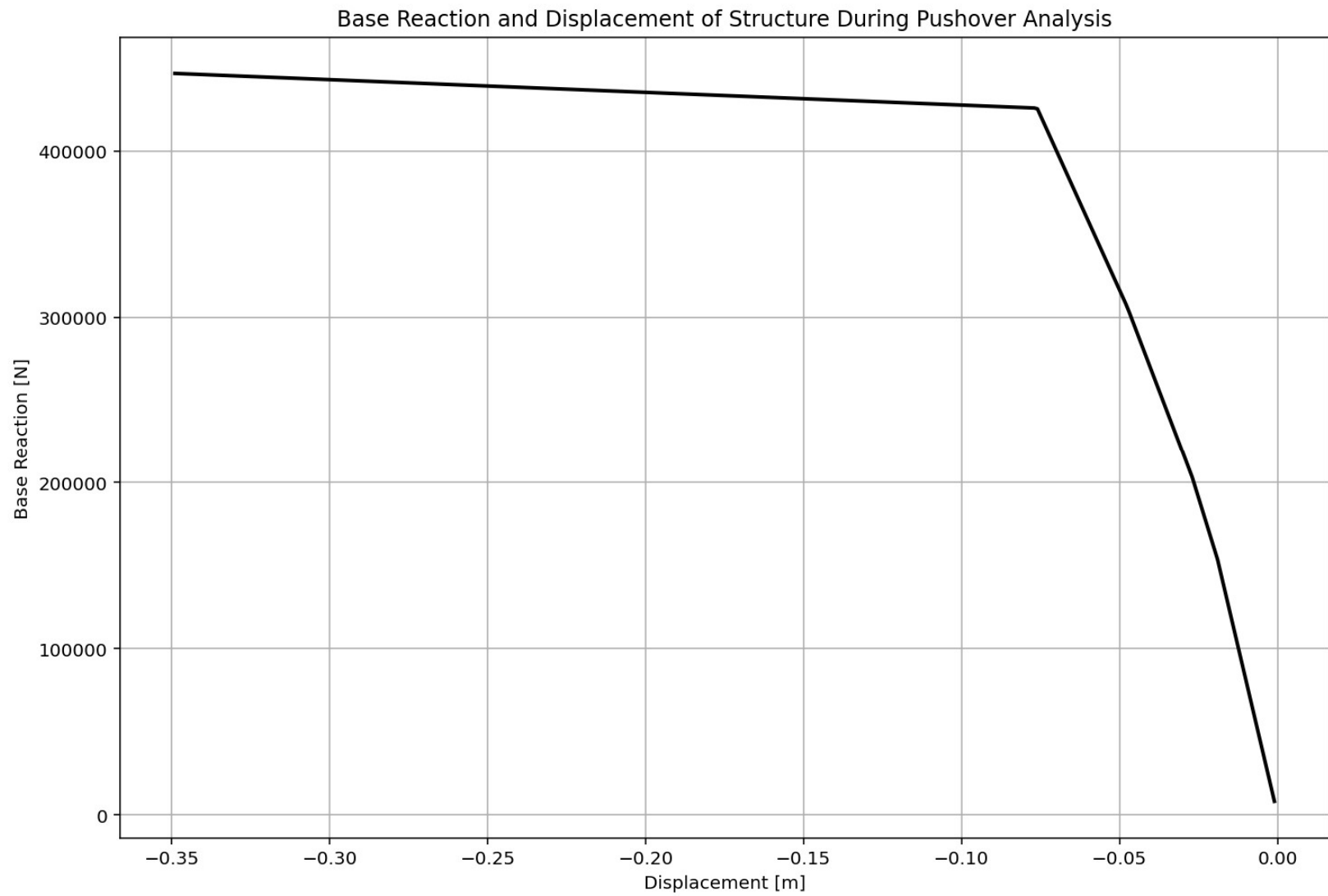
P(t)_MDOF_TWO_8_ANA.py X OPENSEES_HYSTERETICSM_FORCE_DISP_FUN.py X

```
287 if ANAL_TYPE == 'PUSHOVER': # STATIC TIME-HISTORY ANALYSIS
288     IDctrlDOF = 1 # 1: Horizontal Displacement - 2: Vertical Displacement
289     DINCR = -0.001 # [m] Incremental Vertical Displacement
290     DMAX = -0.35 # [m] Max. Displacement
291     ops.integrator('DisplacementControl', center_node, IDctrlDOF, DINCR)
292     ops.analysis('Static')
293     Nsteps = int(np.abs(DMAX/ DINCR))
294     STEP = 0.0
295     for step in range(Nsteps):
296         OK = ops.analyze(1)
297         S02.ANALYSIS(OK, 1, MAX_TOLERANCE, MAX_ITERATIONS)
298         ops.reactions()
299         reaction.append(ops.nodeReaction(1, 1)) # BASE REACTION
300         disp.append(ops.nodeDisp(center_node, 1)) # DISPLACEMENT
301         # IN EACH STEP, STRUCTURE PERIOD GOING TO BE CALCULATED
302         #PERIODmin, PERIODmax = S06.RAYLEIGH_DAMPING(2, 0.5*DR, DR, 0, 1)
303         PERIODmin, PERIODmax = S05.EIGENVALUE_ANALYSIS(2, PLOT=True)
304         PERIOD_MIN.append(PERIODmin)
305         PERIOD_MAX.append(PERIODmax)
306         DI.append(100*(np.abs(disp[-1])-DY)/(DSU-DY)) # DAMAGE INDEX
307         if DI[-1] <= 0.0: # DAMAGE INDEX -> 0.0 <= DI <=100
308             DI[-1] = 0.0
309         if DI[-1] >= 100:
310             DI[-1] = 100.0
311         # Store forces and displacements
312         for ele_id in ele_force.keys():
313             ele_force[ele_id].append(ops.eleResponse(ele_id, 'force')[0]) # [N]
314         # Store displacements
315         for node_id in node_displacements.keys():
316             node_displacements[node_id].append(ops.nodeDisp(node_id, 1))
317         STEP += 1
318         #print(STEP, disp[-1], reaction[-1])
319         print(f"Step: {STEP}, Displacement: {disp[-1]:.4f} mm, Reaction: {reaction[-1]}
320     else:
```

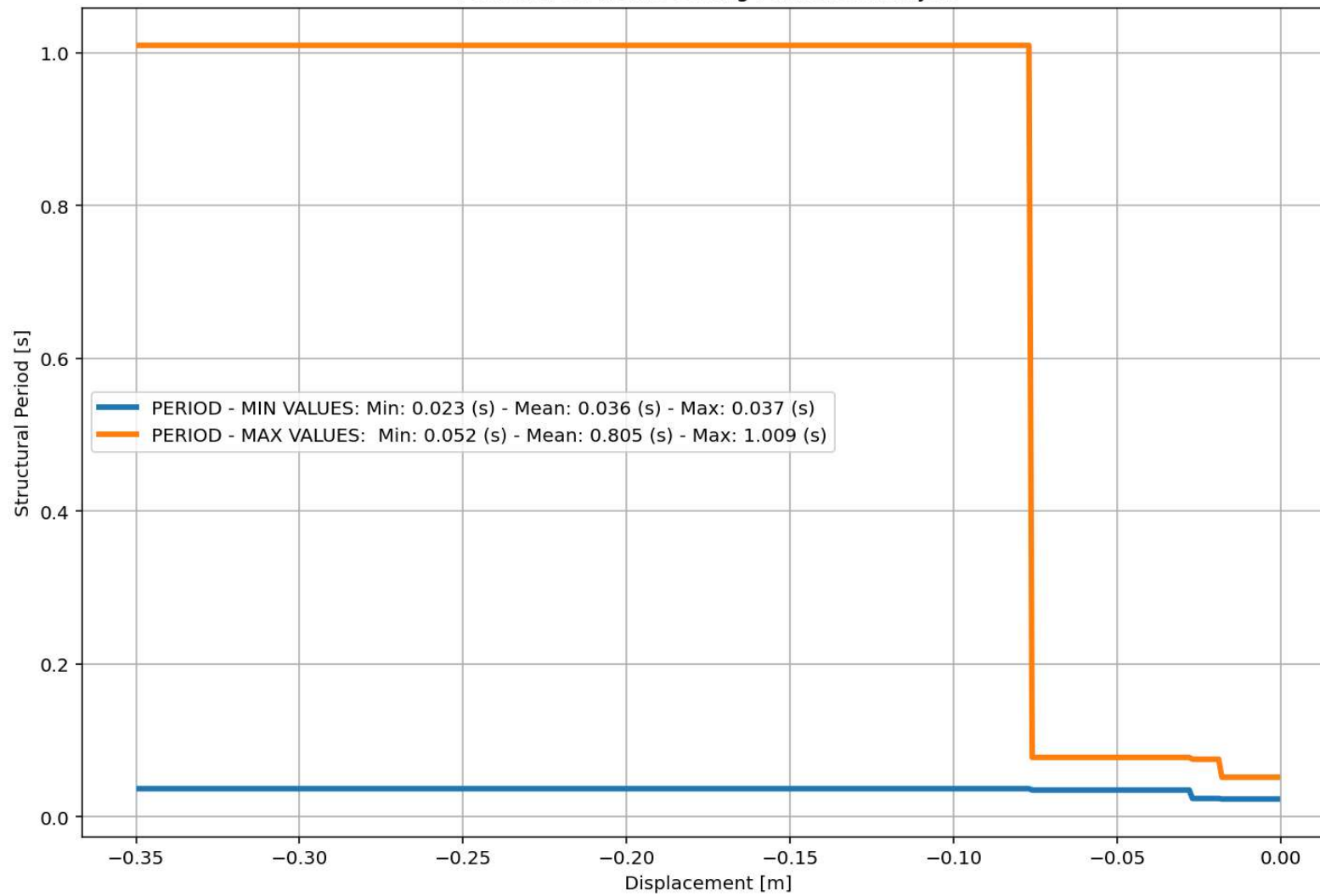
Base Reaction and Displacement of Structure During Pushover Analysis

Python Console Files Help Variable Explorer Debugger Plots History

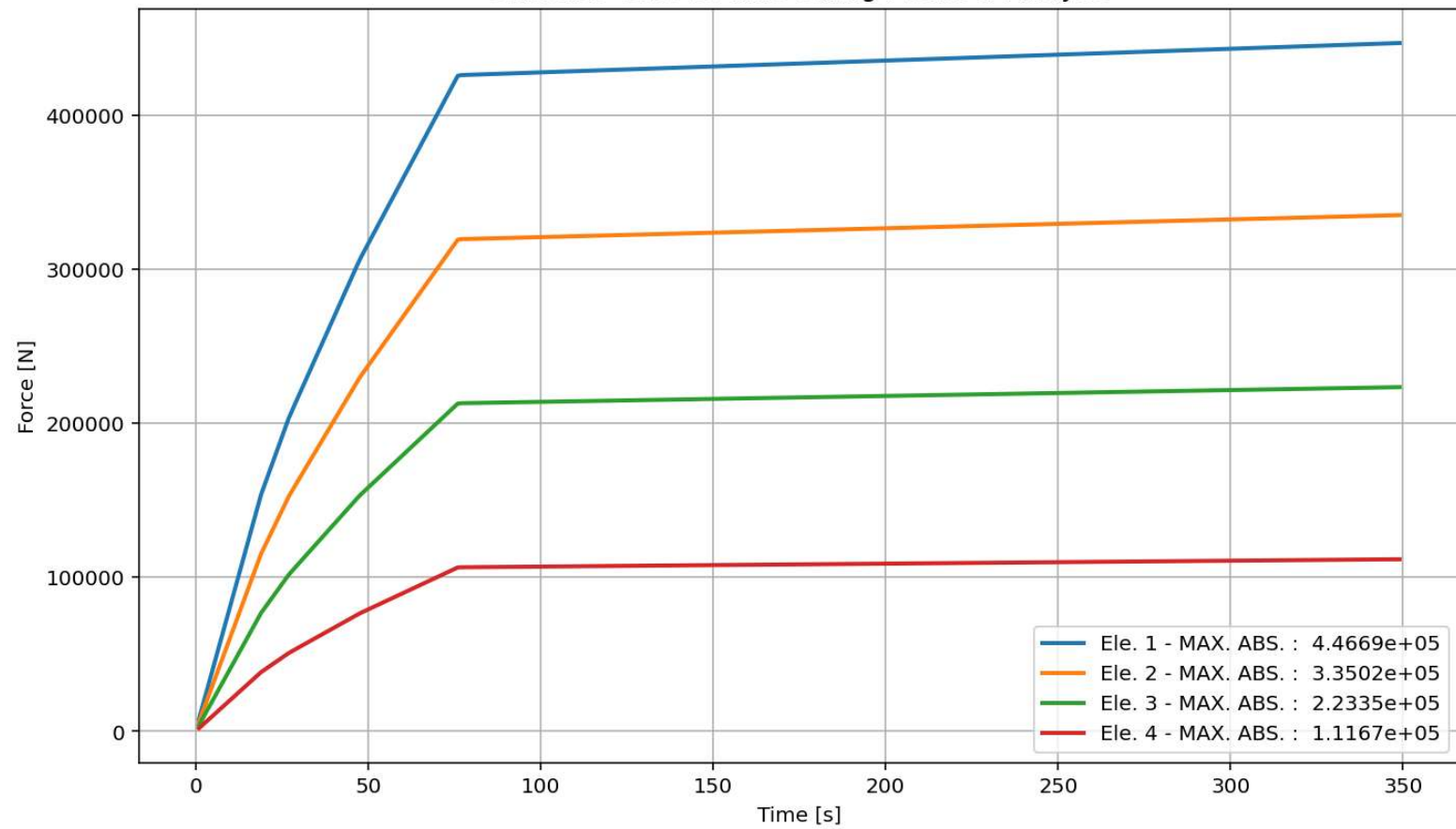
Inline Conda: anaconda3 (Python 3.12.7) ✓ LSP: Python Line 198, Col 94 UTF-8 CRLF RW Mem 47%



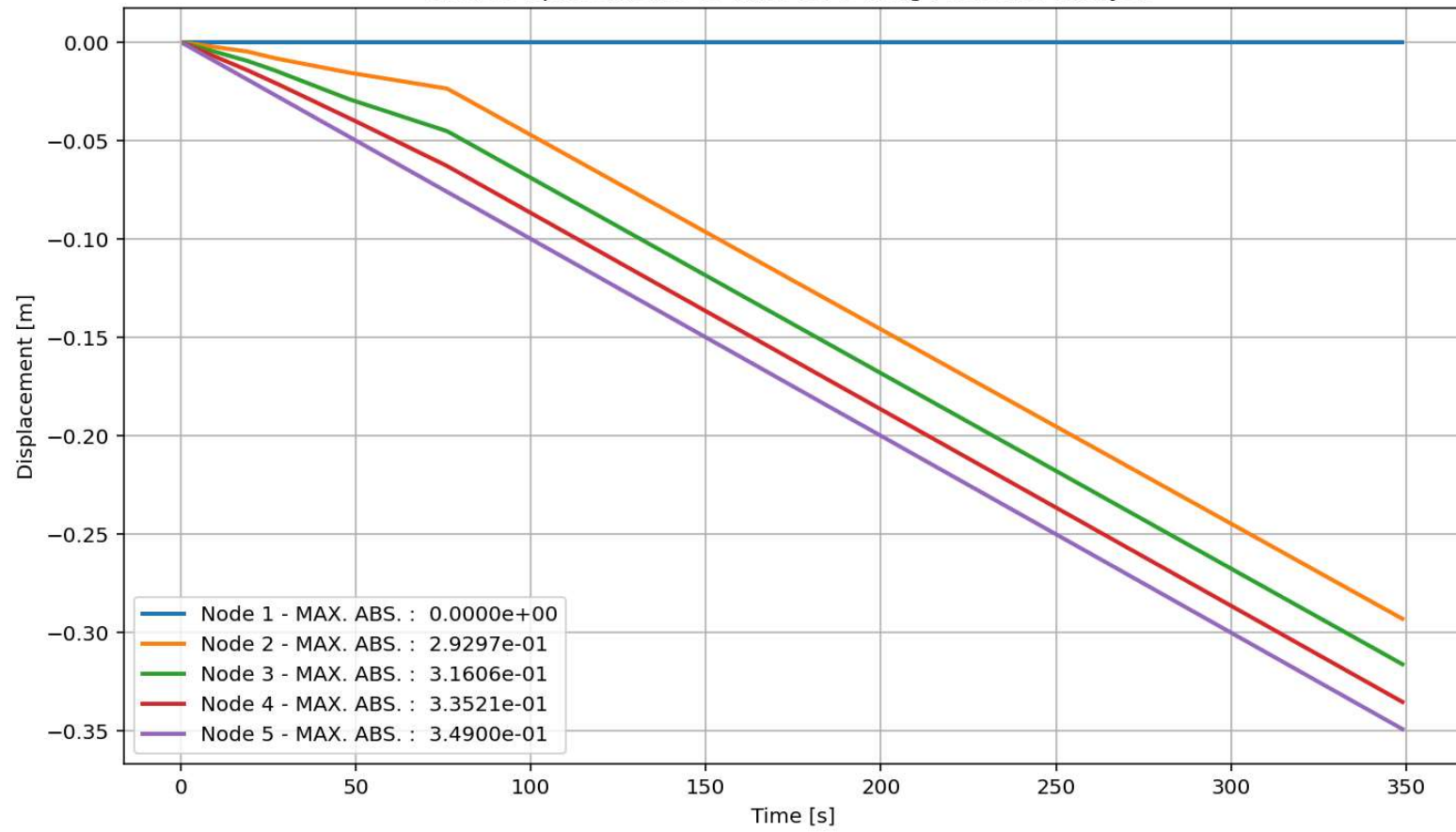
Period of Structure During Pushover Analysis

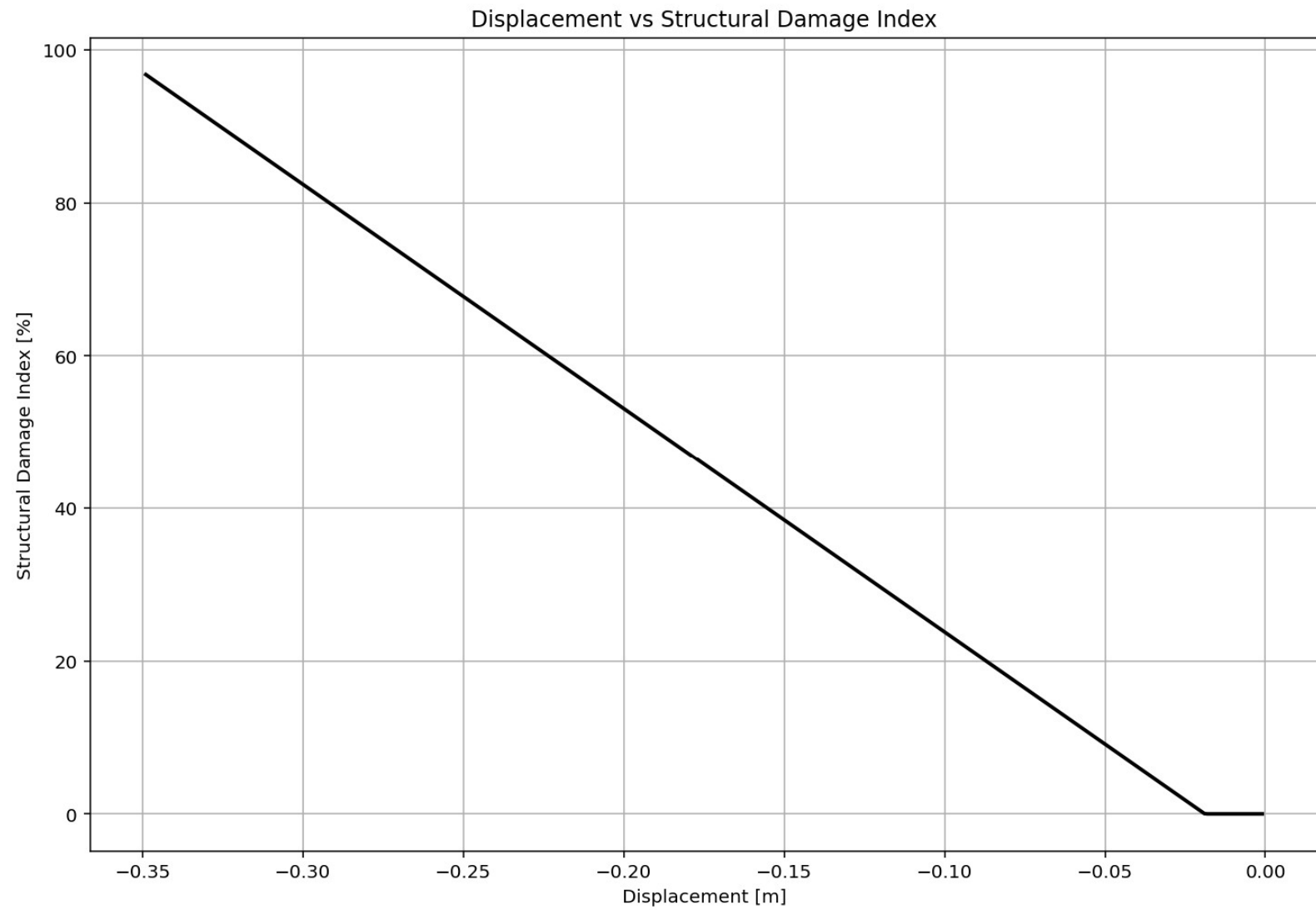


elements force vs Time During Pushover Analysis



Node Displacements vs Time for During Pushover Analysis





Spyder (Python 3.12)

File Edit Search Source Run Debug Consoles Projects Tools View Help

C:\Users\Dell\Desktop\OPENSEES_FILES\+EIGHT_ANALY..._ANA\COMPUTE_EFFECTIVE_PROPERTIES_FUN_PUSHOVER.py

P(t)_MDOF_TWO_8_ANA.py x COMPUTE_EFFECTIVE...ES_FUN_PUSHOVER.py x

```
1  """ EQUIVALENT SDOF SYSTEM DERIVATION VIA DISPLACEMENT-BASED SEISMIC DESIGN PROCEDURE WITH P
2  # Change MDOF to SDOF System with Displacement Based Design Concept
3  # THIS PROGRAM WRITTEN BY SALAR DELAVAR GHASHGHAEEI (QASHQAI)
4  """
5  This script implements a displacement-based pushover transformation,
6  converting a multi-degree-of-freedom (MDOF) system into an equivalent
7  single-degree-of-freedom (SDOF) system for seismic assessment.
8  It calculates effective modal properties-displacement, mass, and
9  stiffness-by weighting element forces and nodal displacements according
10 to a presumed deformed shape.
11 The derived effective period provides a simplified dynamic characteristic
12 for performance-based engineering. The visualizations effectively track the
13 evolution of these equivalent parameters throughout the nonlinear analysis steps.
14 """
15
16 displacement_X_1 = np.array(list(node_displacements.values())[1]) # DOF 02
17 displacement_X_2 = np.array(list(node_displacements.values())[2]) # DOF 03
18 displacement_X_3 = np.array(list(node_displacements.values())[3]) # DOF 04
19 displacement_X_4 = np.array(list(node_displacements.values())[4]) # DOF 05
20
21 ele_force_01 = np.array(list(ele_force.values())[0]) # ELEMENT 01
22 ele_force_02 = np.array(list(ele_force.values())[1]) # ELEMENT 02
23 ele_force_03 = np.array(list(ele_force.values())[2]) # ELEMENT 03
24 ele_force_04 = np.array(list(ele_force.values())[3]) # ELEMENT 04
25
26 STIFF_X_1 = np.abs(ele_force_01 / displacement_X_1)
27 STIFF_X_2 = np.abs(ele_force_02 / displacement_X_2)
28 STIFF_X_3 = np.abs(ele_force_03 / displacement_X_3)
29 STIFF_X_4 = np.abs(ele_force_04 / displacement_X_4)
30
31 MX2 = (MASS[0] * np.square(displacement_X_1) +
32        MASS[1] * np.square(displacement_X_2) +
33        MASS[2] * np.square(displacement_X_3) +
34        MASS[3] * np.square(displacement_X_4))
```

Effective Displacement - Median: 0.14592

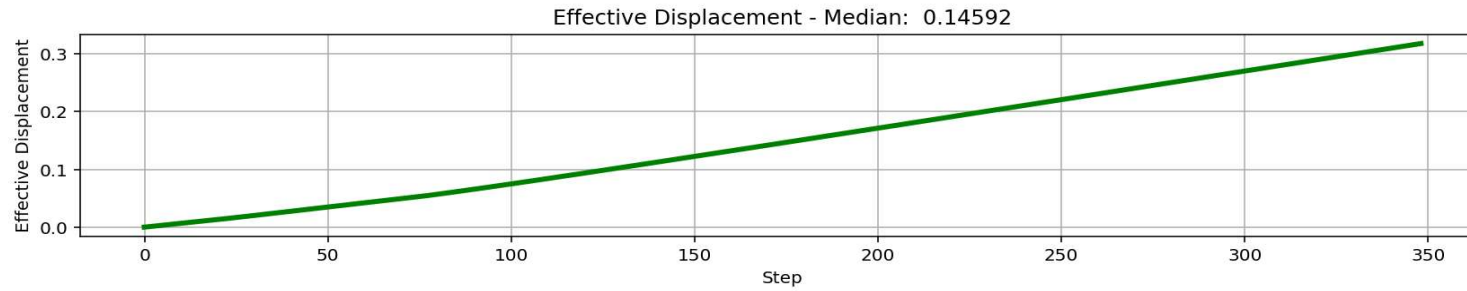
Effective Mass - Median: 4905.18619

Effective Stiffness - Median: 7424871.58691

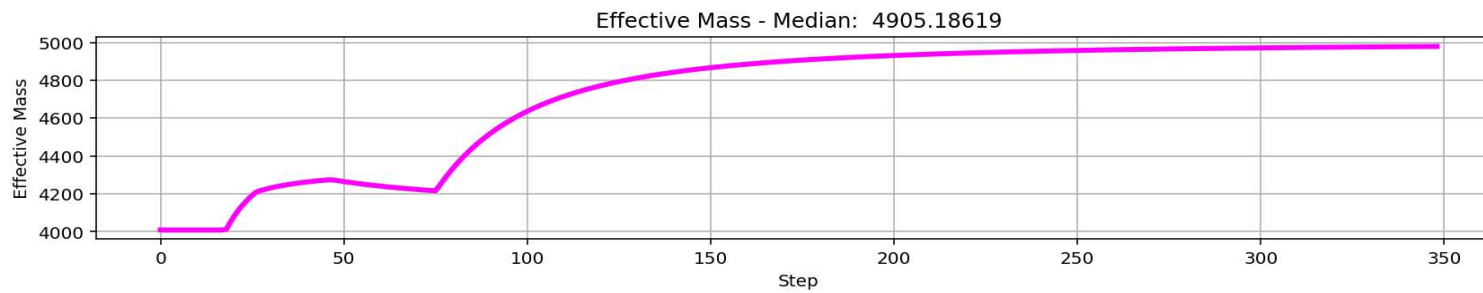
Effective Period - Median: 0.16150

[Python Console] Files Help Variable Explorer Debugger Plots History

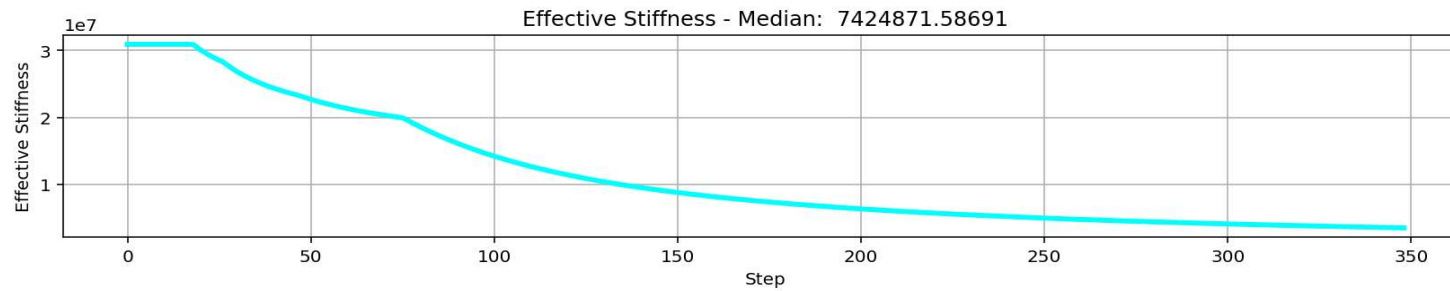
Inline Conda: anaconda3 (Python 3.12.7) ✓ LSP: Python Line 1, Col 1 UTF-8 CRLF RW Mem 49%



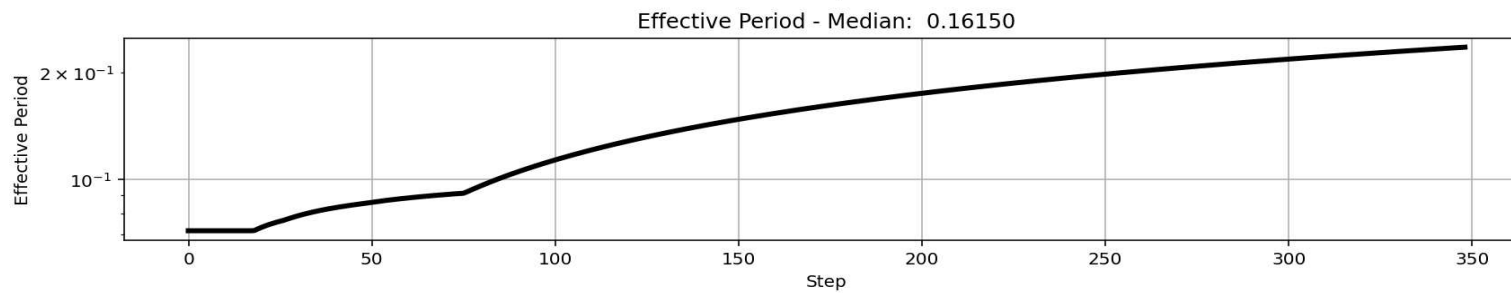
$$\Delta_d = \frac{\sum_{i=1}^n m_i \Delta_i^2}{\sum_{i=1}^n m_i \Delta_i}$$



$$m_{eff} = \frac{\sum_{i=1}^n m_i \Delta_i}{\Delta_d}$$



$$k_{eff} = \frac{\sum_{i=1}^n k_i \Delta_i}{\Delta_d}$$



$$T_{eff} = \frac{2\pi}{\sqrt{\frac{k_{eff}}{m_{eff}}}}$$

CYCLIC DISPLACEMENT ANALYSIS

Spyder (Python 3.12)

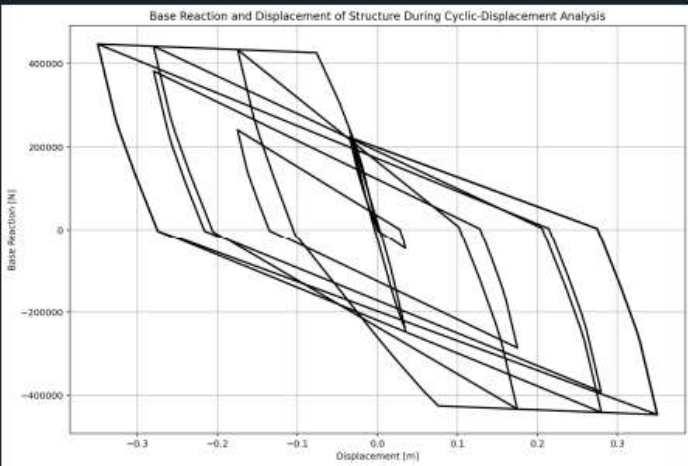
File Edit Search Source Run Debug Consoles Projects Tools View Help

C:\Users\Dell\Desktop\OPENSEES_FILES\+EIGHT_ANALY...TY_INDEX\33_MDOF_TWO_8_ANA\P(t)_MDOF_TWO_8_ANA.py

P(t)_MDOF_TWO_8_ANA.py COMPUTE_EFFECTIVE_PROPERTIES_FUN.py

```
331
332 if ANAL_TYPE == 'CYCLIC_DISPLACEMENT': # STATIC TIME-HISTORY ANALYSIS
333     IDctrlDOF = 1 # 1: Horizontal Displacement - 2: Vertical Displacement
334     DMAX = -0.35 # [m] Max. Displacement
335     n_points = 10000
336     # 4. CYCLIC DISPLACEMENT PROTOCOL
337     # Key strain points (same logic as your protocol)
338     key_disp = np.array([
339         0.0,
340         0.1*DMAX, -0.1*DMAX,
341         0.5*DMAX, -0.5*DMAX,
342         0.8*DMAX, -0.8*DMAX,
343         DMAX, -DMAX,
344         0.1*DMAX, -0.1*DMAX,
345         0.5*DMAX, -0.5*DMAX,
346         0.8*DMAX, -0.8*DMAX,
347         DMAX, -DMAX,
348         0.0
349     ])
350
351     # Generate 1000-point displacement protocol
352     disp_protocol = np.interp(
353         np.linspace(0, len(key_disp) - 1, n_points),
354         np.arange(len(key_disp)),
355         key_disp
356     )
357
358     ops.analysis('Static')
359     STEP = 0.0
360     for target_disp in disp_protocol:
361         current_disp = ops.nodeDisp(center_node, 1) # DISPLACEMENT APPLIED IN MIDDLE N
362         dU = target_disp - current_disp
363         ops.integrator('DisplacementControl', center_node, IDctrlDOF, dU)
364         OK = ops.analyze(1)
```

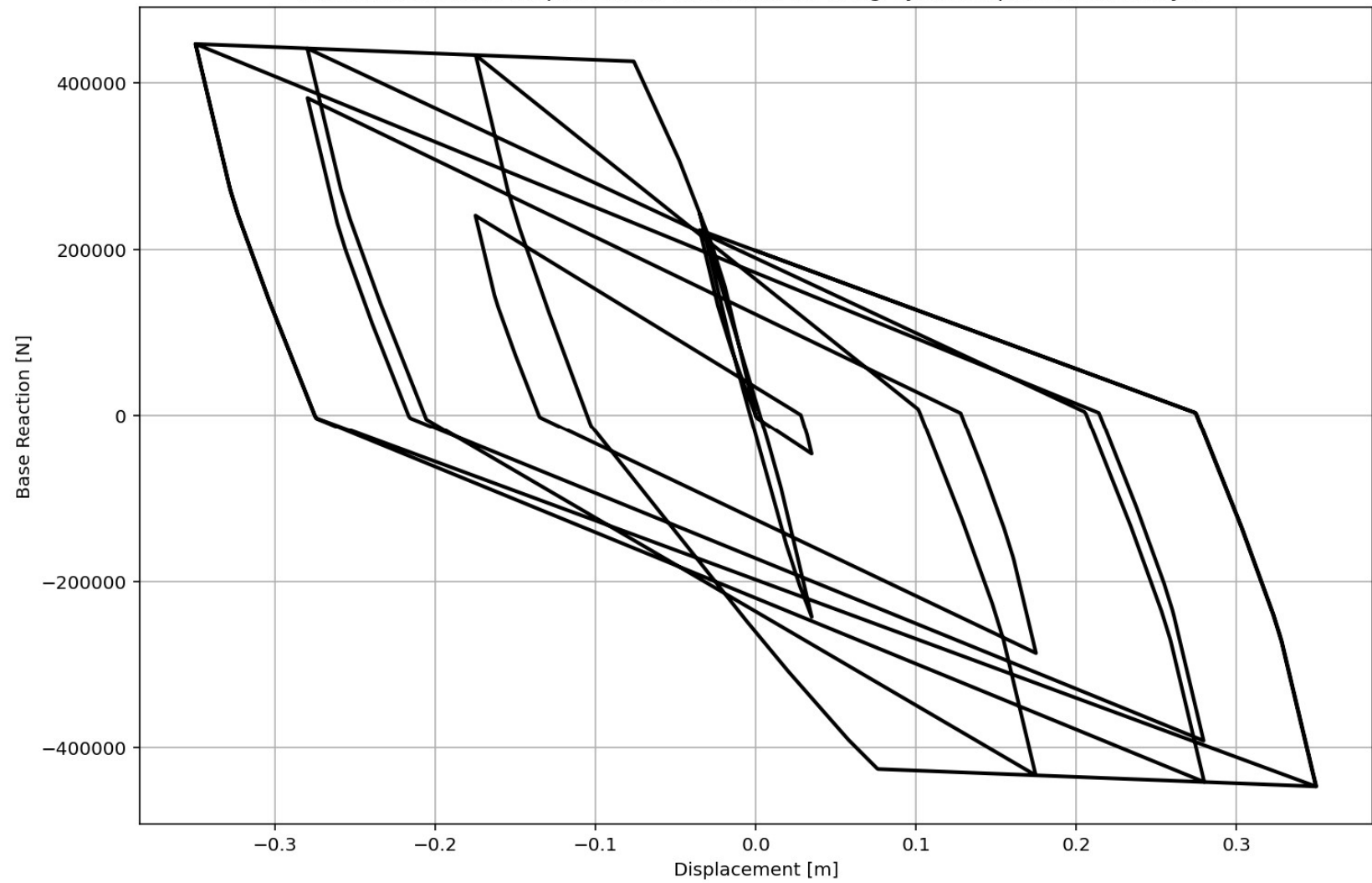
Base Reaction and Displacement of Structure During Cyclic-Displacement Analysis



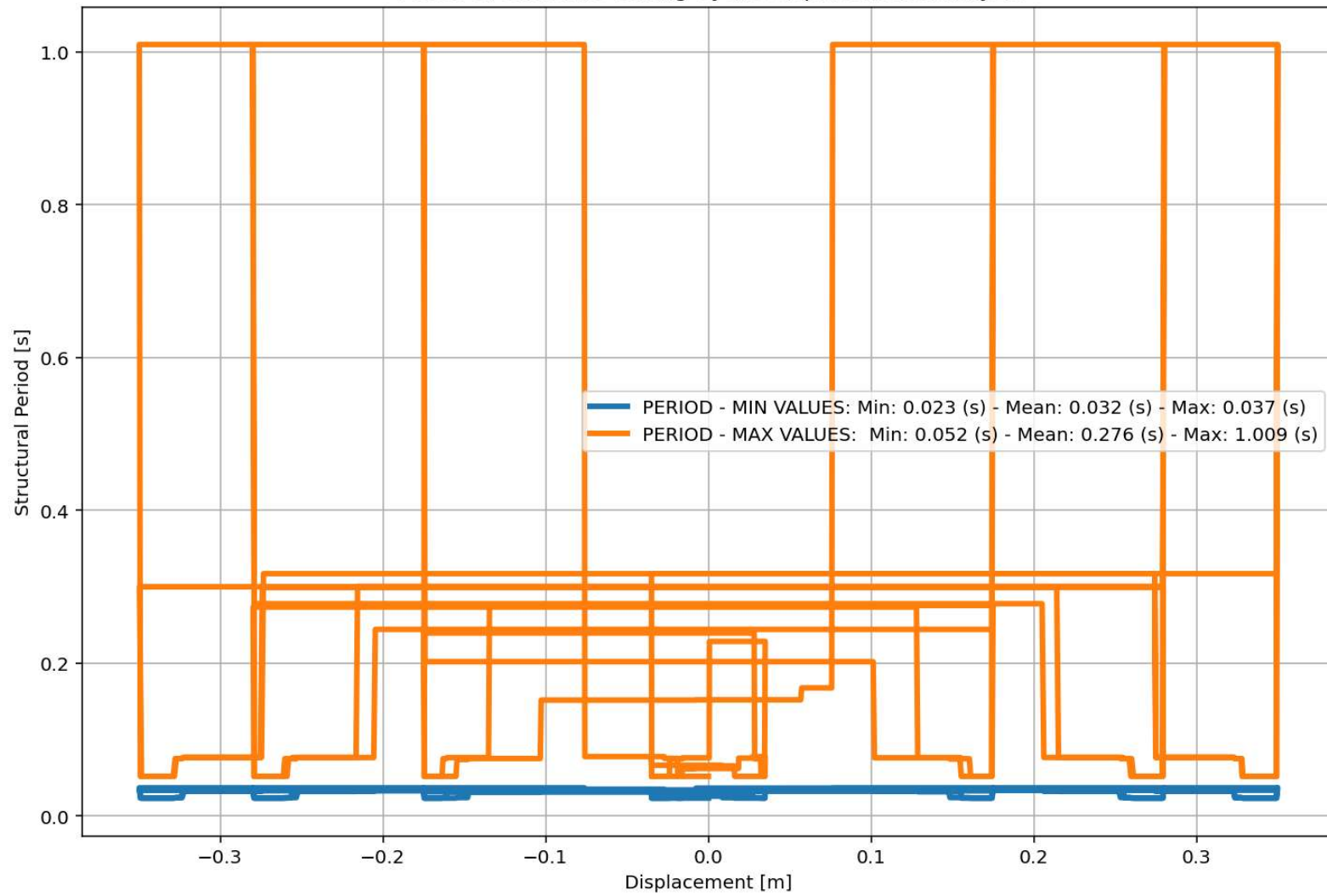
Python Console Files Help Variable Explorer Debugger Plots History

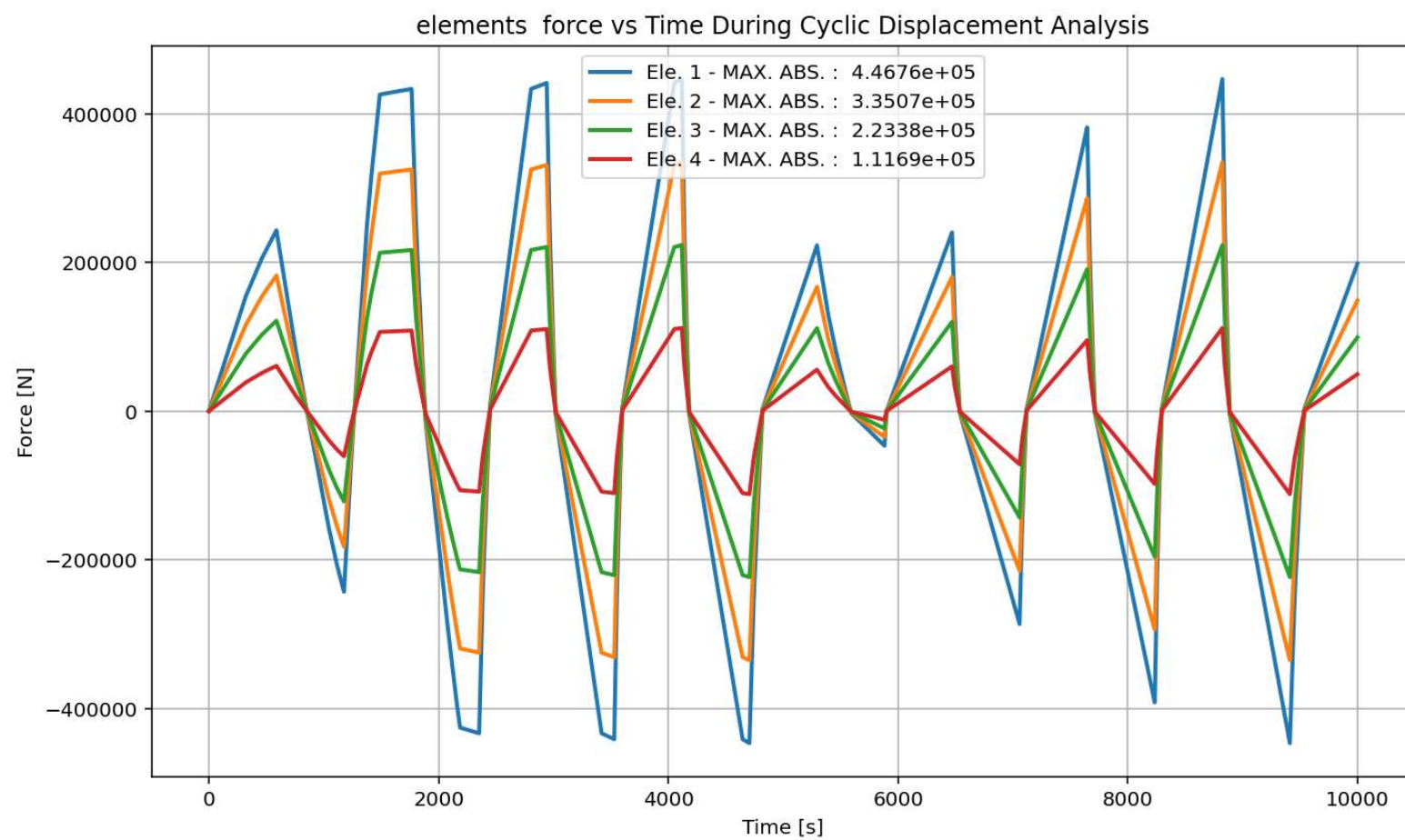
Inline Conda: anaconda3 (Python 3.12.7) LSP: Python Line 198, Col 94 UTF-8 CRLF RW Mem 49%

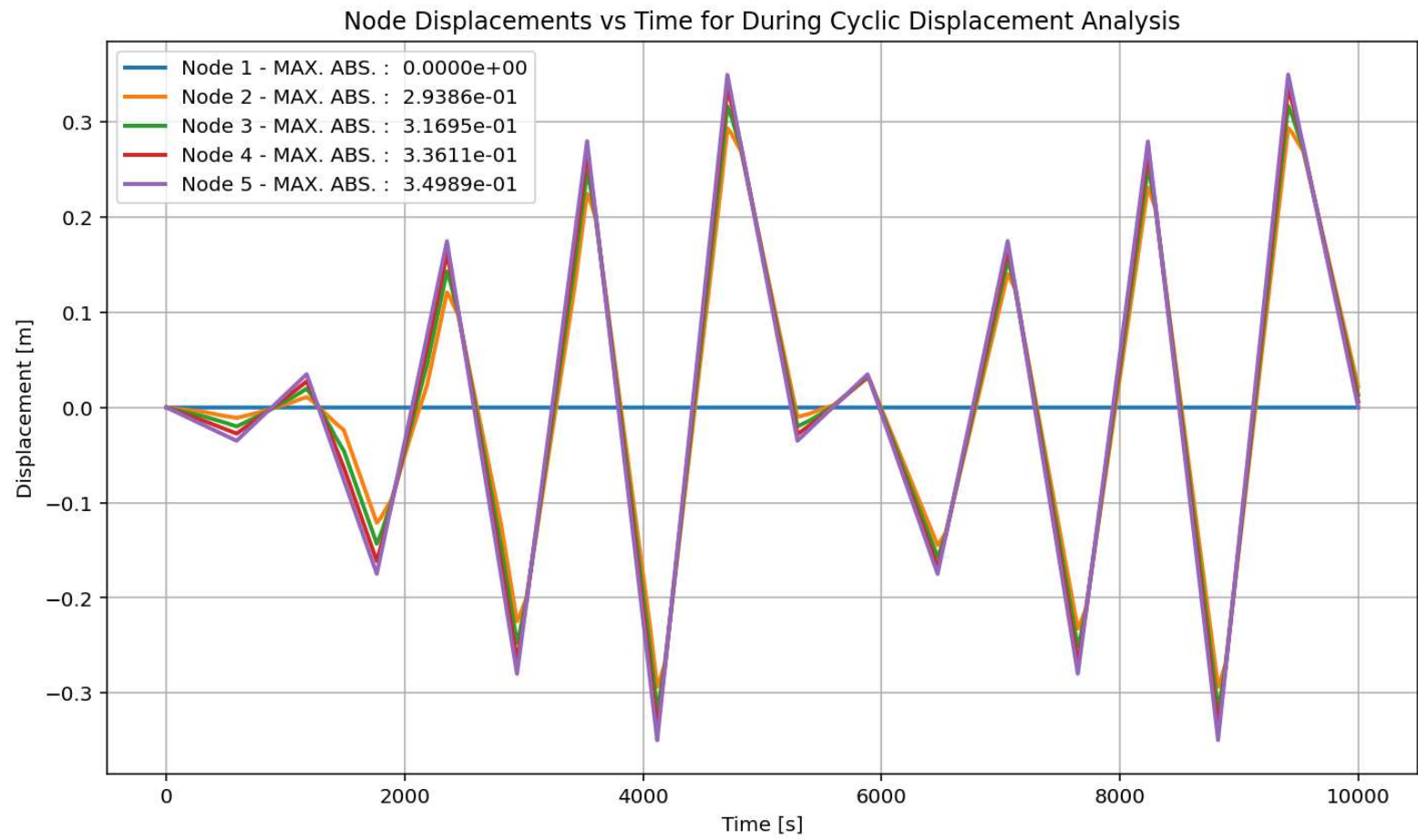
Base Reaction and Displacement of Structure During Cyclic-Displacement Analysis



Period of Structure During Cyclic Displacement Analysis

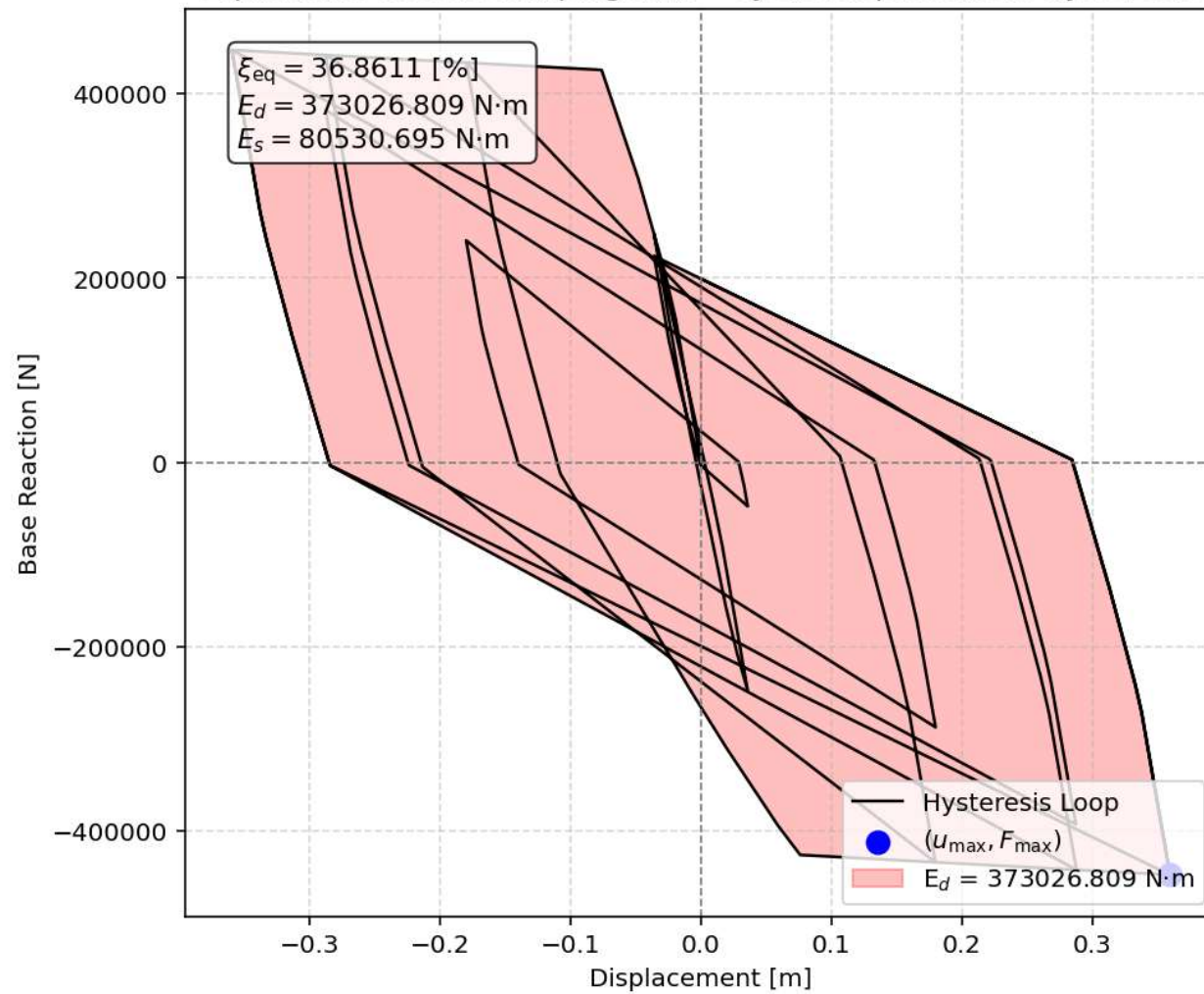








Equivalent viscous damping ratio - Cyclic Displacement Hysteresis



STATIC TIME-HISTORY WITH EXTERNAL TIME-DEPENDENT LOADING ANALYSIS

Spyder (Python 3.12)

File Edit Search Source Run Debug Consoles Projects Tools View Help

C:\Users\Dell\Desktop\OPENSEES_FILES\HEIGHT_ANALY...N_CAPACITY_INDEX\22_MDOF_8_ANA\P(t)_MDOF_8_ANA.py

P(t)_MDOF_8_ANA.py

```
349 if ANAL_TYPE == 'STATIC EXTERNAL TIME-DEPENDENT LOADING': # STATIC TIME-HISTORY ANALYSIS
350     %% DEFINE EXTERNAL TIME-DEPENDENT LOADING PROPERTIES
351     # IN HERE ANALYSIS TIME AND DURATION ARE LOAD STEPS
352     duration = 20.0 # [s] Analysis duration
353     dt = 0.01 # [s] Time step
354     DT = dt # [s] Time step
355     DT_time = 5.0 # [s] Total external Load Analysis Durations [*****]
356     force_amplitude = 5960.0 # [N] Amplitude Force
357     omega_DT = 5.0715 # [rad/s] Natural angular frequency
358     time_steps = int(duration/dt)
359
360     # Check function
361     def CHECK_FUN(DT_time ,duration):
362         if DT_time > duration:
363             print('\n\nAnalysis Duration Must be greater than External Load Duration\n\n')
364             exit()
365         return -1
366
367     CHECK_FUN(DT_time ,duration)
368     def EXTERNAL_TIME_DEPENDENT(force_amplitude, omega_DT, DT, DT_time): # P(t) = P0 sin(omega*t)
369         import numpy as np
370         import matplotlib.pyplot as plt
371         # External Load Durations
372         num_steps = int(DT_time / DT)
373         load_time = np.linspace(0, DT_time, num_steps)
374         target_frequency = 1.0 * omega_DT # Target excitation frequency
375         DT_load = force_amplitude * np.sin(target_frequency * load_time)
376         # Plot External Time-dependent Loading
377         plt.figure(figsize=(10, 6))
378         plt.plot(load_time, DT_load, label=f'External Loading - Max: {np.max(DT_load)}')
379         plt.title('External Time-dependent Loading Over Time')
380         plt.xlabel('Time (s)')
381         plt.ylabel('Force (N)')
382         plt.grid(True)
```

External Time-dependent Loading Over Time

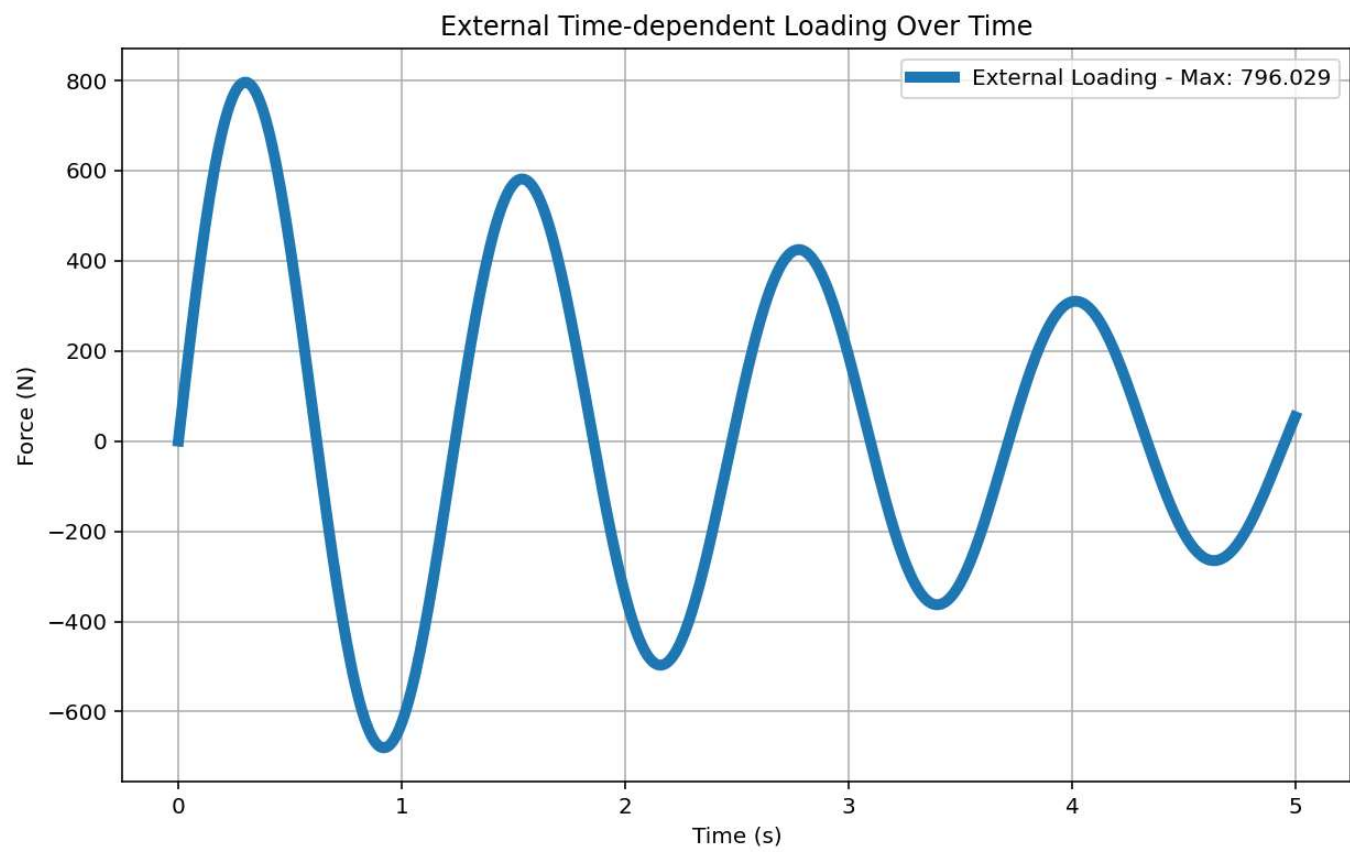
Force (N)

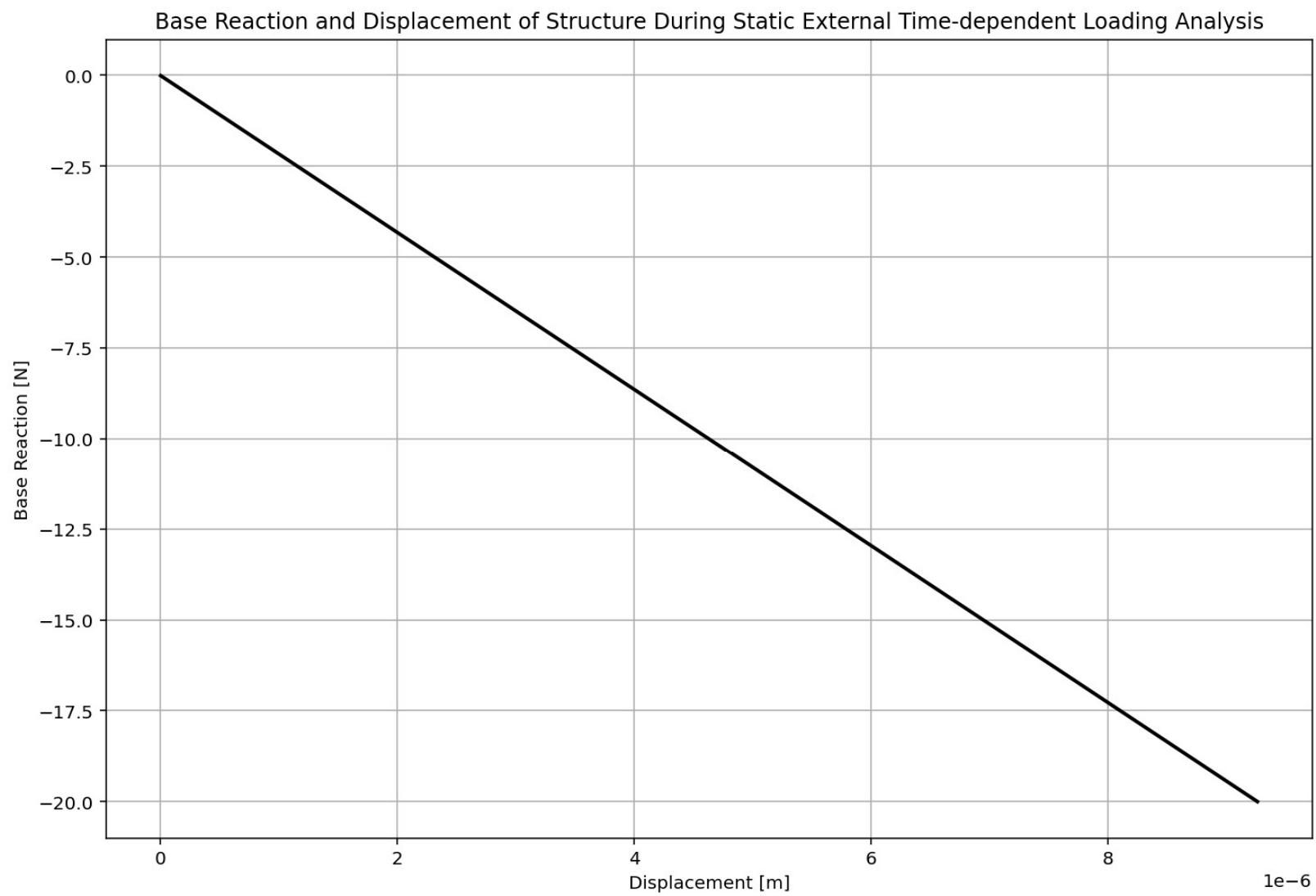
Time (s)

External Loading - Max: 5516.665

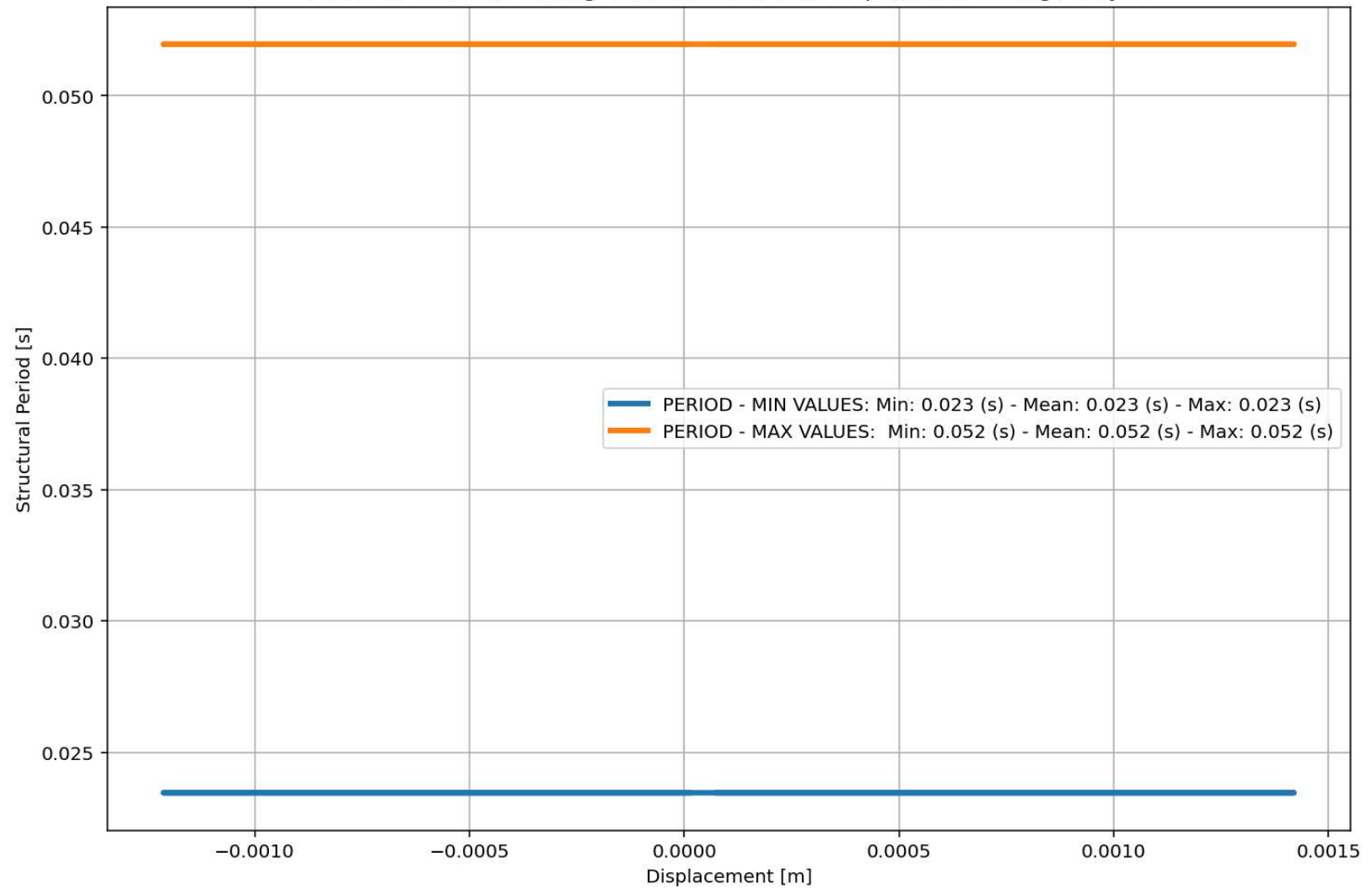
IPython Console Files Help Variable Explorer Debugger Plots History

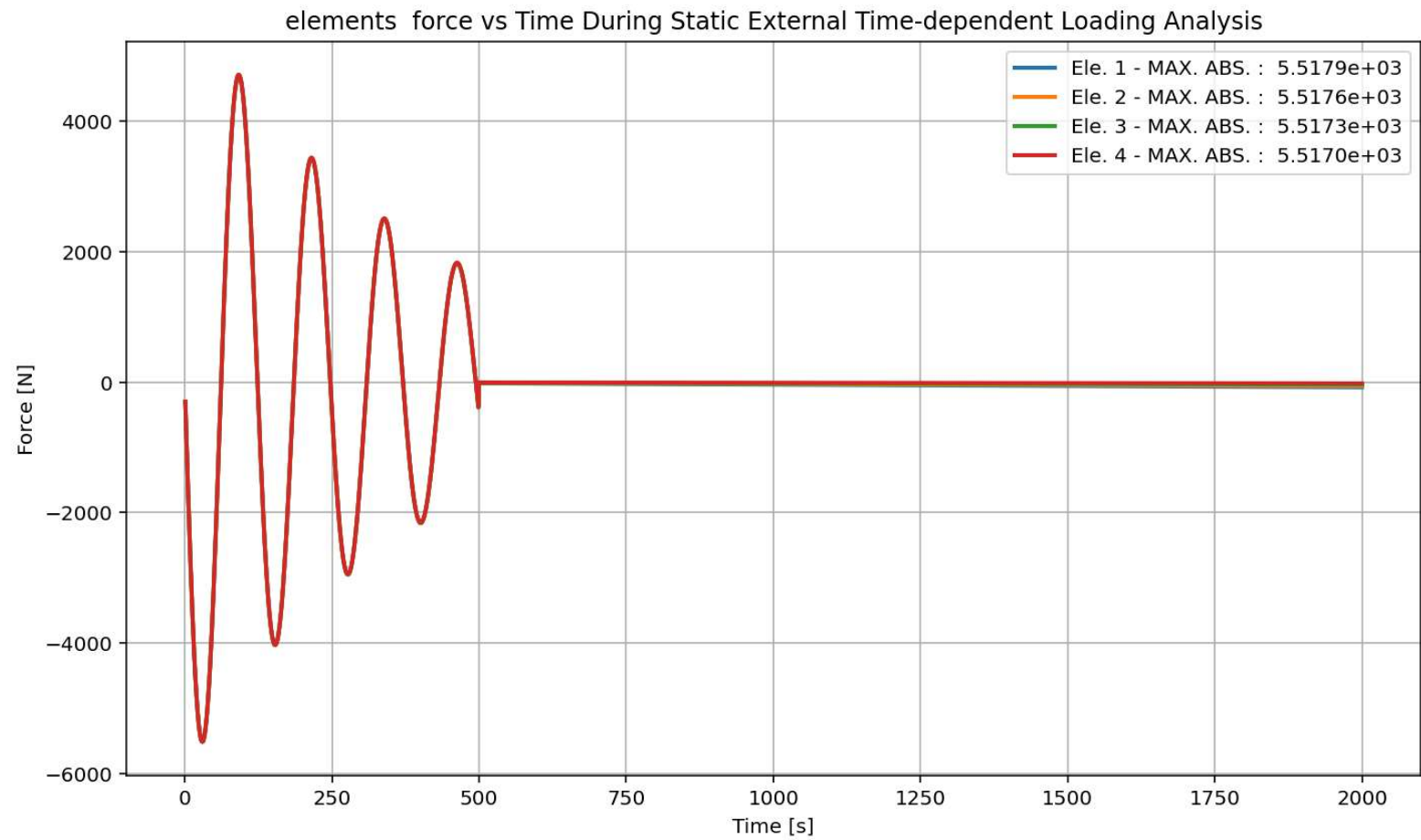
Inline Conda: anaconda3 (Python 3.12.7) LSP: Python Line 1307, Col 23 UTF-8 CRLF RW Mem 45%

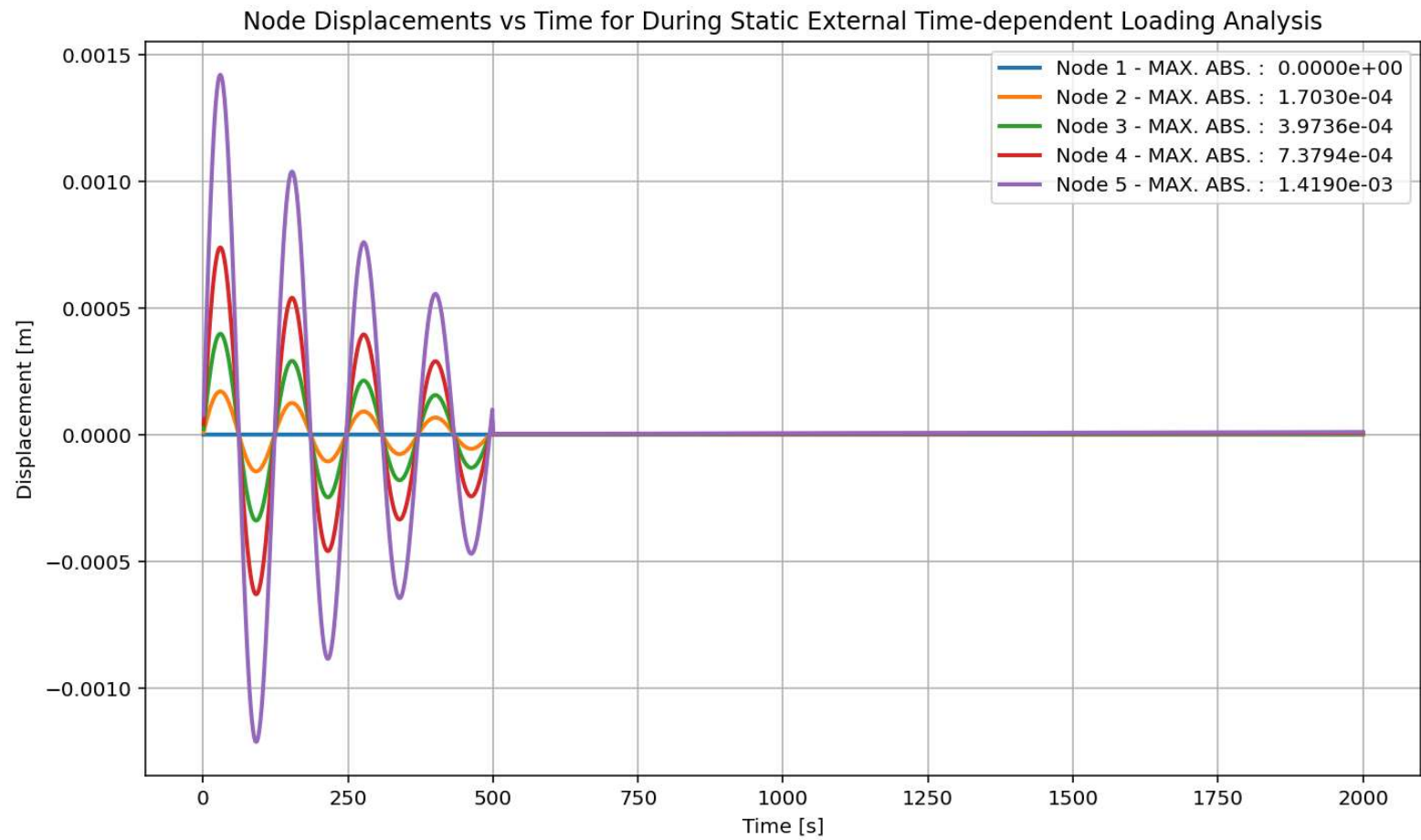


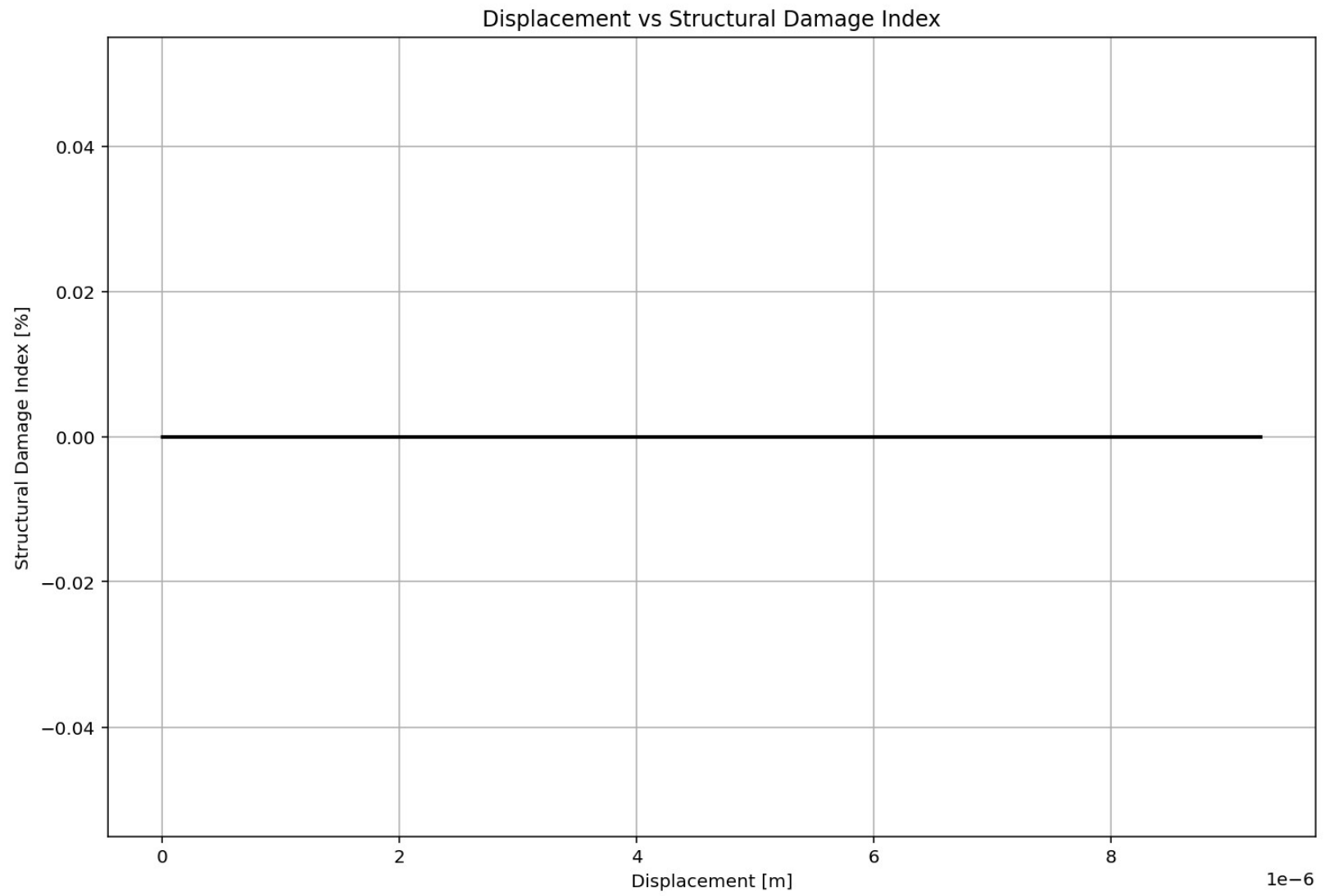


Period of Structure During Static External Time-dependent Loading Analysis









DYNAMIC TIME-HISTORY WITH EXTERNAL TIME-DEPENDENT LOADING ANALYSIS

Spyder (Python 3.12)

File Edit Search Source Run Debug Consoles Projects Tools View Help

C:\Users\Dell\Desktop\OPENSEES_FILES\+EIGHT_ANALY...TY_INDEX\33_MDOF_TWO_8_ANA\P(t)_MDOF_TWO_8_ANA.py

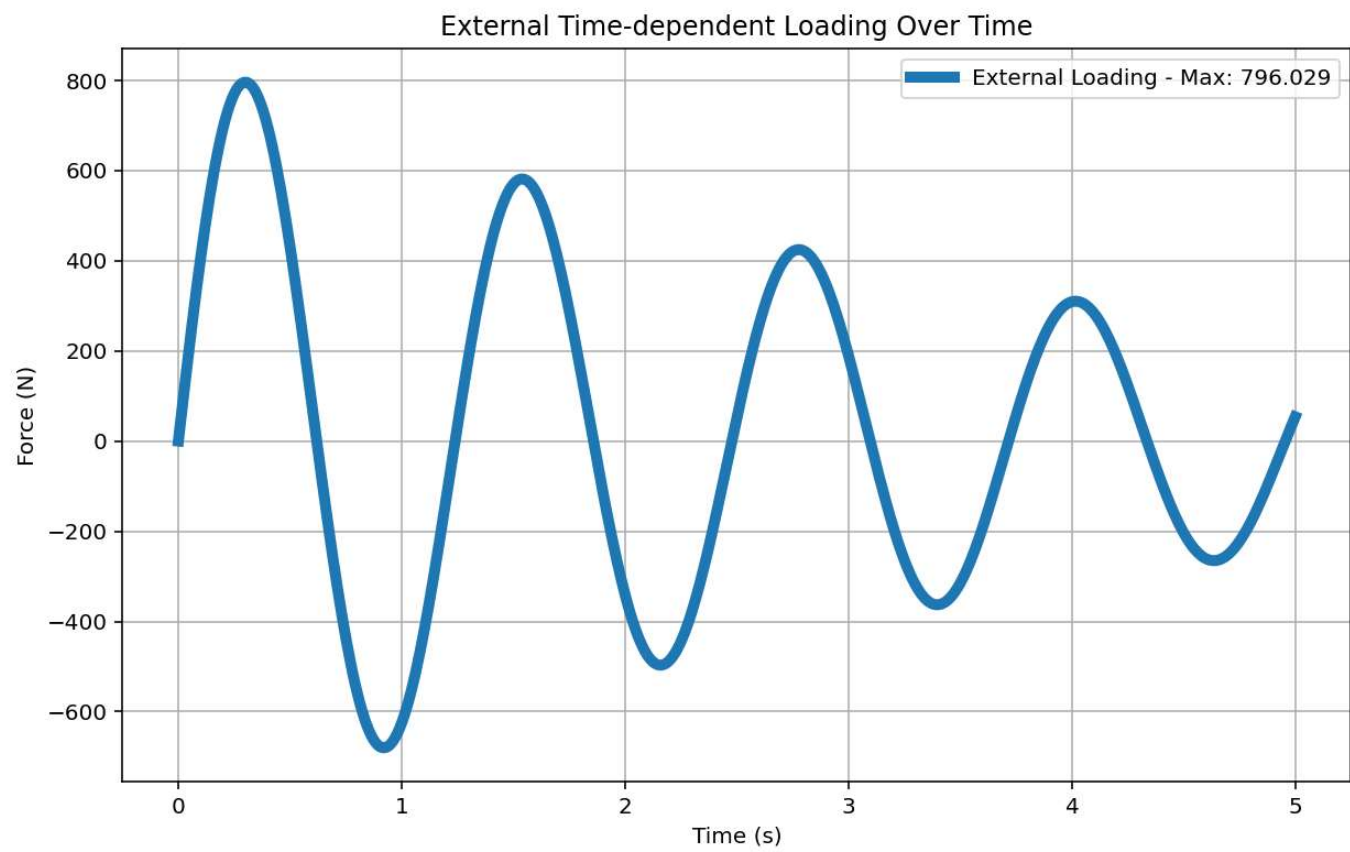
P(t)_MDOF_8_ANA.py P(t)_MDOF_TWO_8_ANA.py

```
491
492 if ANAL_TYPE == 'DYNAMIC EXTERNAL TIME-DEPENDENT LOADING': # DYNAMIC TIME-HISTORY ANAL
493     %% DEFINE EXTERNAL TIME-DEPENDENT LOADING PROPERTIES
494     duration = 20.0           # [s] Analysis duration
495     dt = 0.01                # [s] Time step
496     DT = dt                  # [s] Time step
497     DT_time = 5.0            # [s] Total external Load Analysis Durations [*****]
498     force_amplitude = 5960.0 # [N] Amplitude Force
499     omega_DT = 5.0715        # [rad/s] Natural angular frequency
500     DR = 0.03                # Damping Ratio
501
502     # Check function
503     def CHECK_FUN(DT_time ,duration):
504         if DT_time > duration:
505             print('\n\nAnalysis Duration Must be greater than External Load Duration\n')
506             exit()
507         return -1
508
509     CHECK_FUN(DT_time ,duration)
510     def EXTERNAL_TIME_DEPENDENT(force_amplitude, omega_DT, DT, DT_time): # P(t) = P0 sin
511         import numpy as np
512         import matplotlib.pyplot as plt
513         # External Load Durations
514         num_steps = int(DT_time / DT)
515         load_time = np.linspace(0, DT_time, num_steps)
516         target_frequency = 1.0 * omega_DT # Target excitation frequency
517         DT_load = force_amplitude * np.sin(target_frequency * load_time)
518         # Plot External Time-dependent Loading
519         plt.figure(figsize=(10, 6))
520         plt.plot(load_time, DT_load, label=f'External Loading - Max: {np.max(DT_load)}')
521         plt.title('External Time-dependent Loading Over Time')
522         plt.xlabel('Time (s)')
523         plt.ylabel('Force (N)')
524         plt.grid(True)
```

Base Reaction and Displacement of Structure During Dynamic External Time-dependent Loading Analysis

[Python Console] Files Help Variable Explorer Debugger Plots History

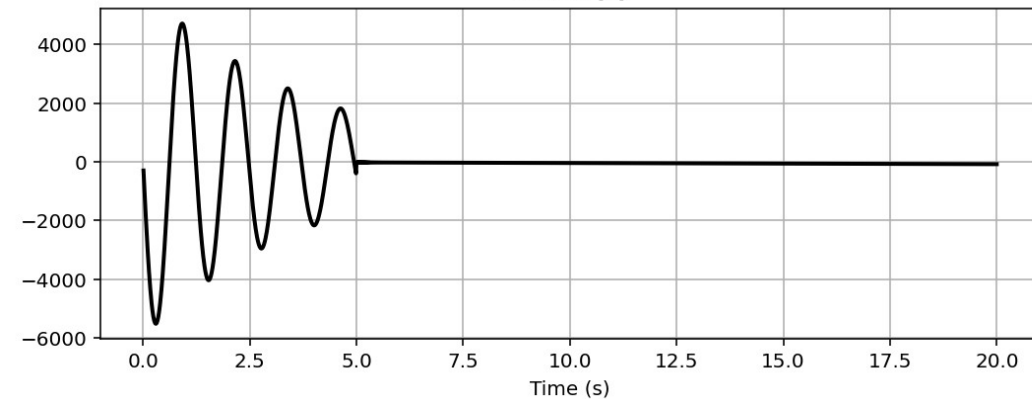
Inline Conda: anaconda3 (Python 3.12.7) ✓ LSP: Python Line 169, Col 37 UTF-8 CRLF RW Mem 44%



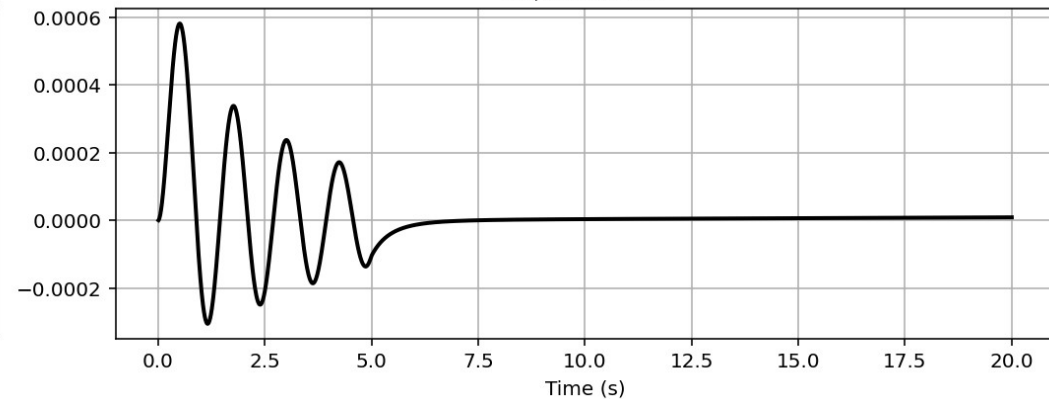
Base Reaction and Displacement of Structure During Dynamic External Time-dependent Loading Analysis



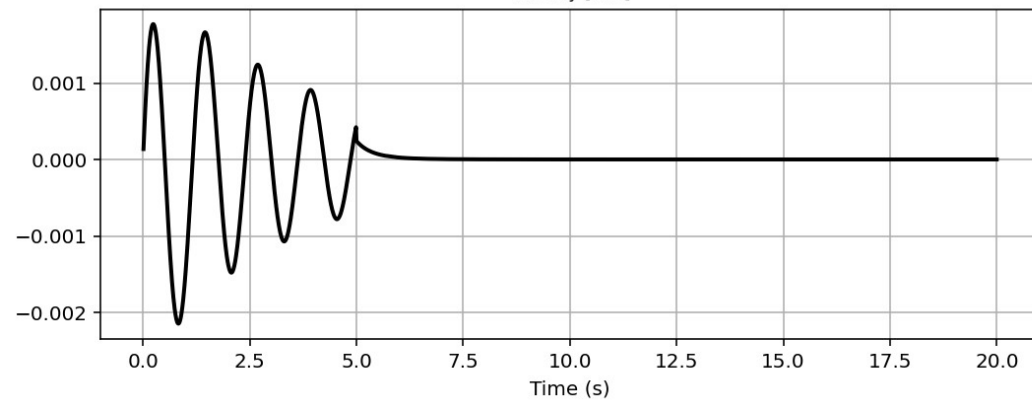
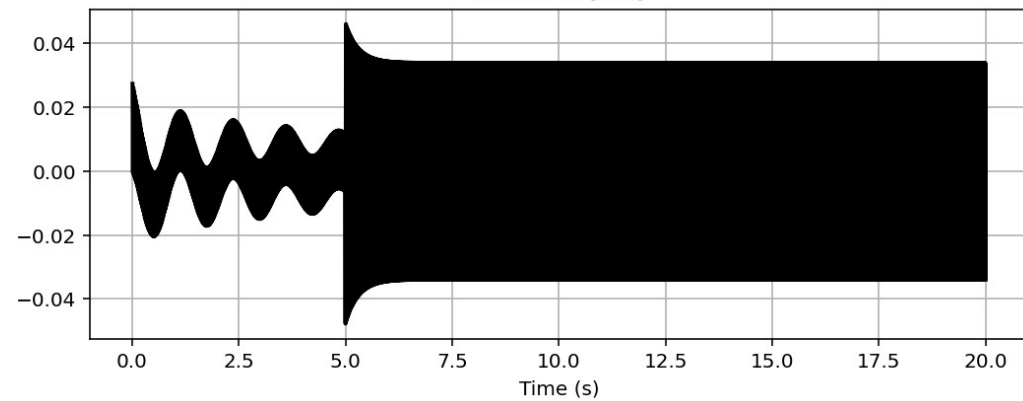
Reaction [N]



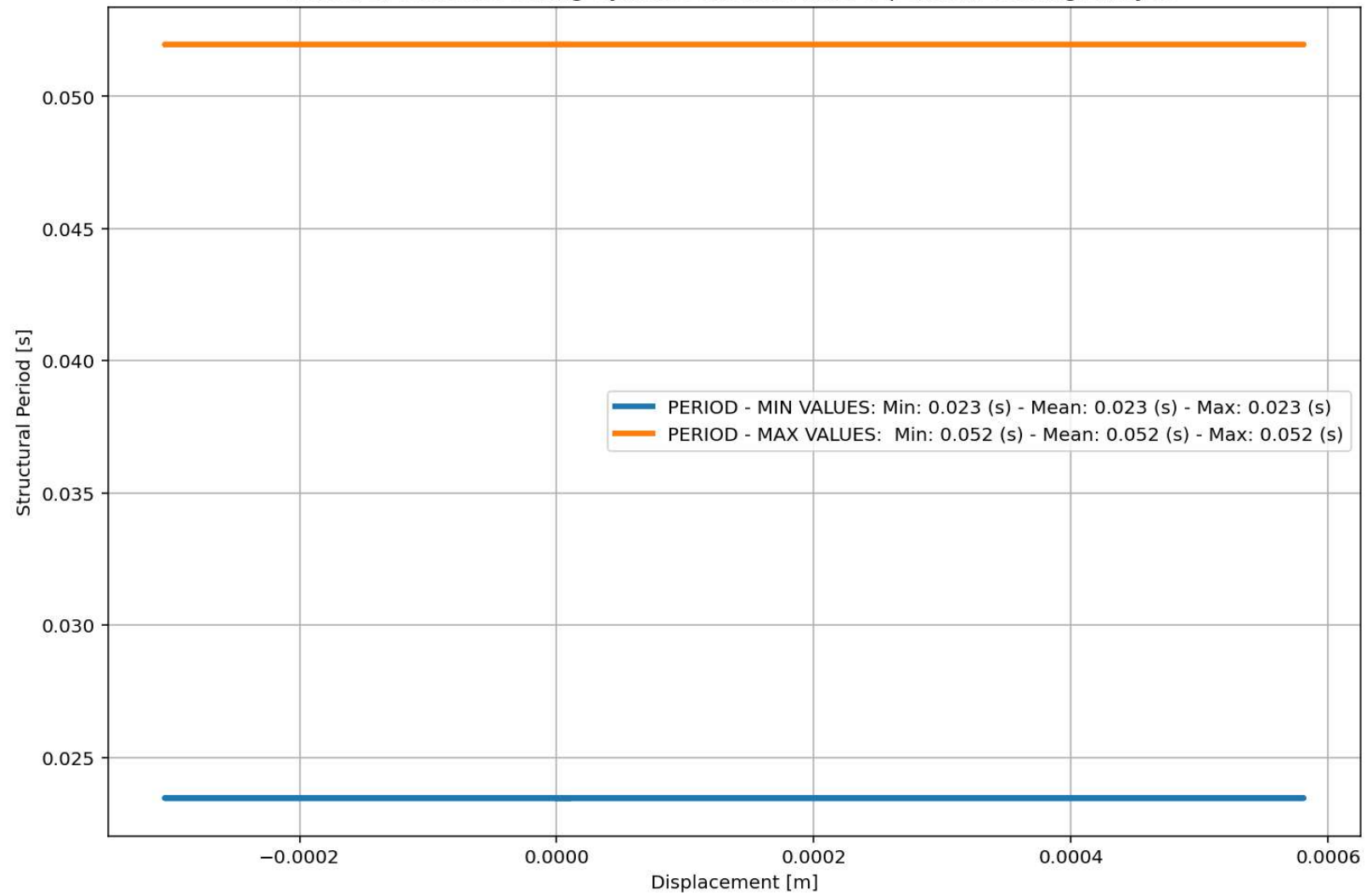
Displacement [m]

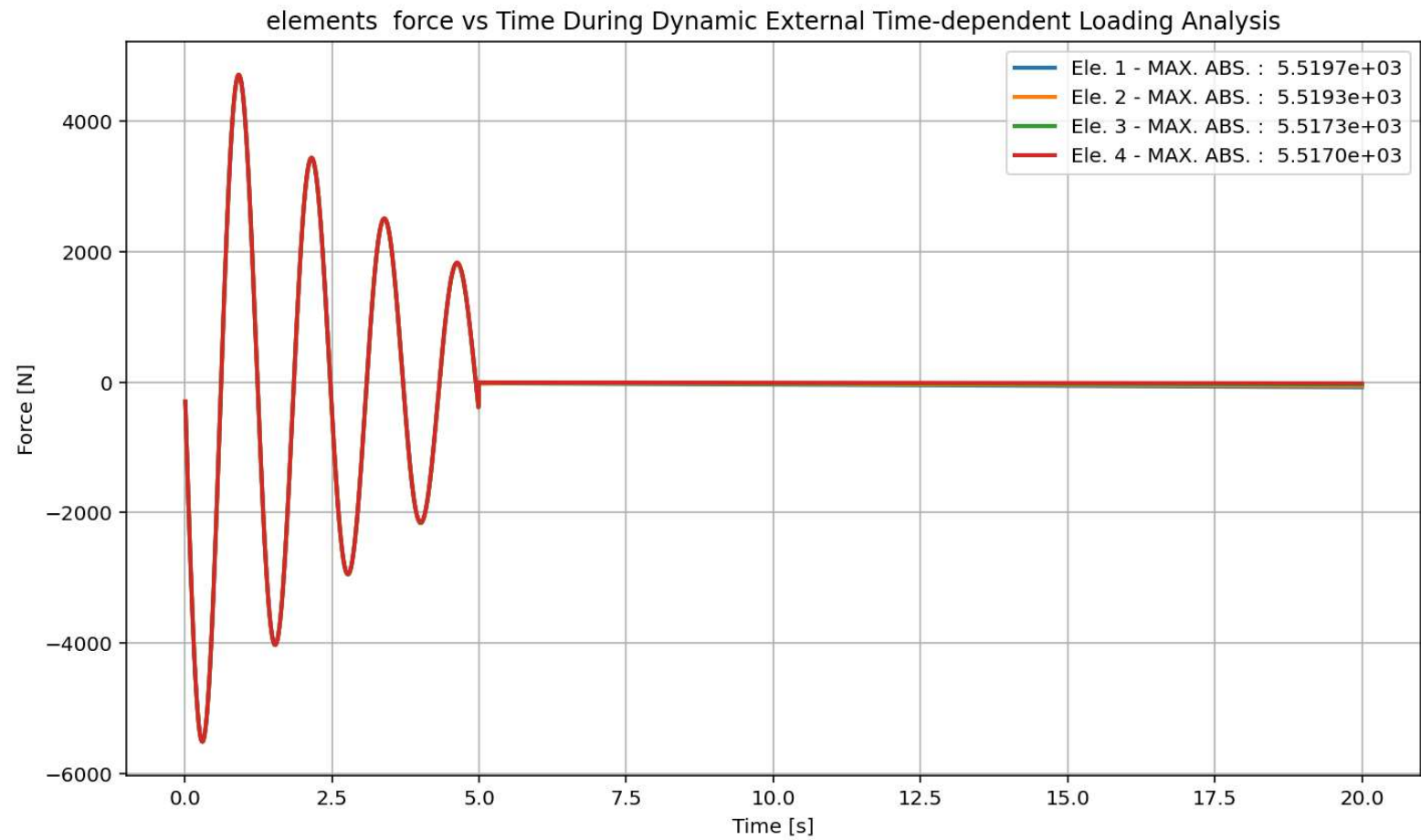


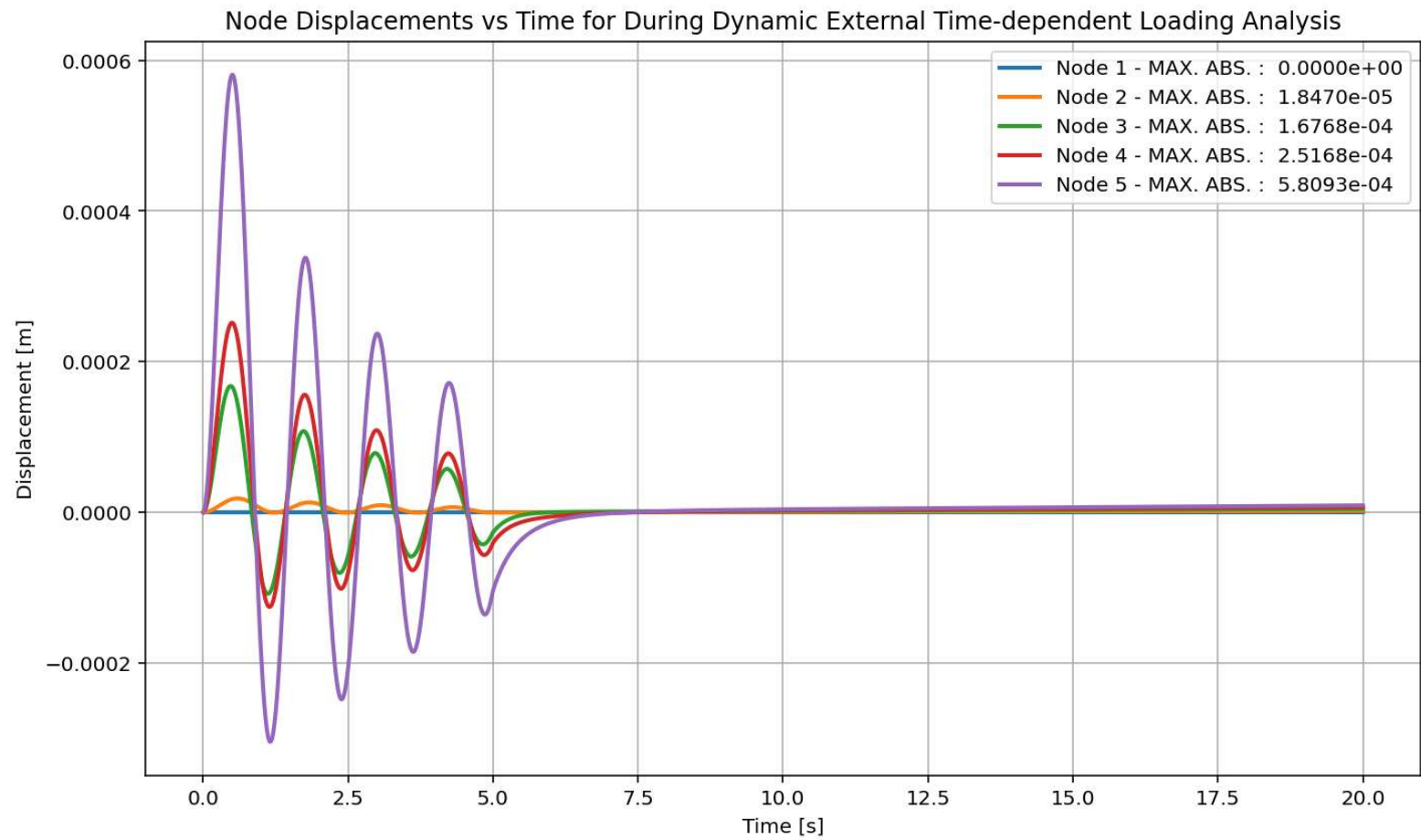
Velocity [m/s]

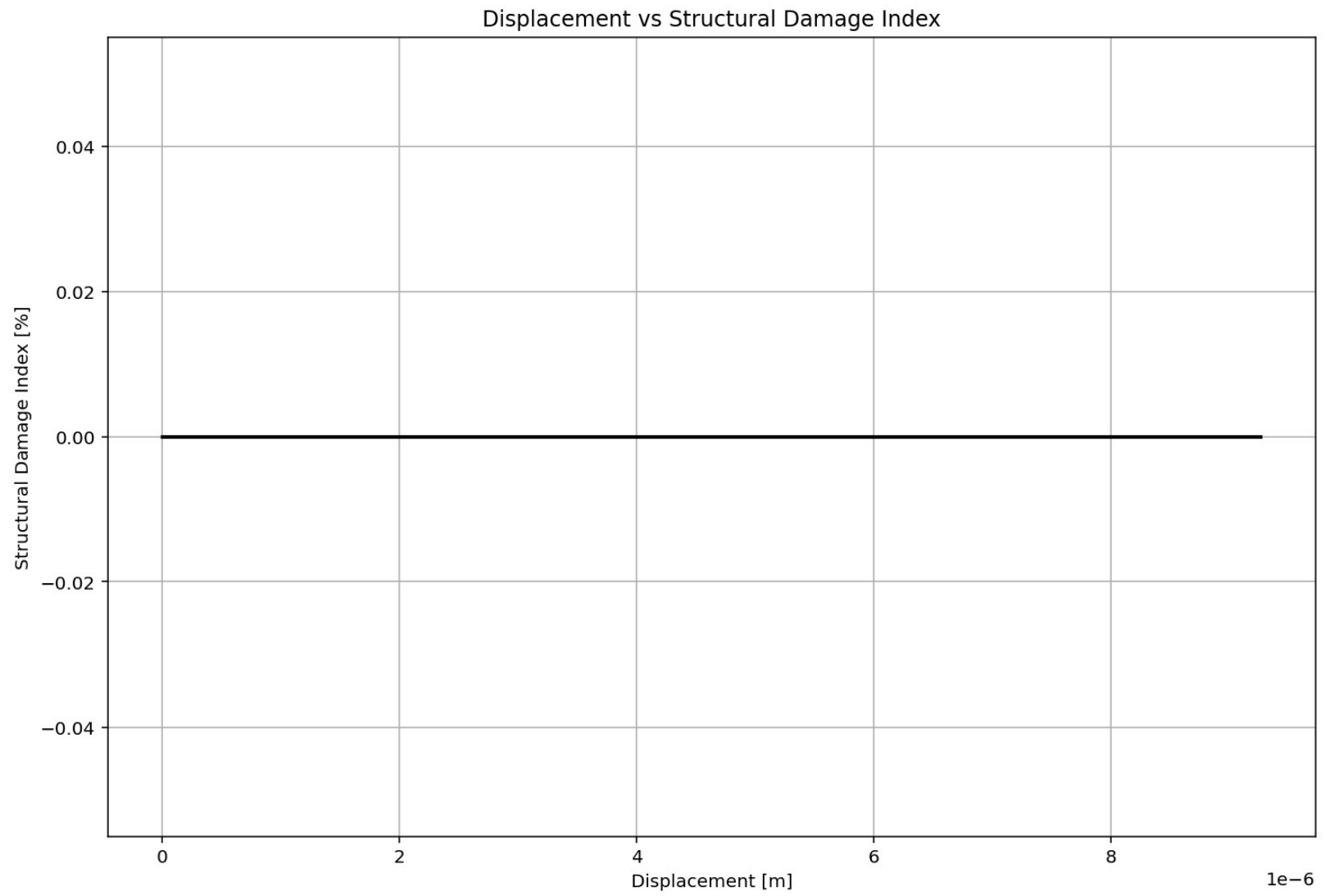
Acceleration [m/s²]

Period of Structure During Dynamic External Time-dependent Loading Analysis









FREE-VIBRATION ANALYSIS

Spyder (Python 3.12)

File Edit Search Source Run Debug Consoles Projects Tools View Help

C:\Users\Dell\Desktop\OPENSEES_FILES\+EIGHT_ANALY...TY_INDEX\33_MDOF_TWO_8_ANA\P(t)_MDOF_TWO_8_ANA.py

P(t)_MDOF_TWO_8_ANA.py COMPUTE_EFFECTIVE_..._FREE_VIBRATION.py

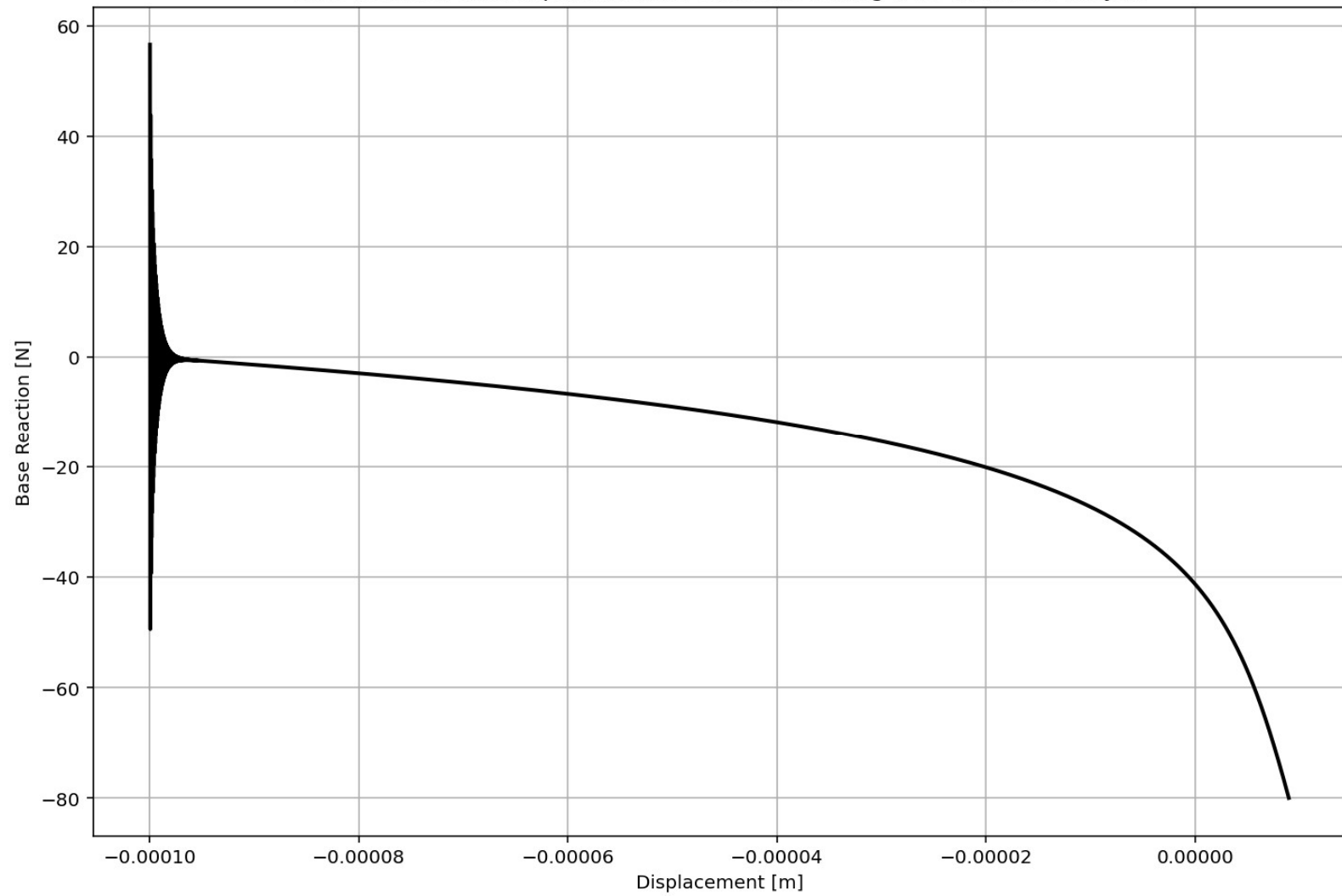
```
620
621 if ANAL_TYPE == 'FREE-VIBRATION': # DYNAMIC TIME-HISTORY ANALYSIS
622     ### DEFINE PARAMETERS FOR FREE-VIBRATION ANALYSIS
623     u0 = -0.00010 # [m] Initial displacement
624     v0 = 0.000015 # [m/s] Initial velocity
625     a0 = 0.000065 # [m/s^2] Initial acceleration
626     IU = True # Free Vibration with Initial Displacement
627     IV = True # Free Vibration with Initial Velocity
628     IA = True # Free Vibration with Initial Acceleration
629     duration = 20.0 # [s] Analysis duration
630     dt = 0.001 # [s] Time step
631     DR = 0.03 # Damping Ratio
632
633     disp_02, disp_03, disp_04, disp_05 = [], [], [], []
634     velo_02, velo_03, velo_04, velo_05 = [], [], [], []
635     accel_02, accel_03, accel_04, accel_05 = [], [], [], []
636
637     if IU == True:
638         # Define initial displacment
639         ops.setNodeDisp(2, 1, u0, '-commit')
640         ops.setNodeDisp(3, 1, u0, '-commit')
641         ops.setNodeDisp(4, 1, u0, '-commit')
642         ops.setNodeDisp(center_node, 1, u0, '-commit')
643         # INFO LINK: https://openseespydoc.readthedocs.io/en/latest/src/setNodeDisp.htm
644     if IV == True:
645         # Define initial velocity
646         ops.setNodeVel(2, 1, v0, '-commit')
647         ops.setNodeVel(3, 1, v0, '-commit')
648         ops.setNodeVel(4, 1, v0, '-commit')
649         ops.setNodeVel(center_node, 1, v0, '-commit')
650         # INFO LINK: https://openseespydoc.readthedocs.io/en/stable/src/setNodeVel.htm
651     if IA == True:
652         # Define initial acceleration
653         ops.setNodeAccel(2, 1, a0, '-commit')
```

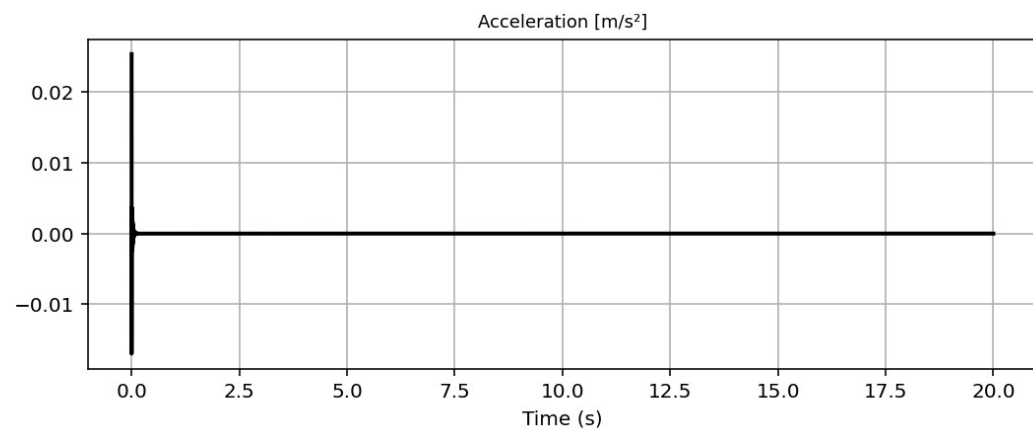
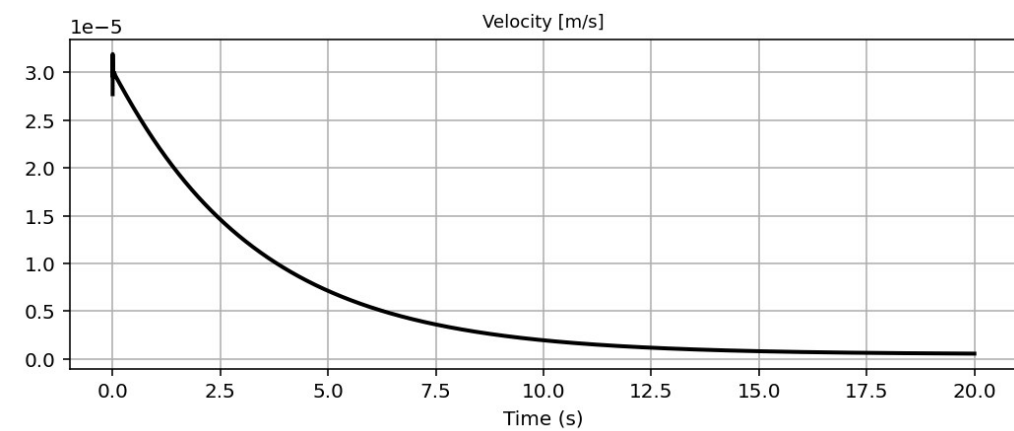
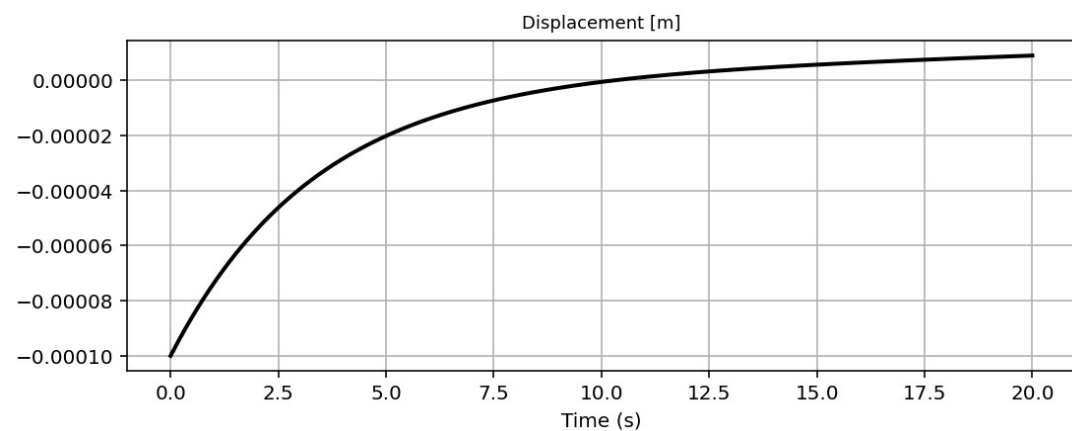
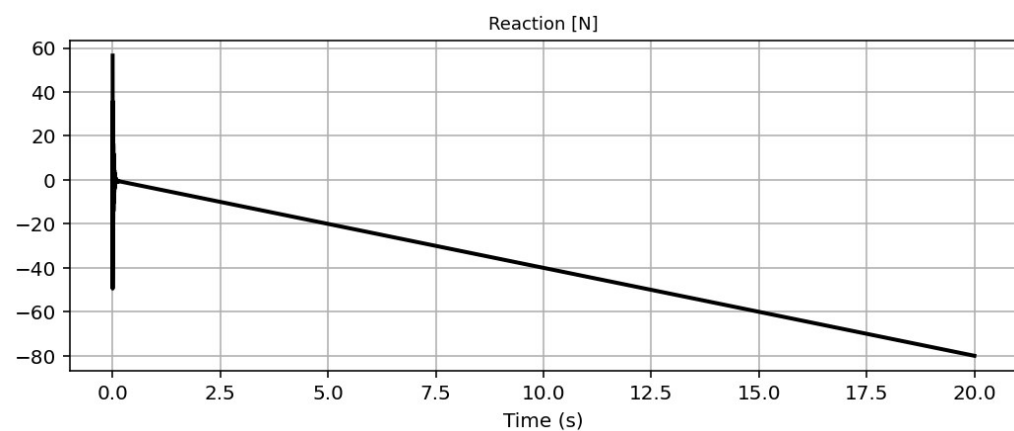
Base Reaction and Displacement of Structure During free-vibration Analysis

IPython Console Files Help Variable Explorer Debugger Plots History

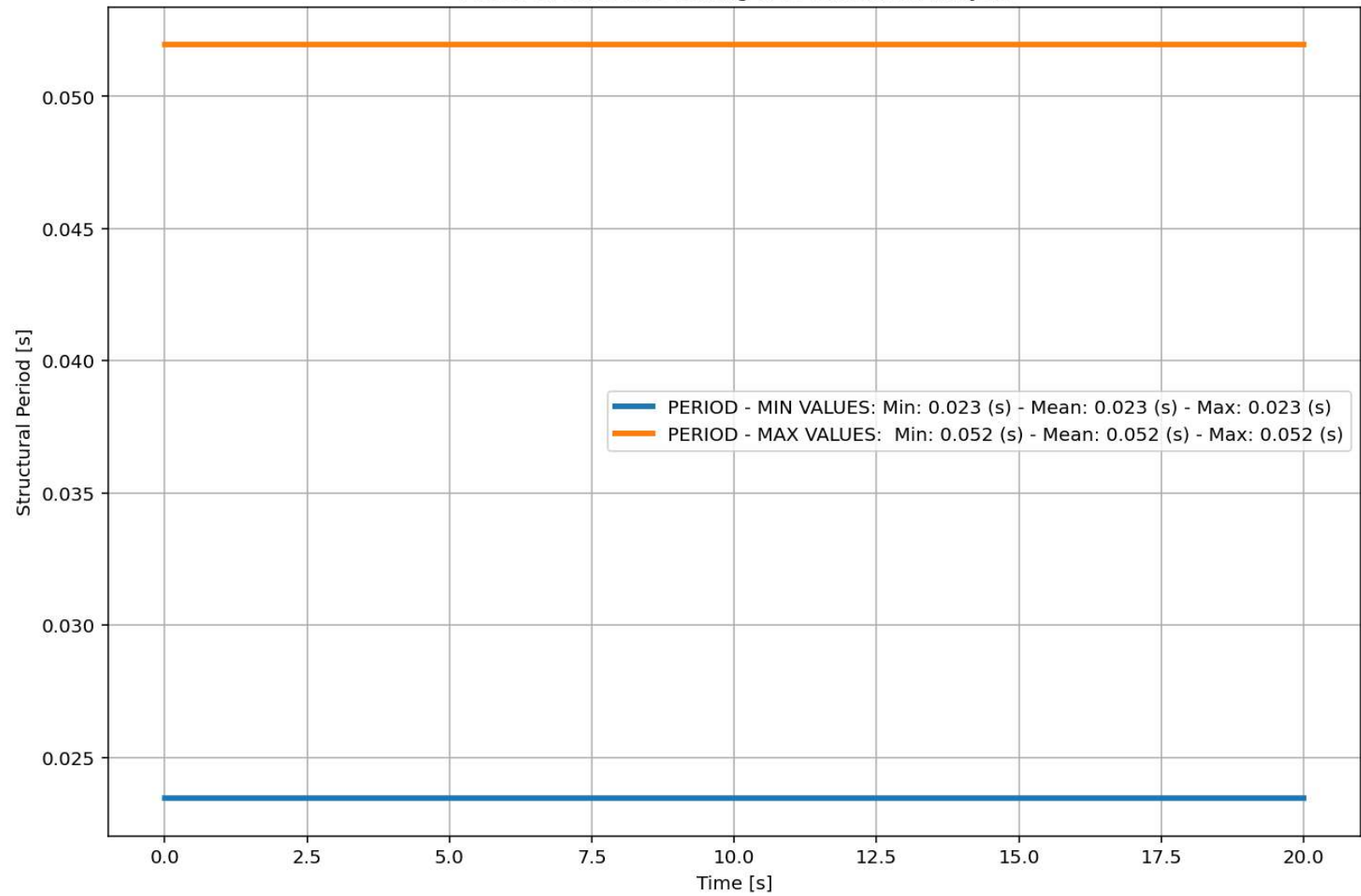
Inline Conda: anaconda3 (Python 3.12.7) ✓ LSP: Python Line 1, Col 1 UTF-8 CRLF RW Mem 48%

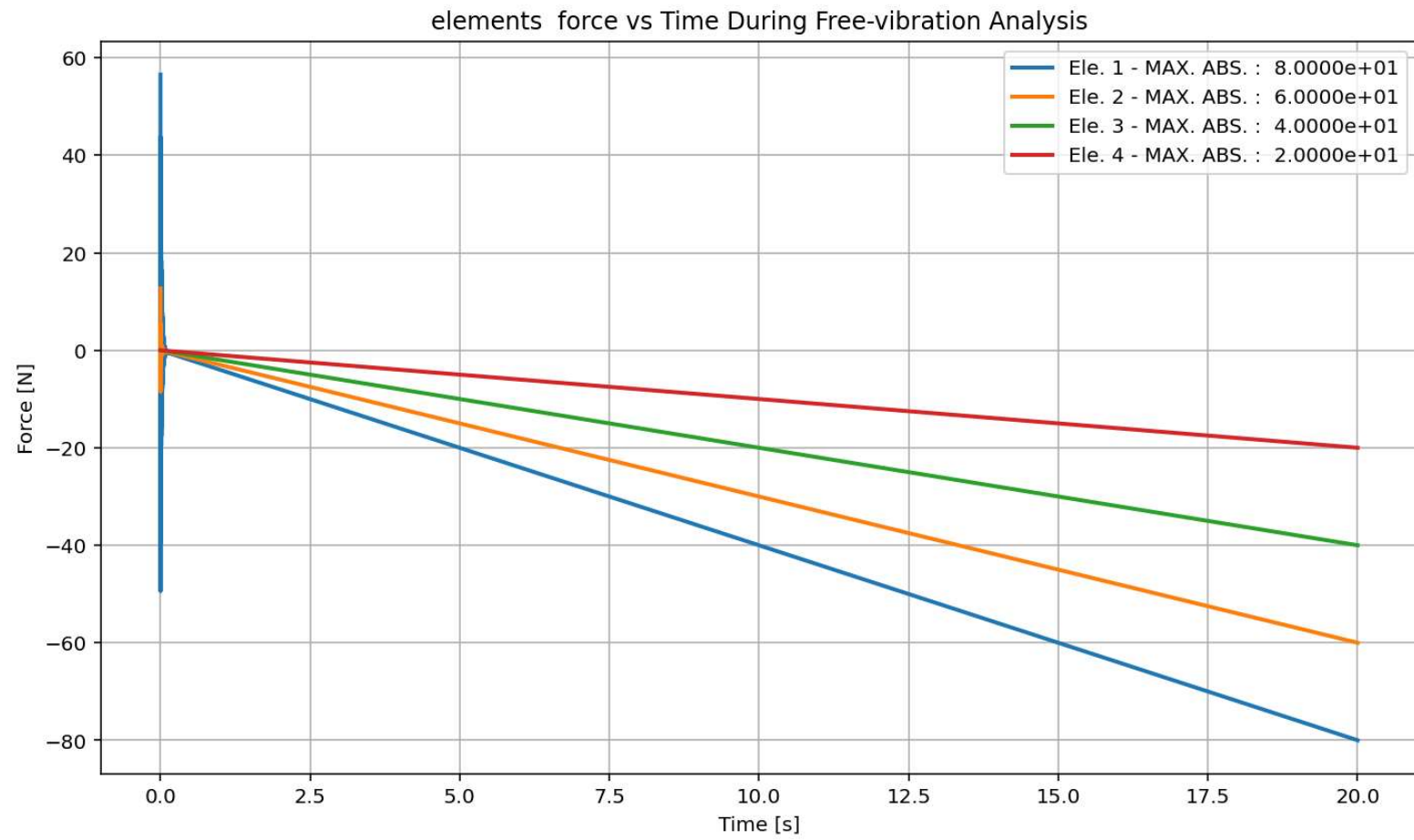
Base Reaction and Displcement of Structure During Free-vibration Analysis



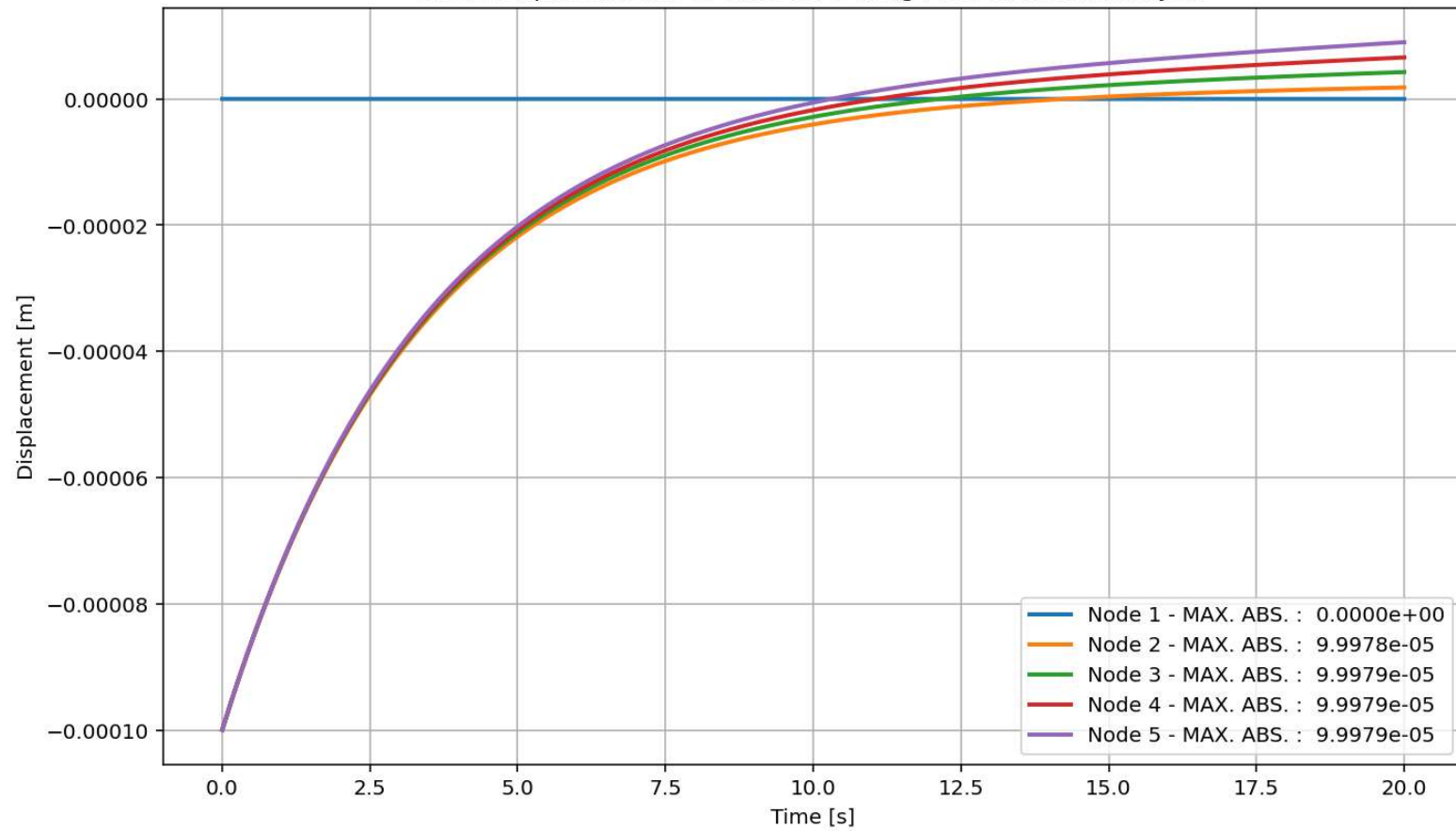


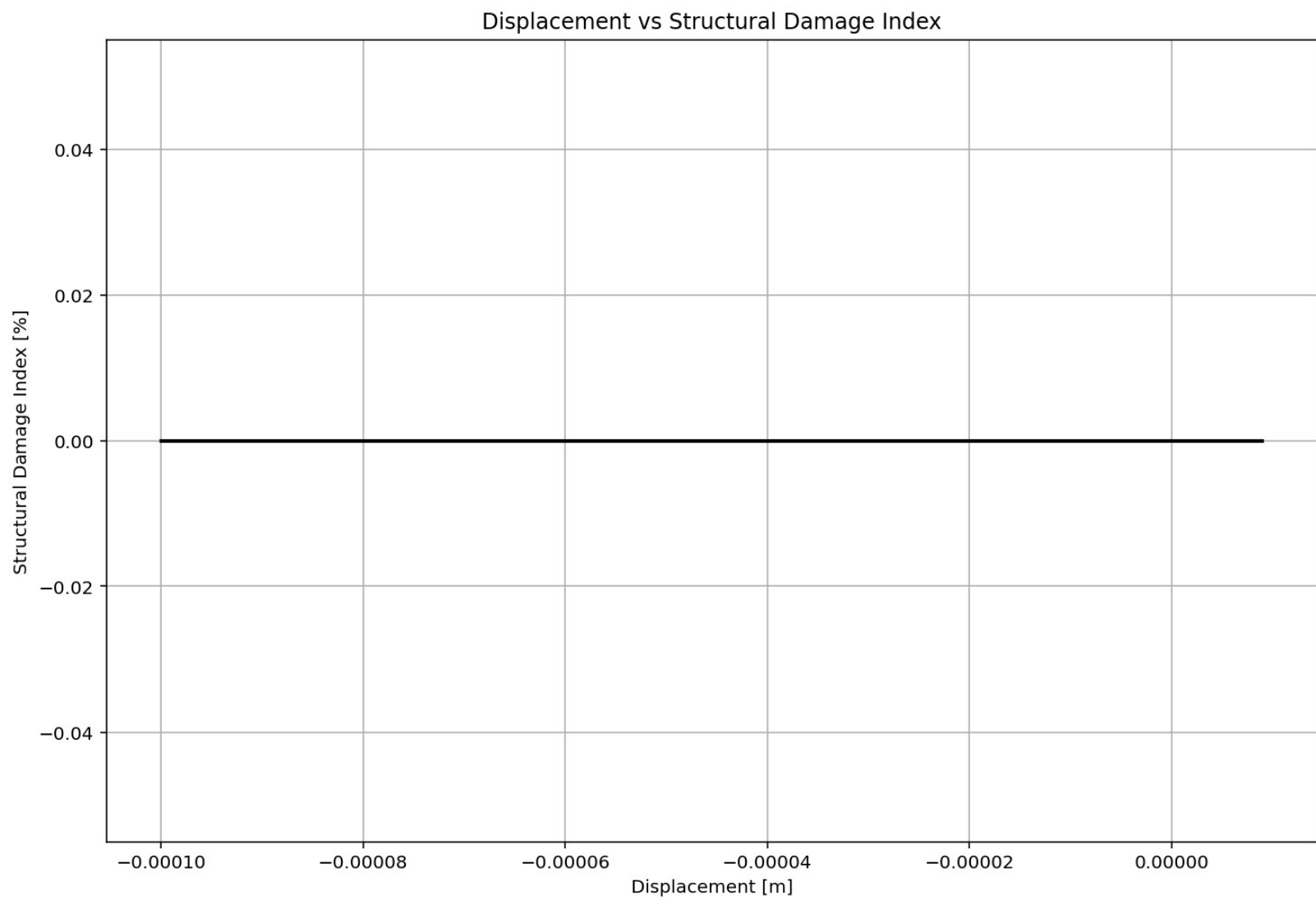
Period of Structure During Free-vibration Analysis



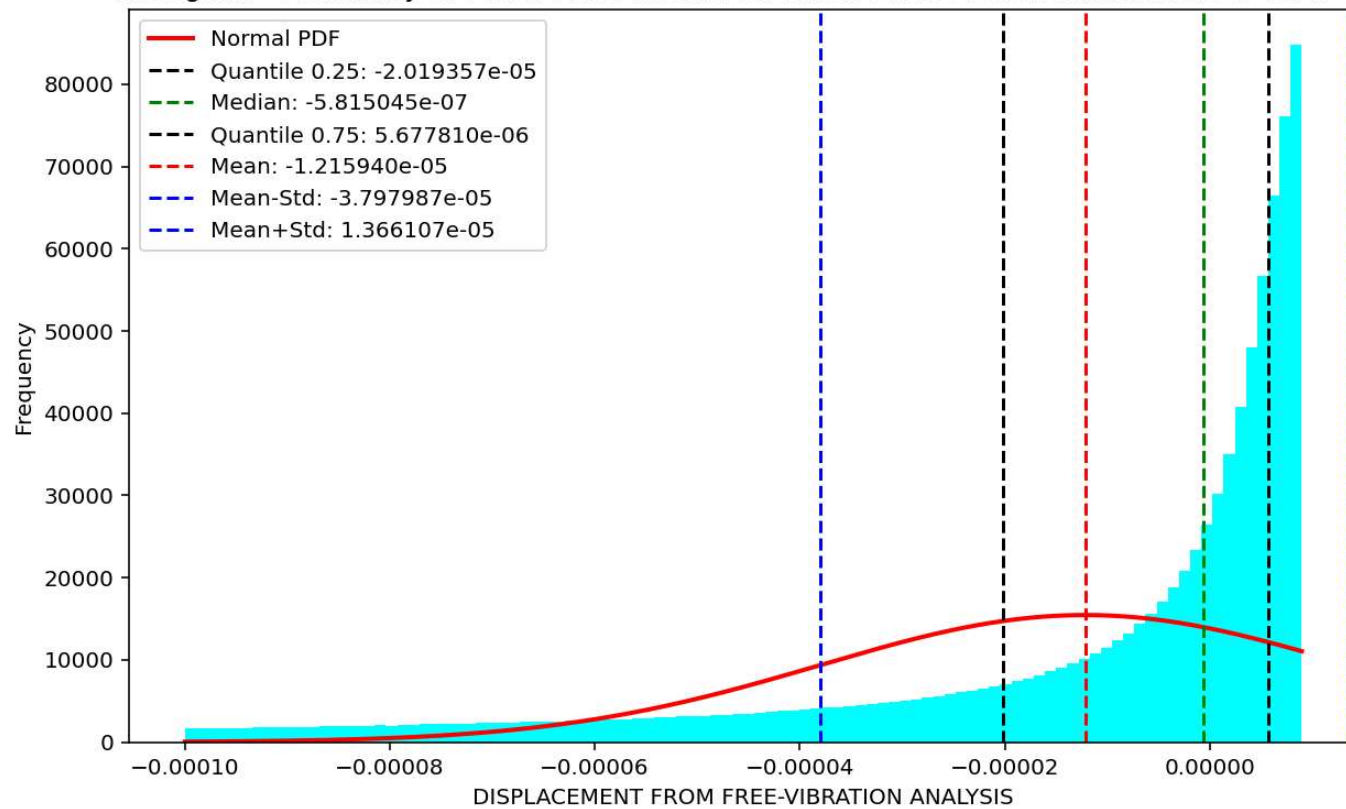


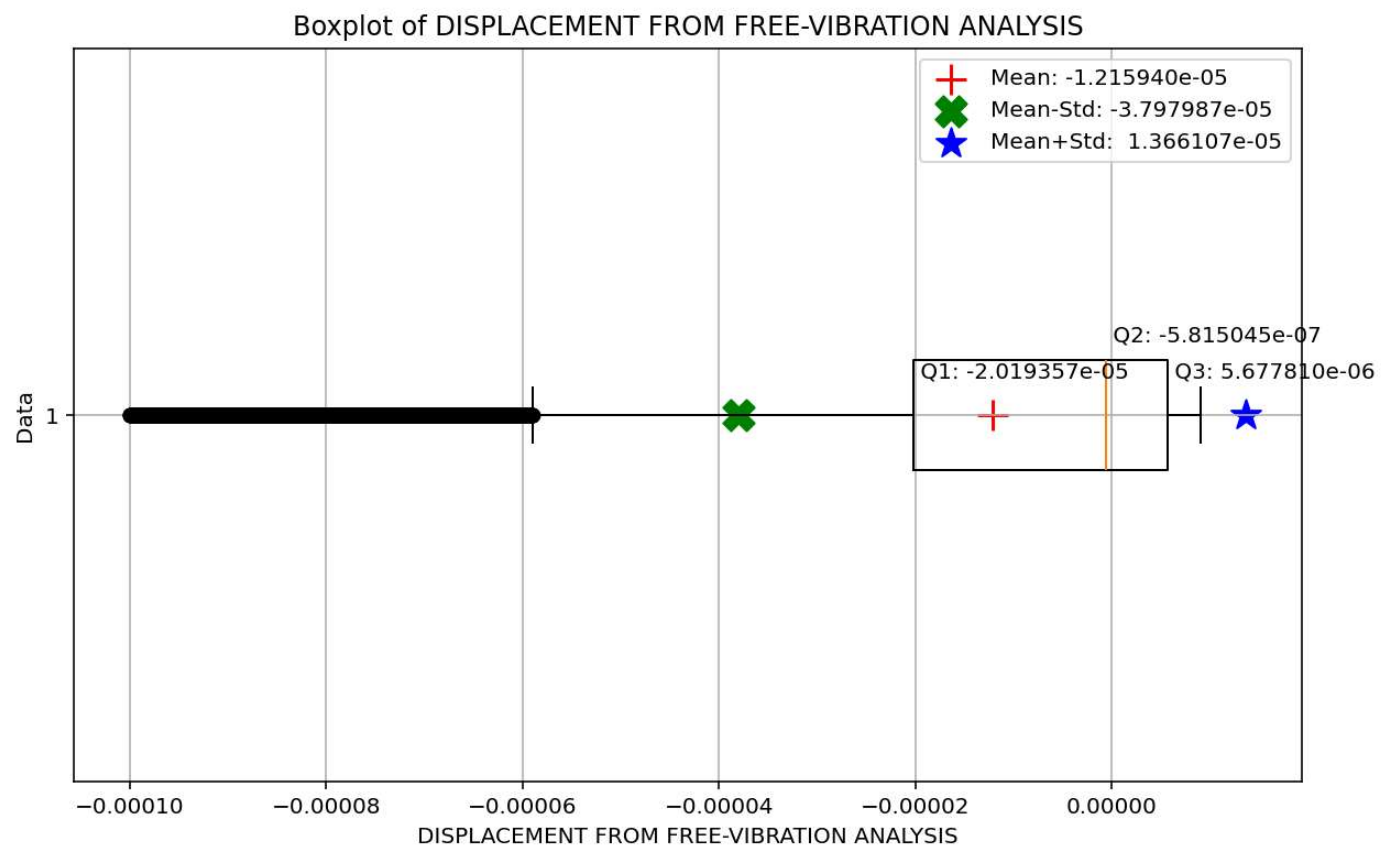
Node Displacements vs Time for During Free-vibration Analysis

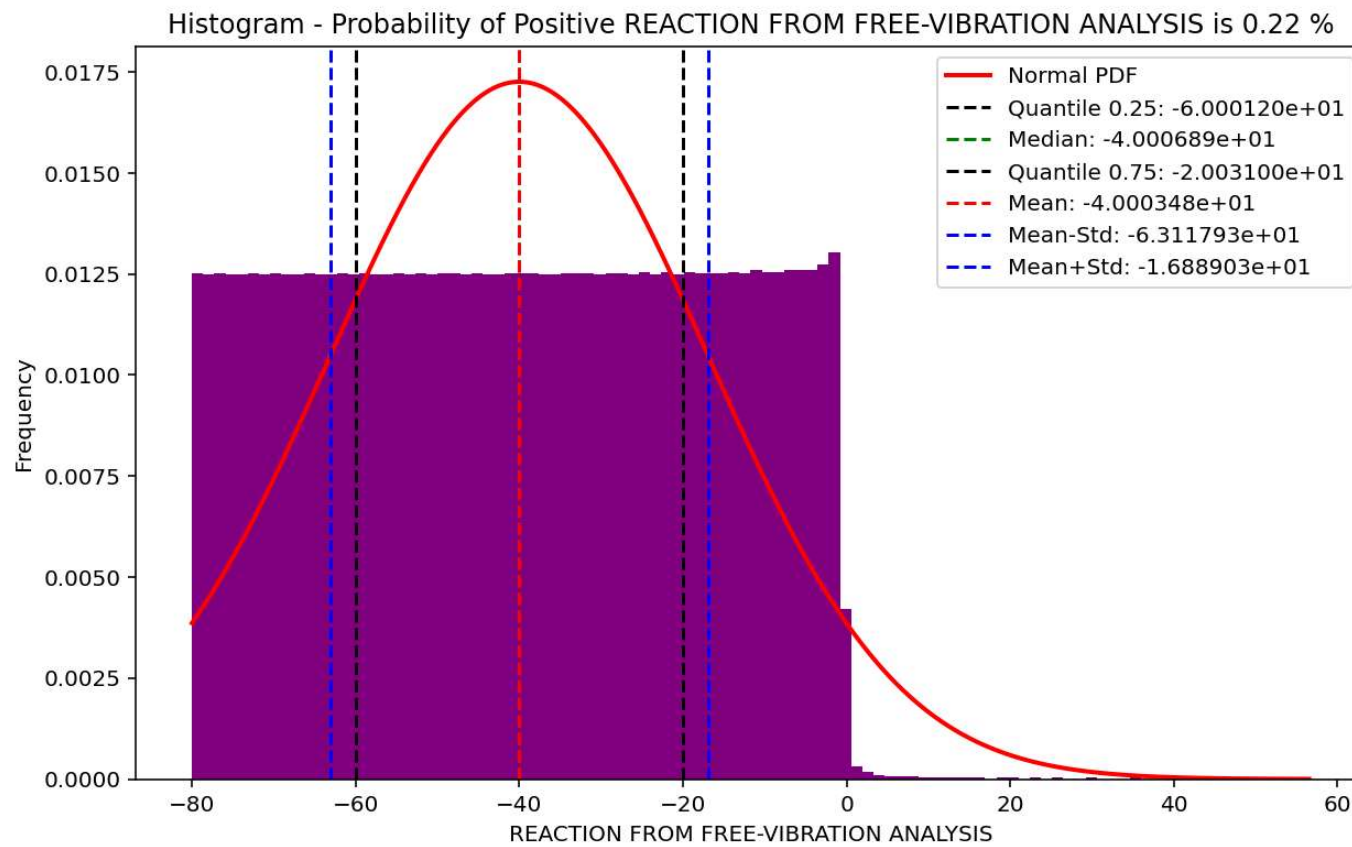




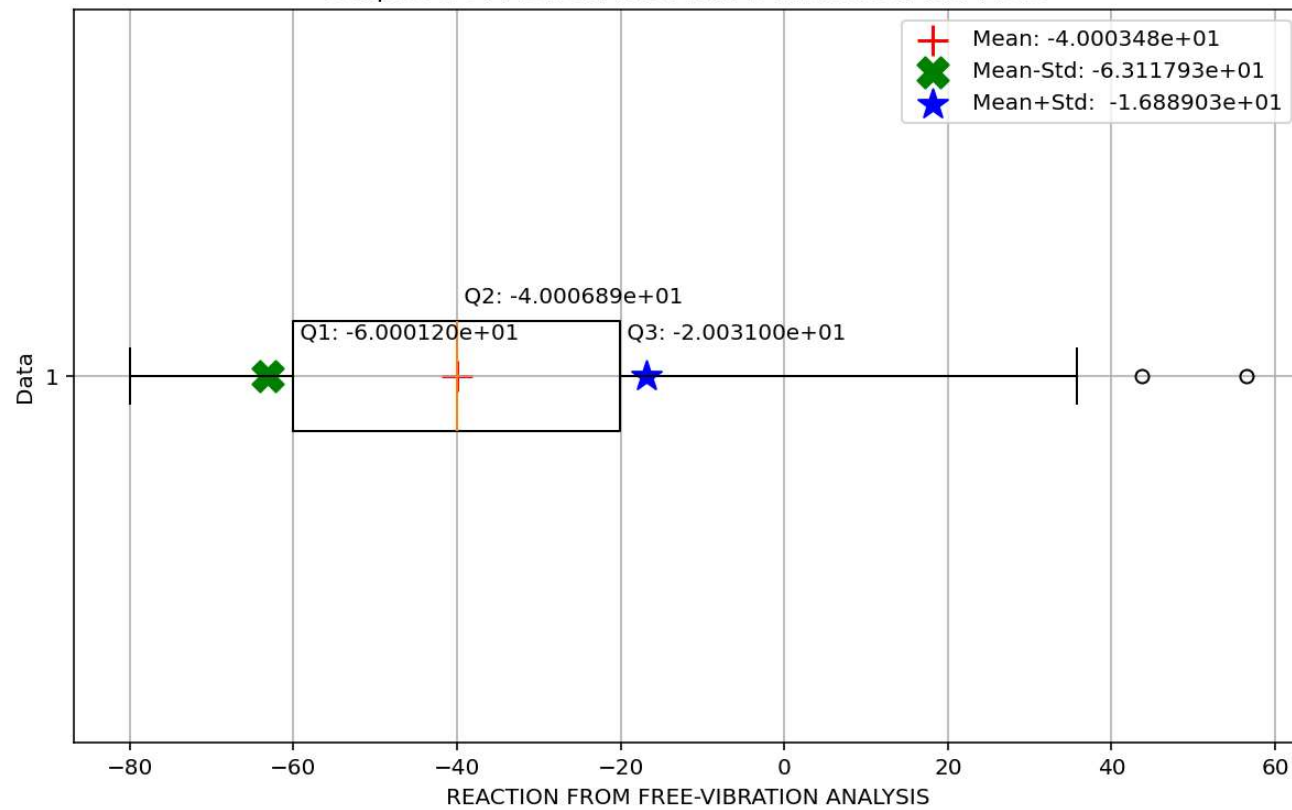
Histogram - Probability of Positive DISPLACEMENT FROM FREE-VIBRATION ANALYSIS is 48.47 %



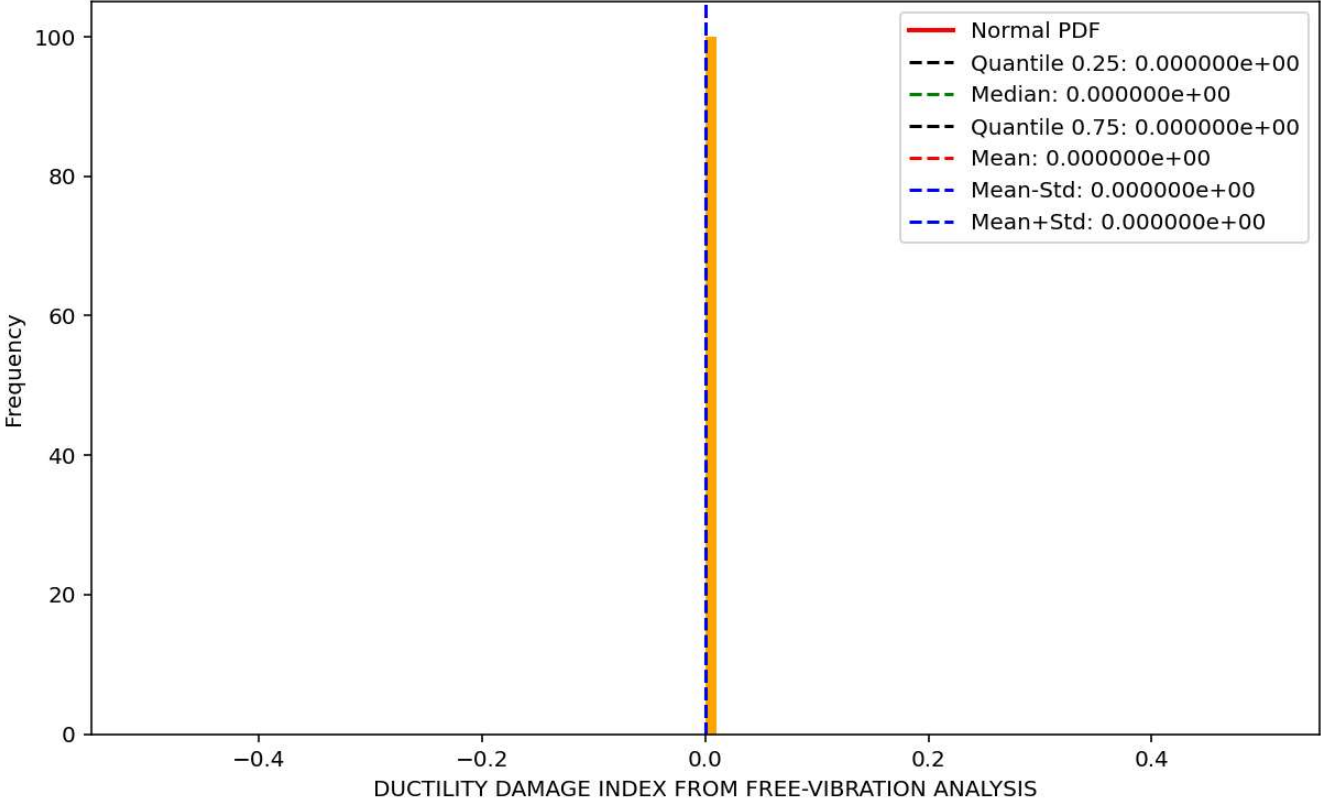


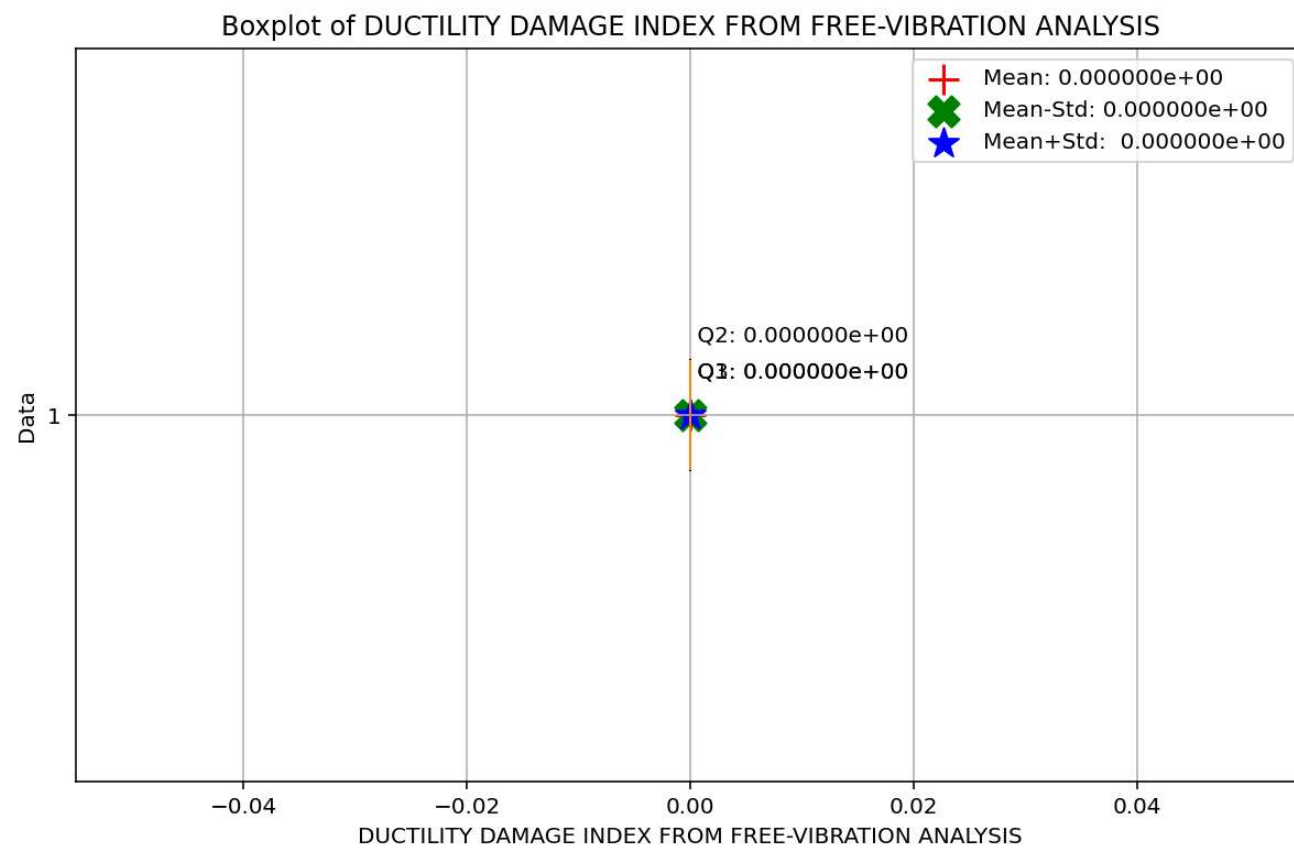


Boxplot of REACTION FROM FREE-VIBRATION ANALYSIS

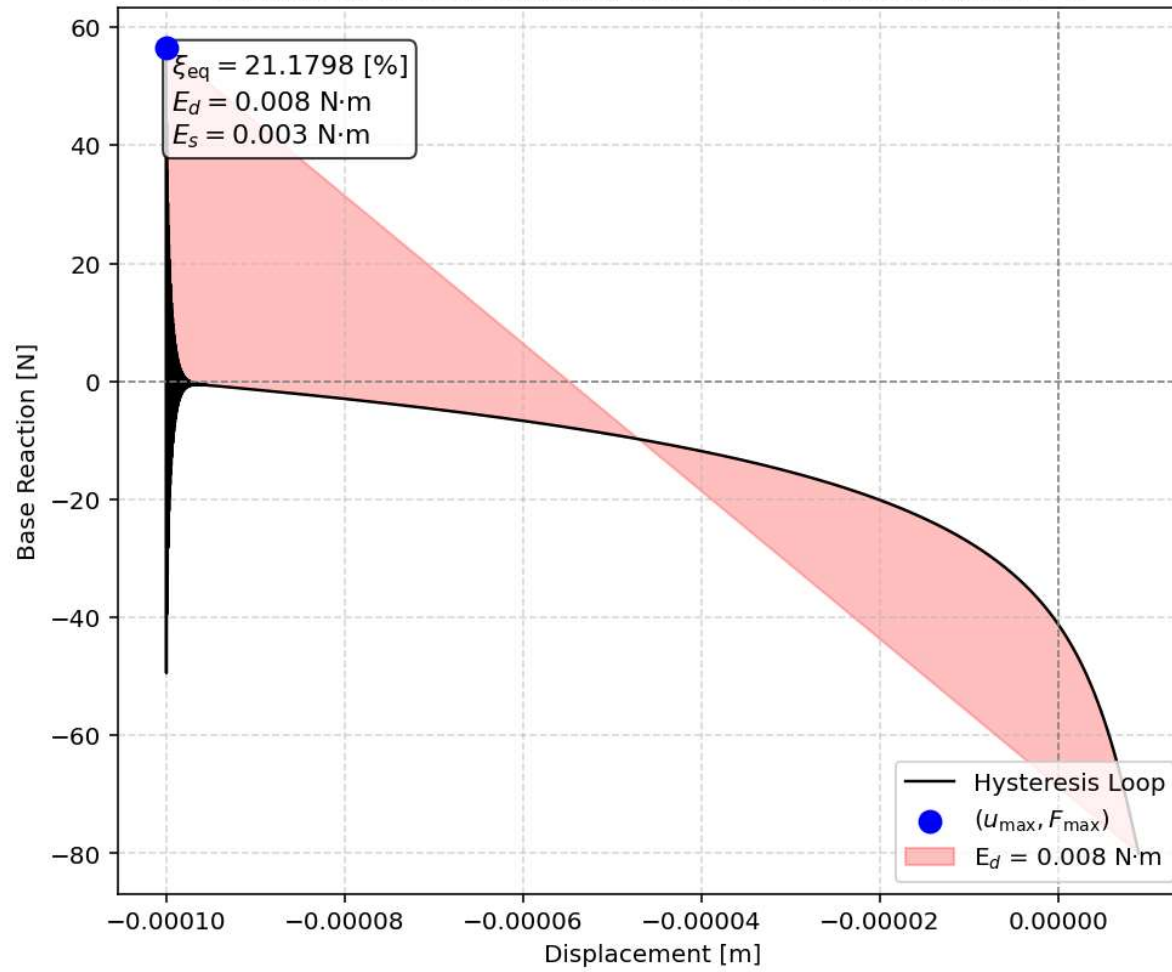


Histogram - Probability of Positive DUCTILITY DAMAGE INDEX FROM FREE-VIBRATION ANALYSIS is 0.00 %





Equivalent viscous damping ratio - Free-vibration Hysteresis



Spyder (Python 3.12)

File Edit Search Source Run Debug Consoles Projects Tools View Help

C:\Users\Dell\Desktop\OPENSEES_FILES\+EIGHT_ANALY...OMPUTE_EFFECTIVE_PROPERTIES_FUN_FREE_VIBRATION.py

P(t)_MDOF_TWO_8_ANA.py X COMPUTE_EFFECTIVE_..._FREE_VIBRATION.py X

```
1  %%% EQUIVALENT SDOF SYSTEM DERIVATION VIA DISPLACEMENT-BASED SEISMIC DESIGN PROCEDURE WITH
2  # Change MDOF to SDOF System with Displacement Based Design Concept
3  # THIS PROGRAM WRITTEN BY SALAR DELAVAR GHASHGHAEI (QASHQAI)
4  %%
5  This script implements a displacement-based pushover transformation,
6  converting a multi-degree-of-freedom (MDOF) system into an equivalent
7  single-degree-of-freedom (SDOF) system for seismic assessment.
8  It calculates effective modal properties-displacement, mass, and
9  stiffness-by weighting element forces and nodal displacements according
10 to a presumed deformed shape.
11 The derived effective period provides a simplified dynamic characteristic
12 for performance-based engineering. The visualizations effectively track the
13 evolution of these equivalent parameters throughout the nonlinear analysis steps.
14 %%
15 #-----
16 import numpy as np
17
18 displacement_X_1 = np.array(list(node_displacements.values())[1]) # DOF 02
19 displacement_X_2 = np.array(list(node_displacements.values())[2]) # DOF 03
20 displacement_X_3 = np.array(list(node_displacements.values())[3]) # DOF 04
21 displacement_X_4 = np.array(list(node_displacements.values())[4]) # DOF 05
22
23 ele_force_01 = np.array(list(ele_force.values())[0]) # ELEMENT 01
24 ele_force_02 = np.array(list(ele_force.values())[1]) # ELEMENT 02
25 ele_force_03 = np.array(list(ele_force.values())[2]) # ELEMENT 03
26 ele_force_04 = np.array(list(ele_force.values())[3]) # ELEMENT 04
27
28 STIFF_X_1 = np.abs(ele_force_01 / displacement_X_1)
29 STIFF_X_2 = np.abs(ele_force_02 / displacement_X_2)
30 STIFF_X_3 = np.abs(ele_force_03 / displacement_X_3)
31 STIFF_X_4 = np.abs(ele_force_04 / displacement_X_4)
32
33 MX2 = (MASS[0] * np.square(displacement_X_1) +
34        MASS[1] * np.square(displacement_X_2) +
```

Effective Displacement - Median: 0.00003

Effective Mass - Median: 4057.01718

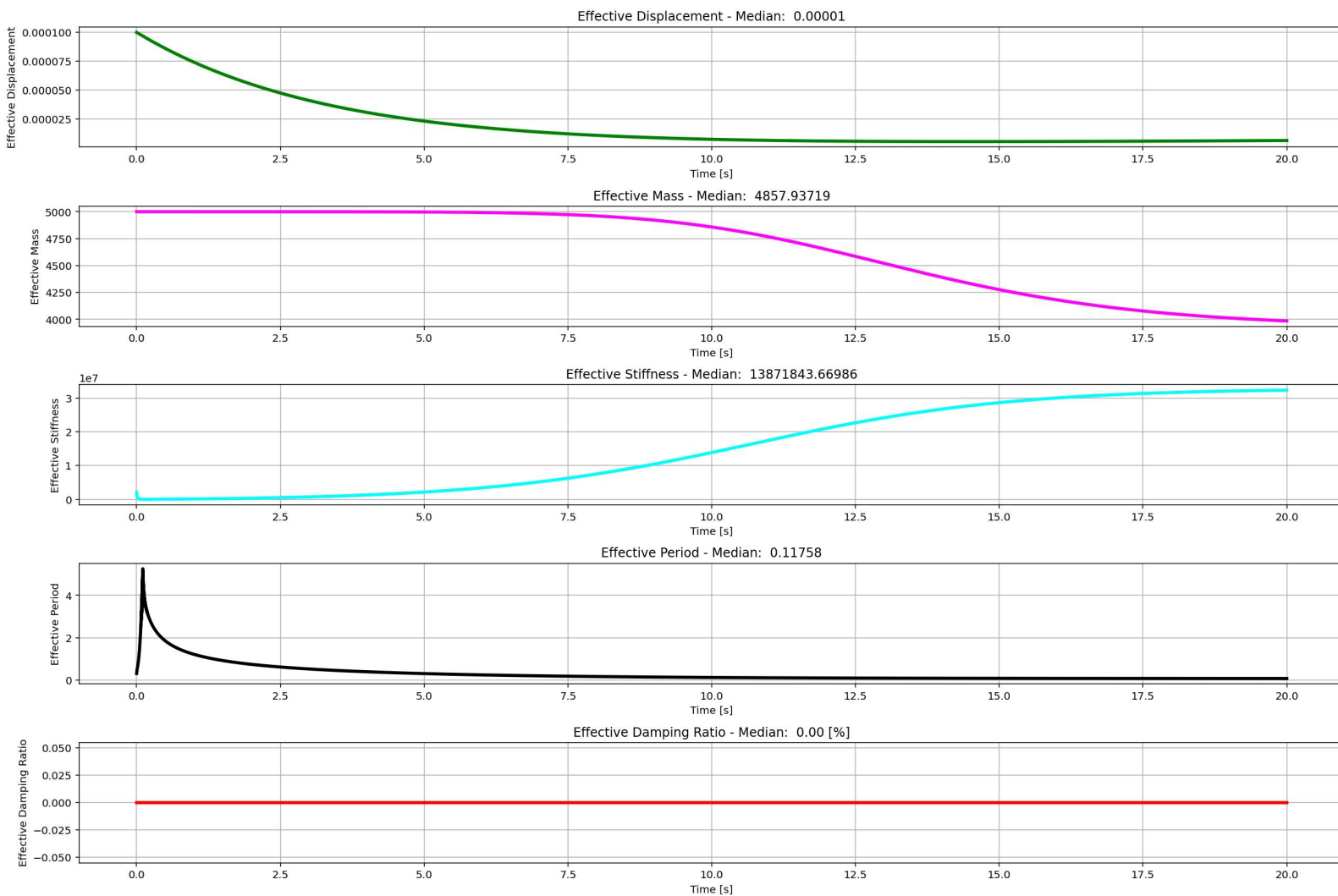
Effective Stiffness - Median: 1367143.00466

Effective Period - Median: 0.11750

Effective Damping Ratio - Median: 0.0067%

IPython Console Files Help Variable Explorer Debugger Plots History

Inline Conda: anaconda3 (Python 3.12.7) LSP: Python Line 1, Col 1 UTF-8 CRLF RW Mem 40%



$$\Delta_d = \frac{\sum_{i=1}^n m_i \Delta_i^2}{\sum_{i=1}^n m_i \Delta_i}$$

$$m_{eff} = \frac{\sum_{i=1}^n m_i \Delta_i}{\Delta_d}$$

$$k_{eff} = \frac{\sum_{i=1}^n k_i \Delta_i}{\Delta_d}$$

$$T_{eff} = \frac{2\pi}{\sqrt{\frac{k_{eff}}{m_{eff}}}}$$

$$\xi_{eff} = \frac{\sum_{i=1}^n \xi_i \Delta_i}{\Delta_d}$$

SEISMIC ANALYSIS

Spyder (Python 3.12)

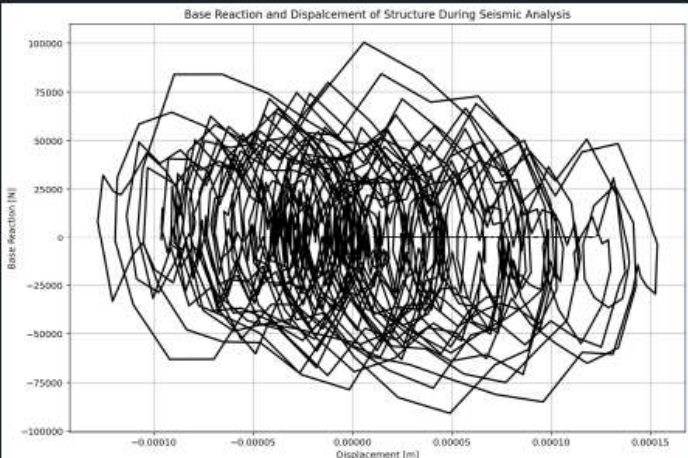
File Edit Search Source Run Debug Consoles Projects Tools View Help

C:\Users\Dell\Desktop\OPENSEES_FILES\+EIGHT_ANALY...TY_INDEX\33_MDOF_TWO_8_ANA\P(t)_MDOF_TWO_8_ANA.py

P(t)_MDOF_8_ANA.py X P(t)_MDOF_TWO_8_ANA.py X

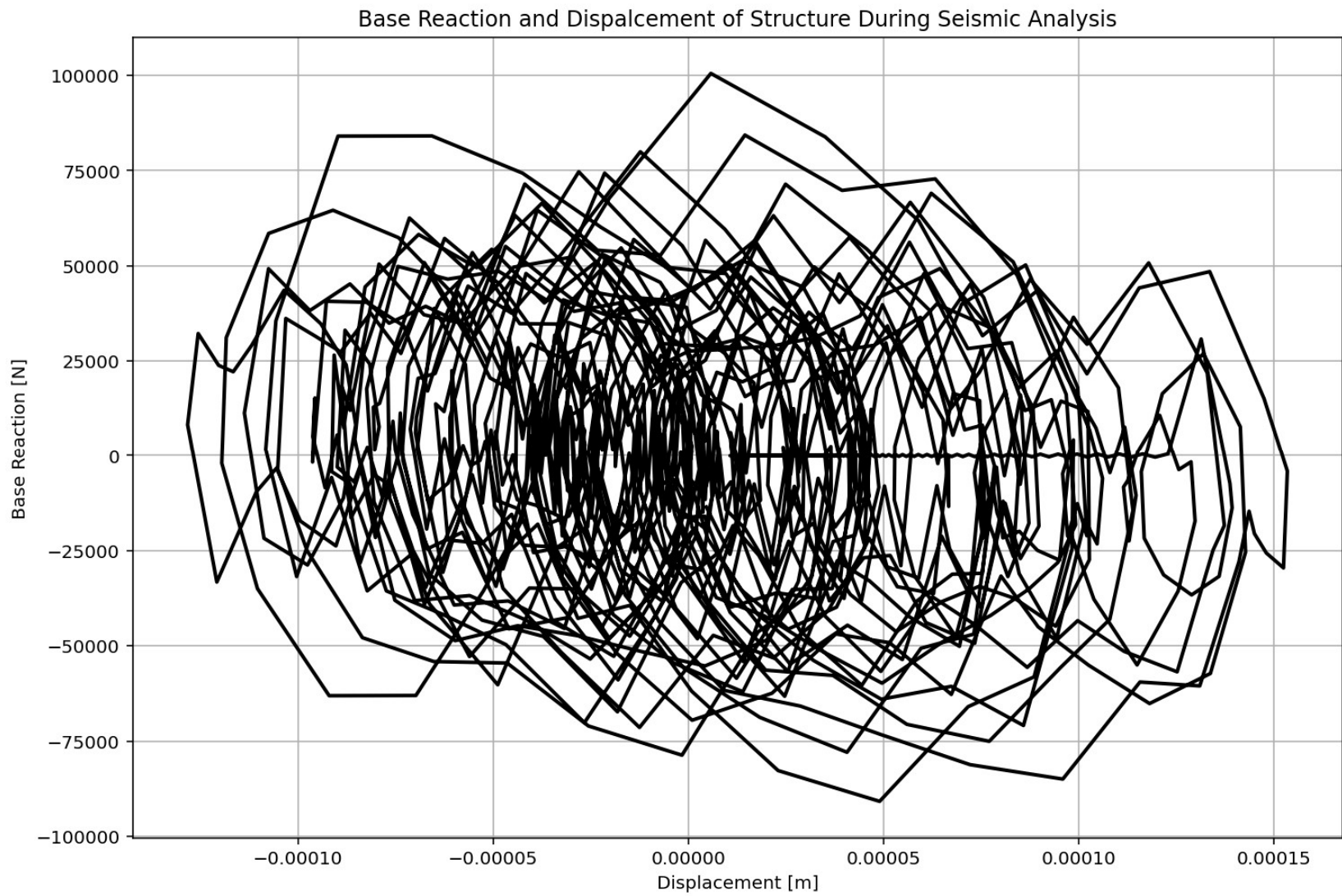
```
702
703 if ANAL_TYPE == 'SEISMIC': # DYNAMIC TIME-HISTORY ANALYSIS
704     %% DEFINE PARAMETERS FOR SEISMIC ANALYSIS
705     duration = 20.0           # [s] Analysis duration
706     dt = 0.01                # [s] Time step
707     GMfact = 9810            # [mm/s2] standard acceleration of gravity or st
708     SSF_X = 0.10             # Seismic Acceleration Scale Factor in X Direct
709     SSF_Y = 0.10             # Seismic Acceleration Scale Factor in Y Direct
710     iv0_X = 0.0005           # [mm/s] Initial velocity applied to the node
711     iv0_Y = 0.0005           # [mm/s] Initial velocity applied to the node
712     st_iv0 = 0.0             # [s] Initial velocity applied starting time
713     SEI = 'X'                # Seismic Direction
714     DR = 0.03                # Damping ratio
715
716 # Define time series for input motion (Acceleration time history)
717 if SEI == 'X':
718     SEISMIC_TAG_01 = 100
719     ops.timeSeries('Path', SEISMIC_TAG_01, '-dt', dt, '-filePath', 'Ground_Acceler
720     # Define load patterns
721     # pattern UniformExcitation $patternTag $dof -accel $tsTag <-vel0 $vel0> <-fac
722     ops.pattern('UniformExcitation', SEISMIC_TAG_01, 1, '-accel', SEISMIC_TAG_01,
723 if SEI == 'Y':
724     SEISMIC_TAG_02 = 200
725     ops.timeSeries('Path', SEISMIC_TAG_02, '-dt', dt, '-filePath', 'Ground_Acceler
726     ops.pattern('UniformExcitation', SEISMIC_TAG_02, 2, '-accel', SEISMIC_TAG_02,
727 if SEI == 'XY':
728     SEISMIC_TAG_01 = 100
729     ops.timeSeries('Path', SEISMIC_TAG_01, '-dt', dt, '-filePath', 'Ground_Acceler
730     # Define load patterns
731     # pattern UniformExcitation $patternTag $dof -accel $tsTag <-vel0 $vel0> <-fac
732     ops.pattern('UniformExcitation', SEISMIC_TAG_01, 1, '-accel', SEISMIC_TAG_01,
733     SEISMIC_TAG_02 = 200
734     ops.timeSeries('Path', SEISMIC_TAG_02, '-dt', dt, '-filePath', 'Ground_Acceler
735     ops.pattern('UniformExcitation', SEISMIC_TAG_02, 2, '-accel', SEISMIC_TAG_02,
```

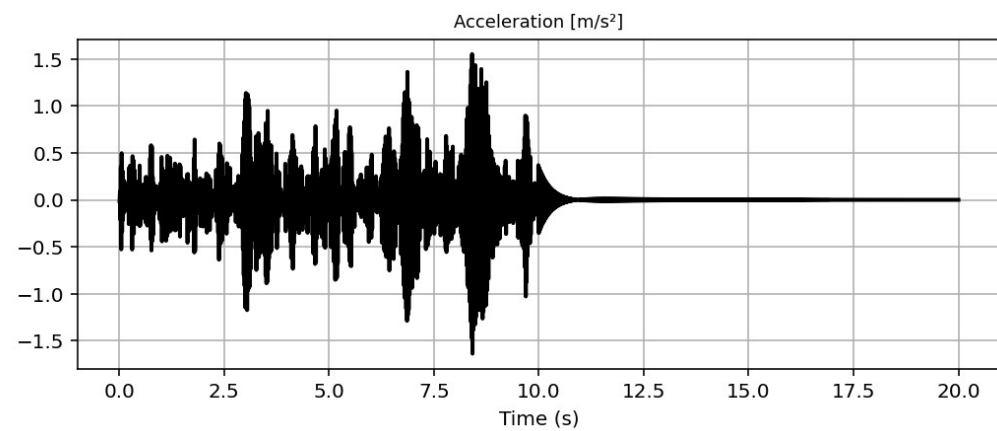
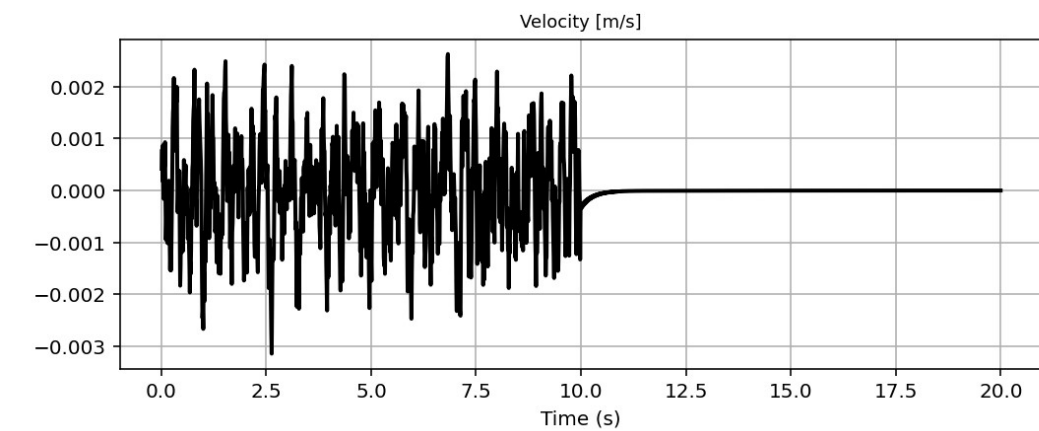
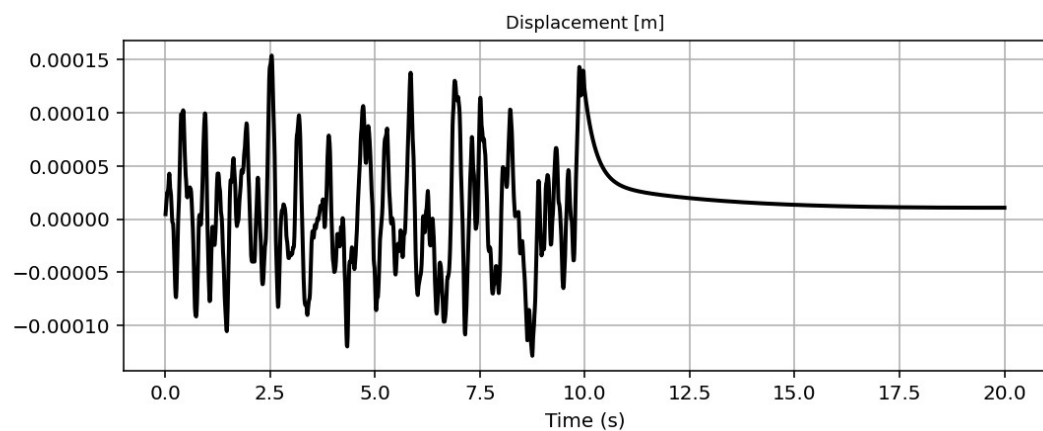
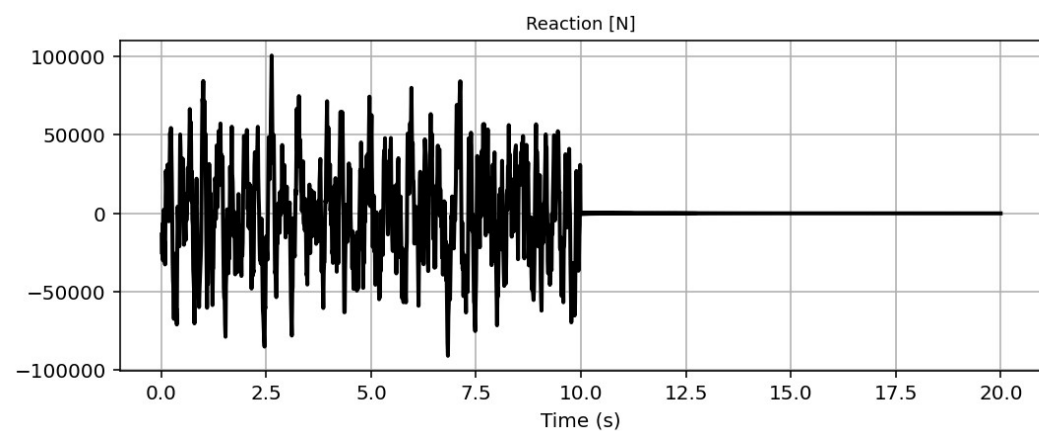
Base Reaction and Displacement of Structure During Seismic Analysis



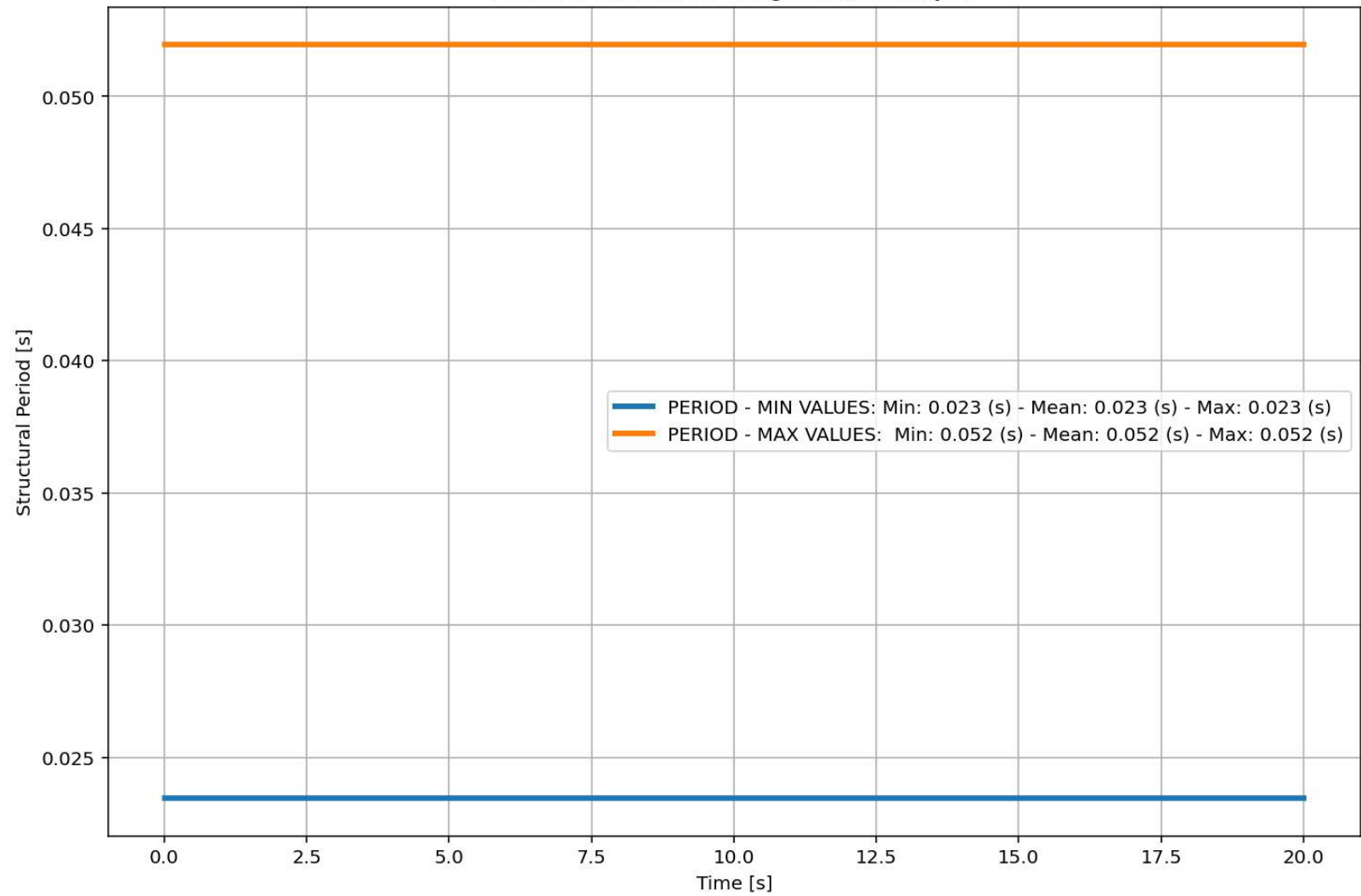
IPython Console Files Help Variable Explorer Debugger Plots History

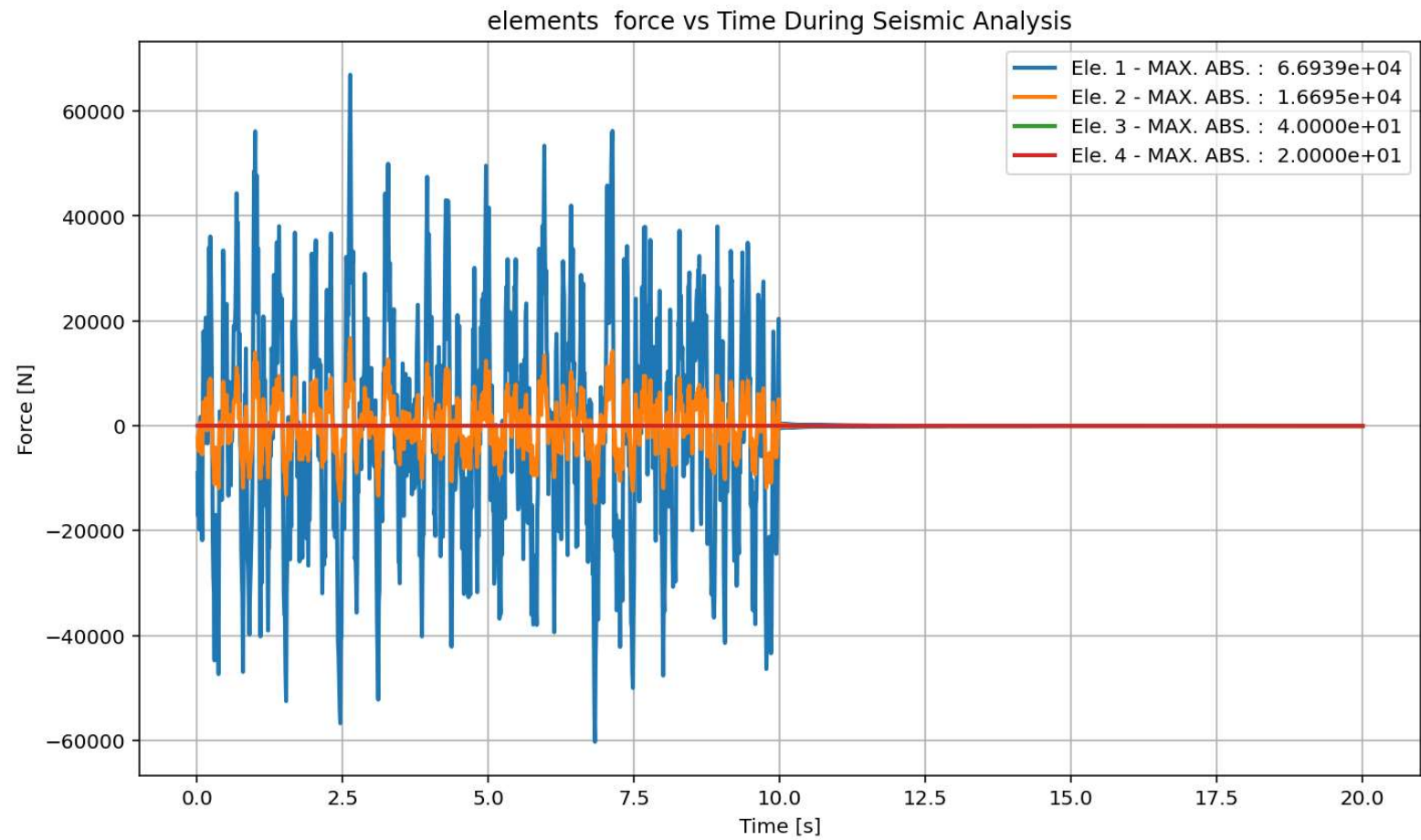
Inline Conda: anaconda3 (Python 3.12.7) LSP: Python Line 702, Col 5 UTF-8 CRLF RW Mem 46%



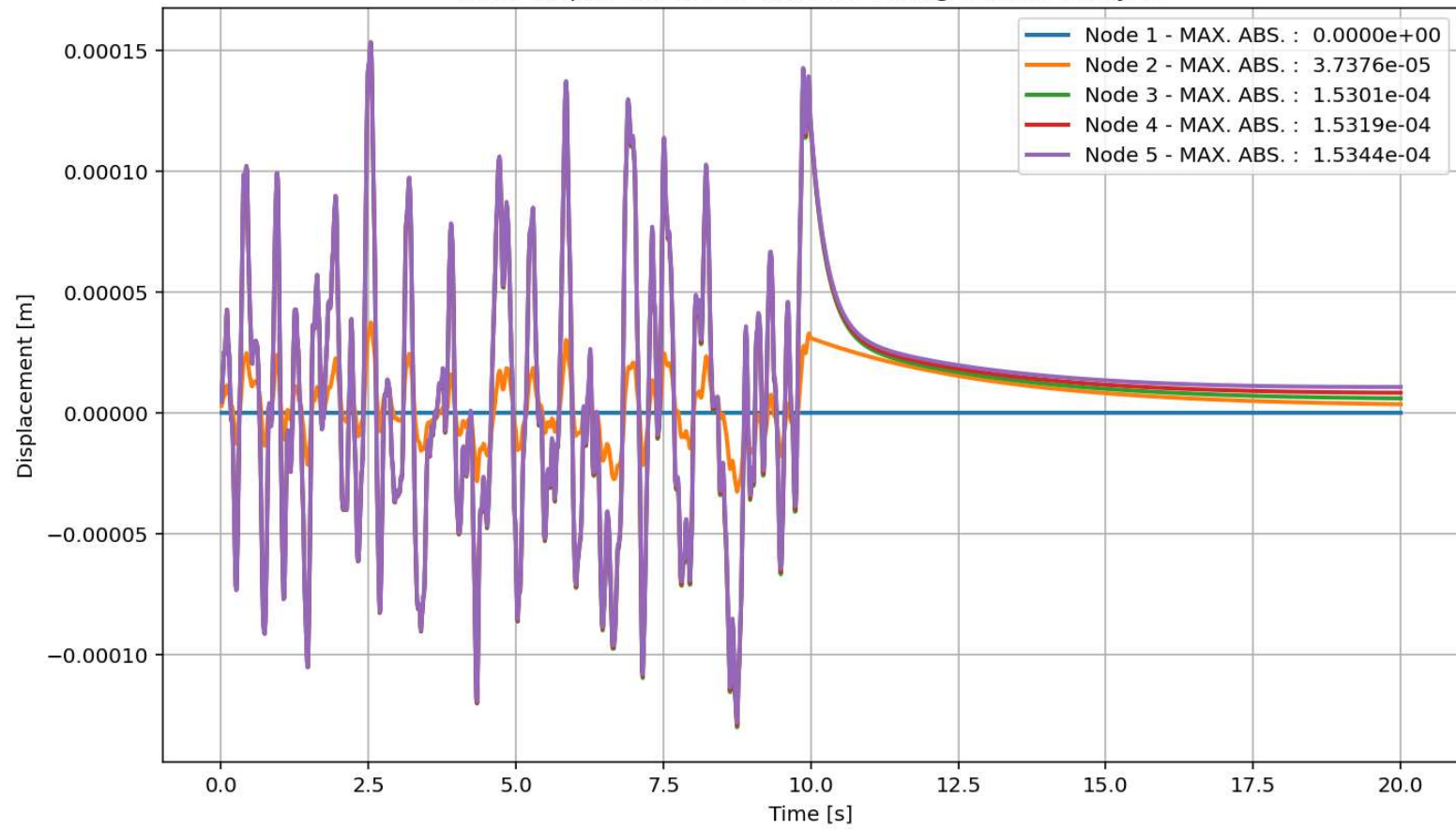


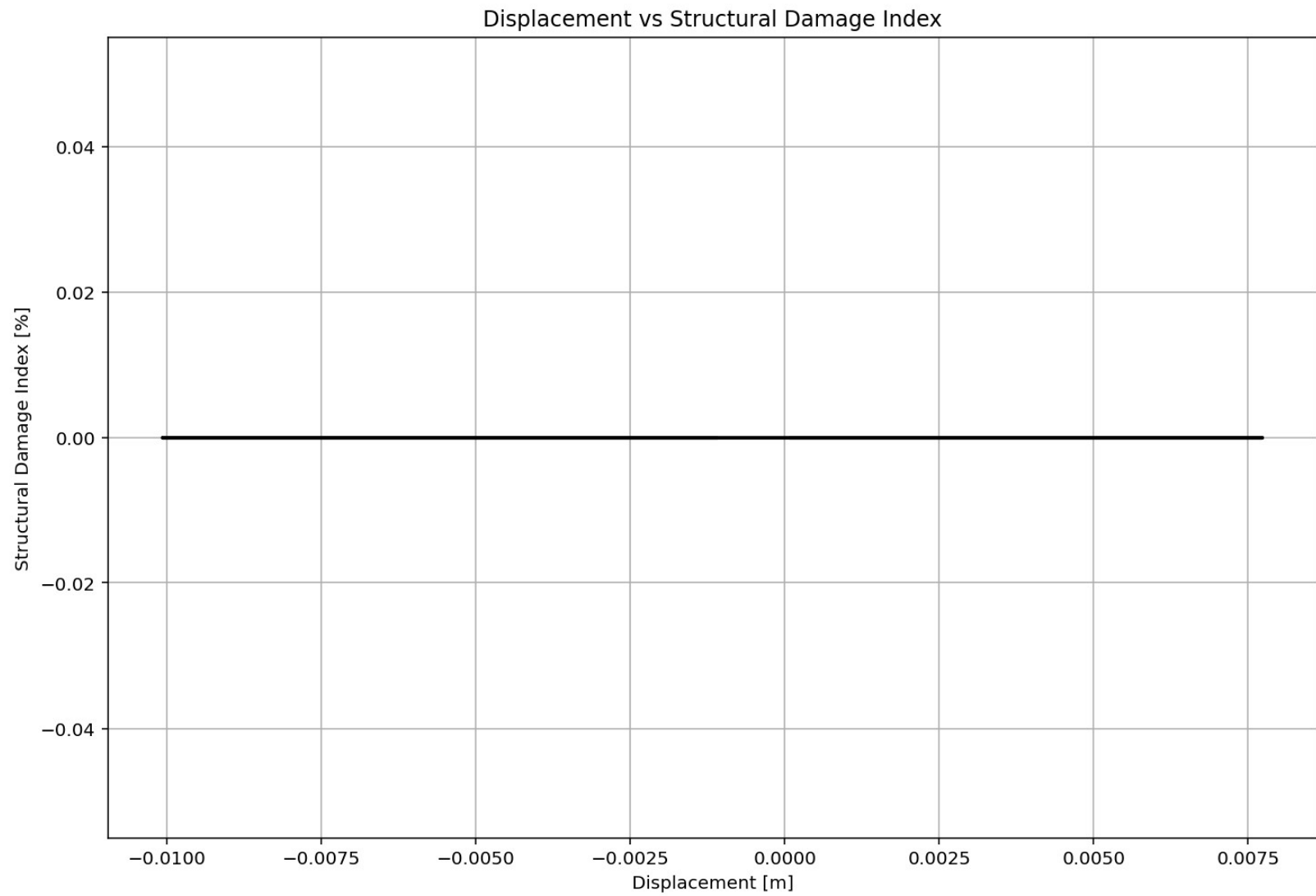
Period of Structure During Seismic Analysis



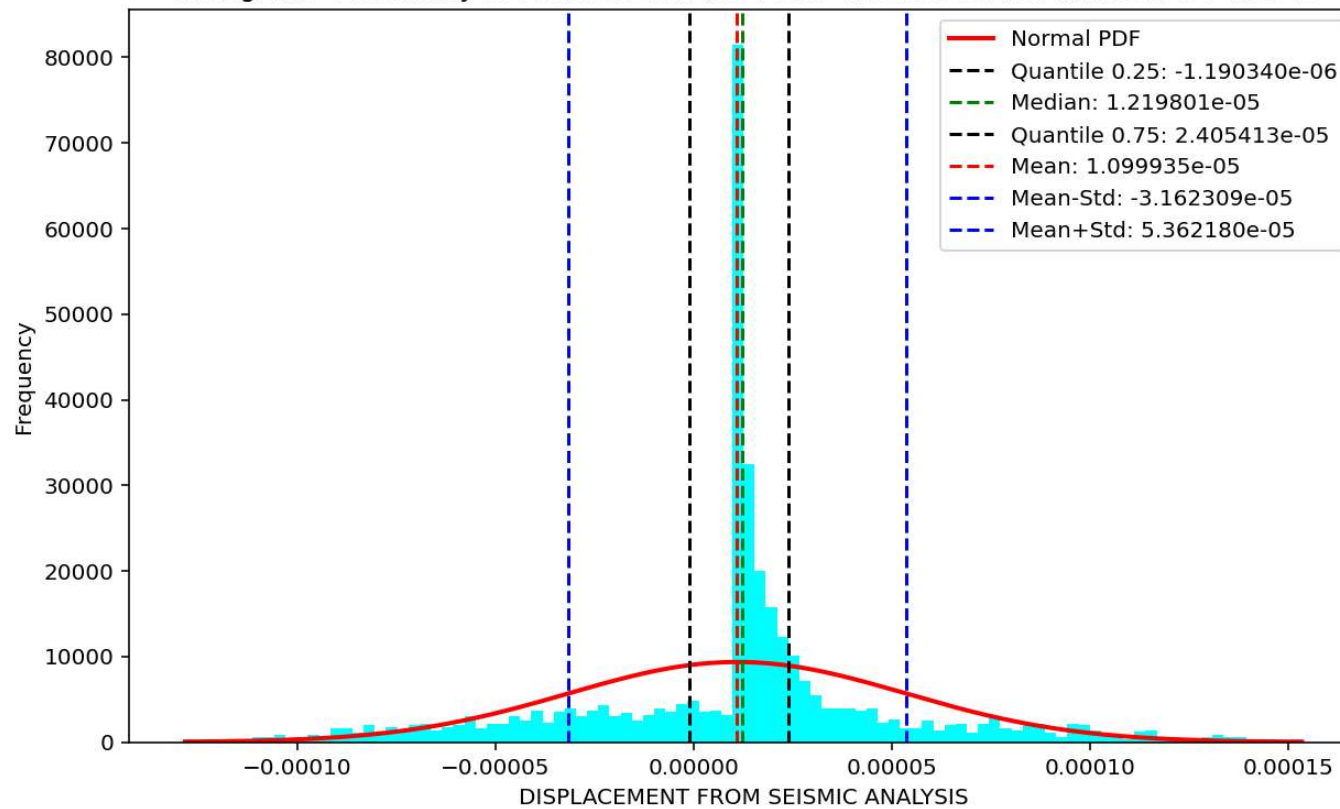


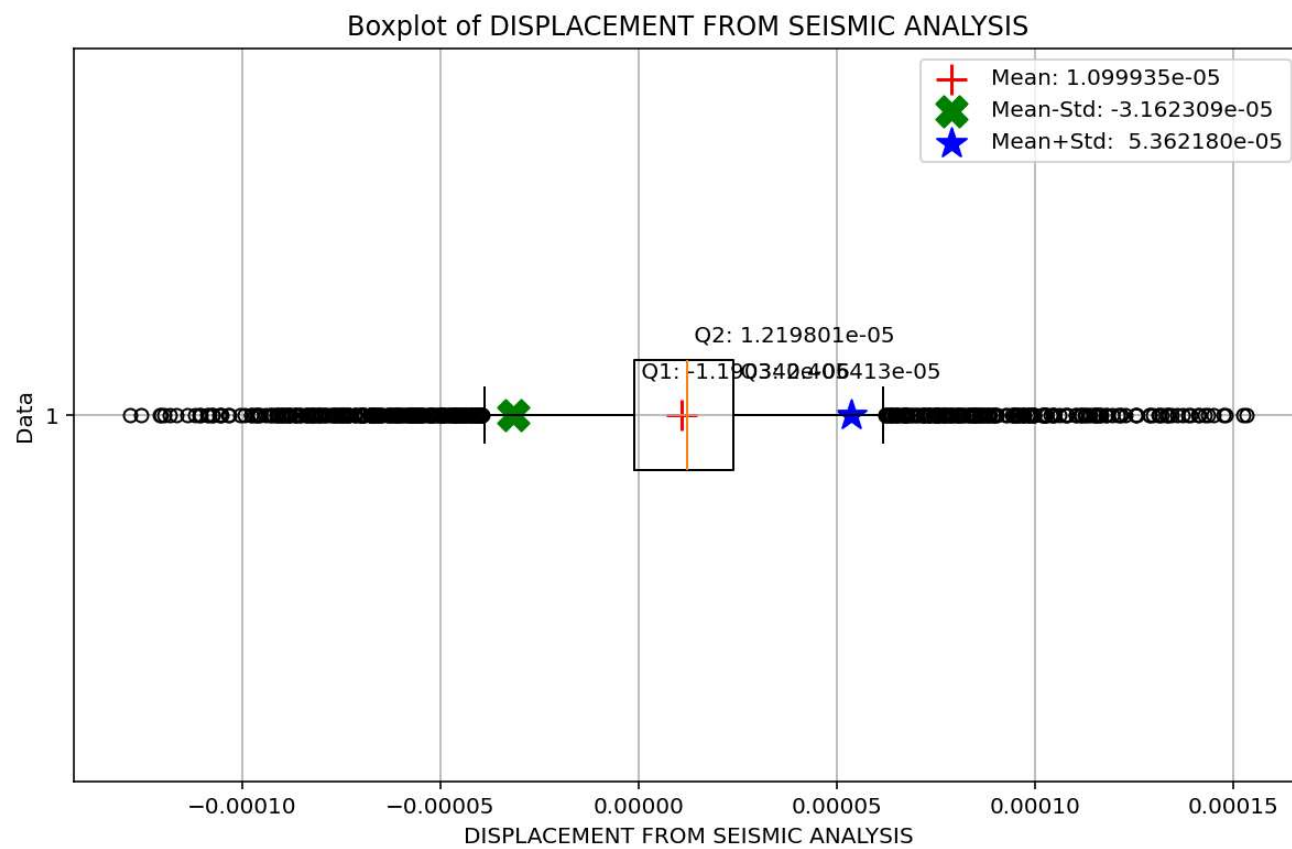
Node Displacements vs Time for During Seismic Analysis

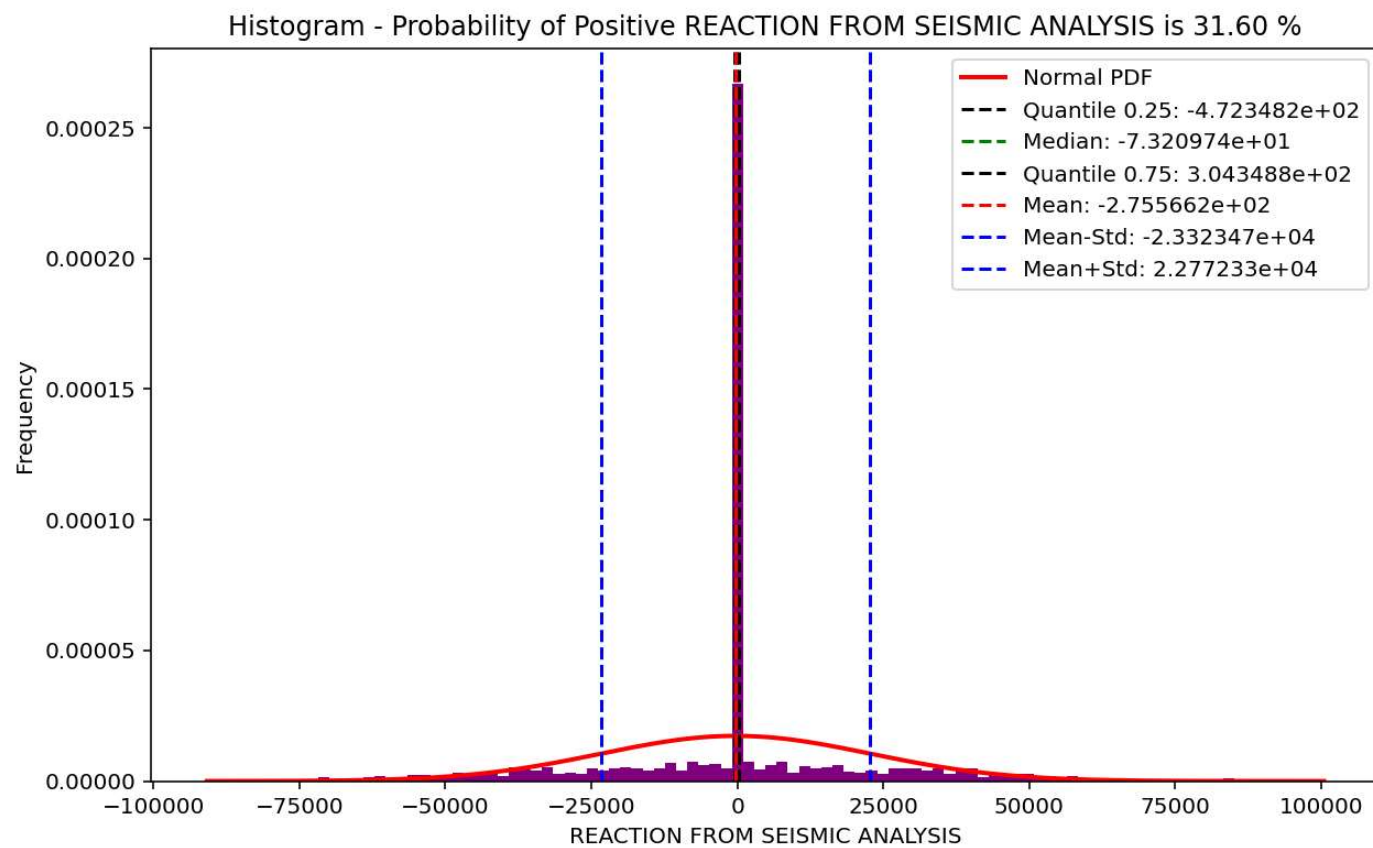


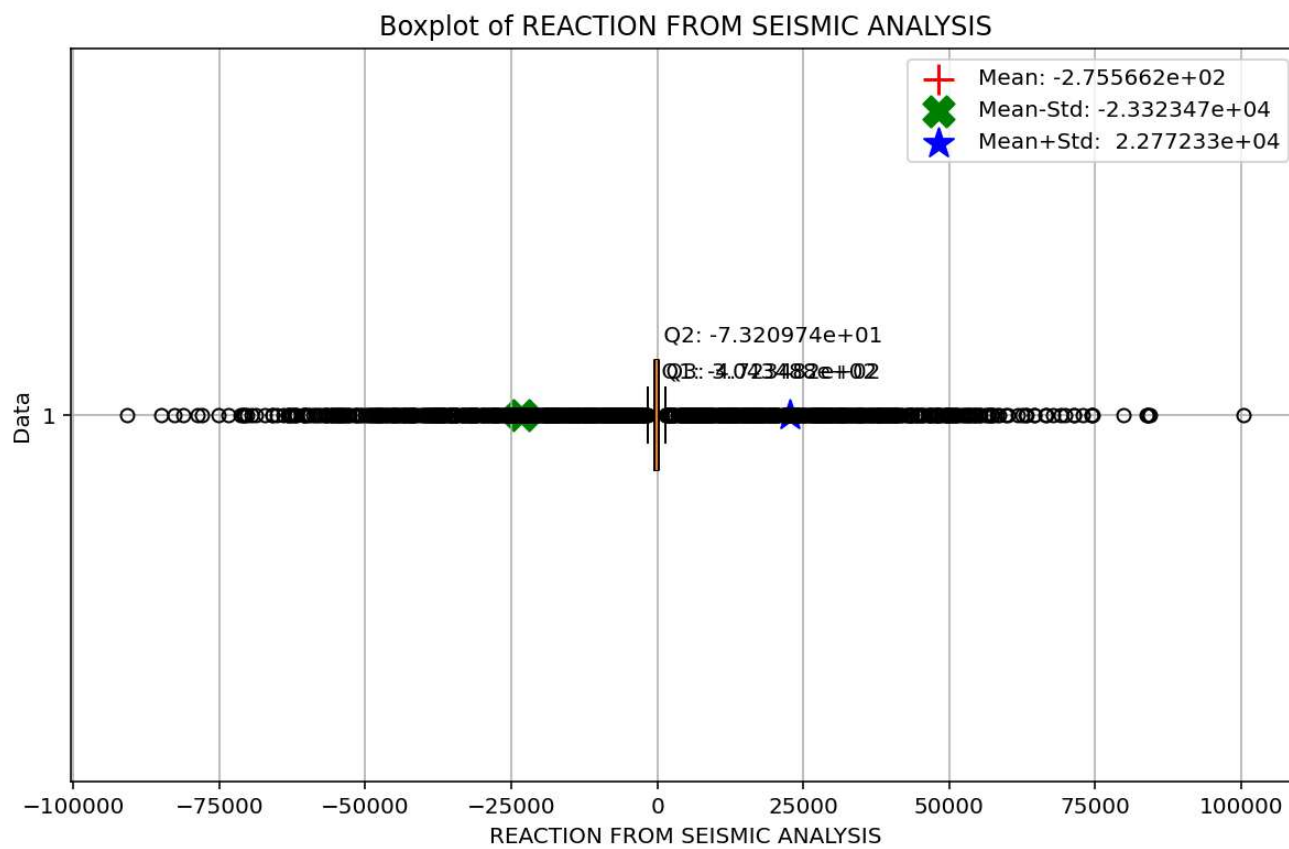


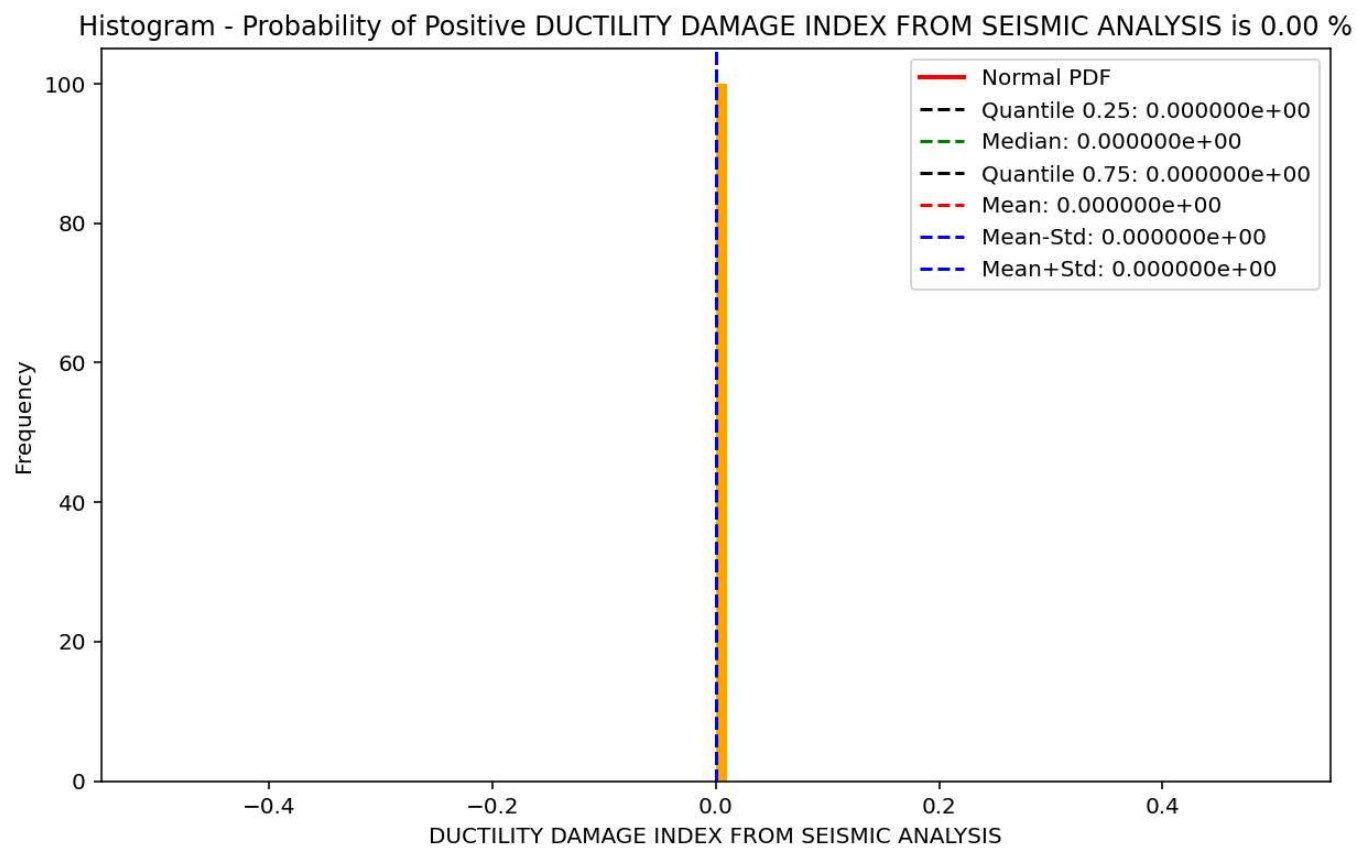
Histogram - Probability of Positive DISPLACEMENT FROM SEISMIC ANALYSIS is 74.50 %

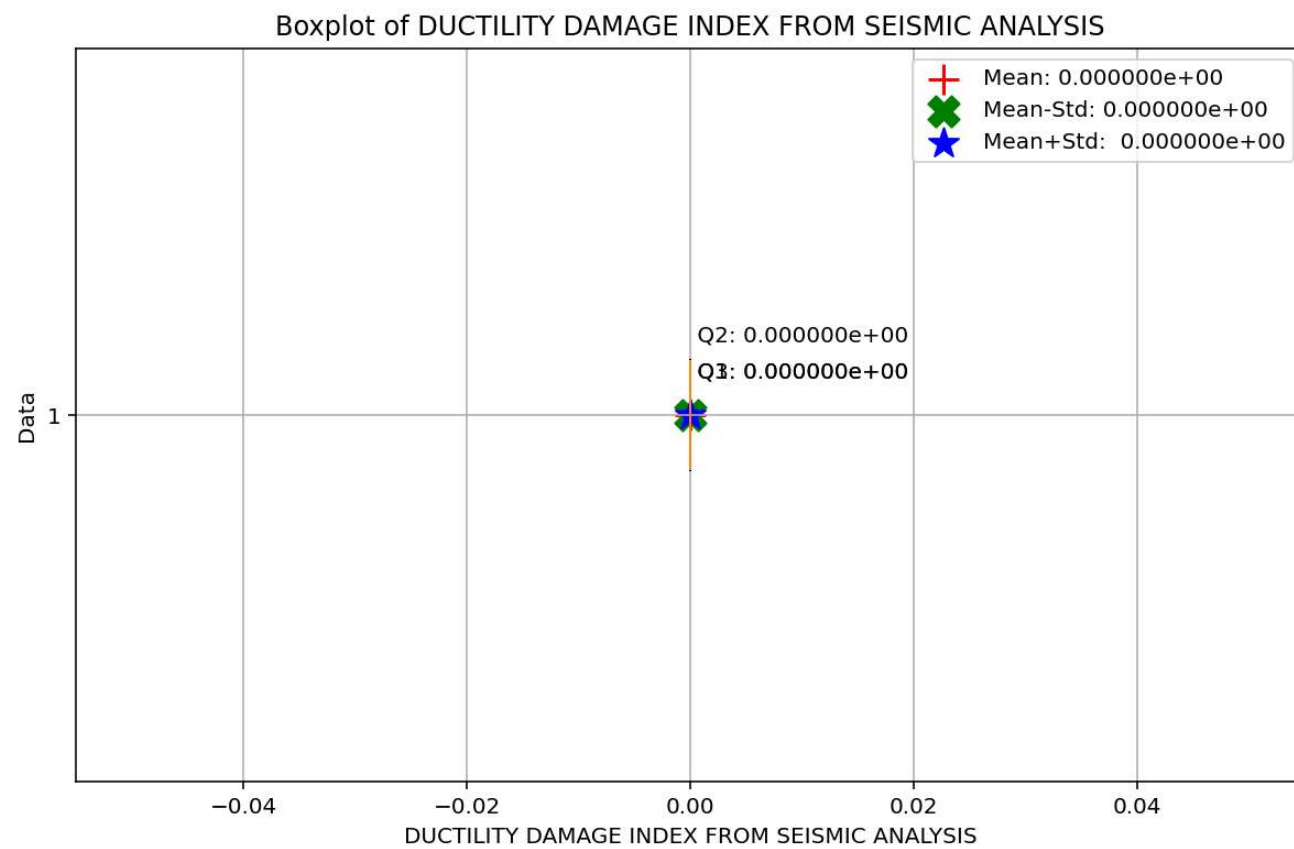




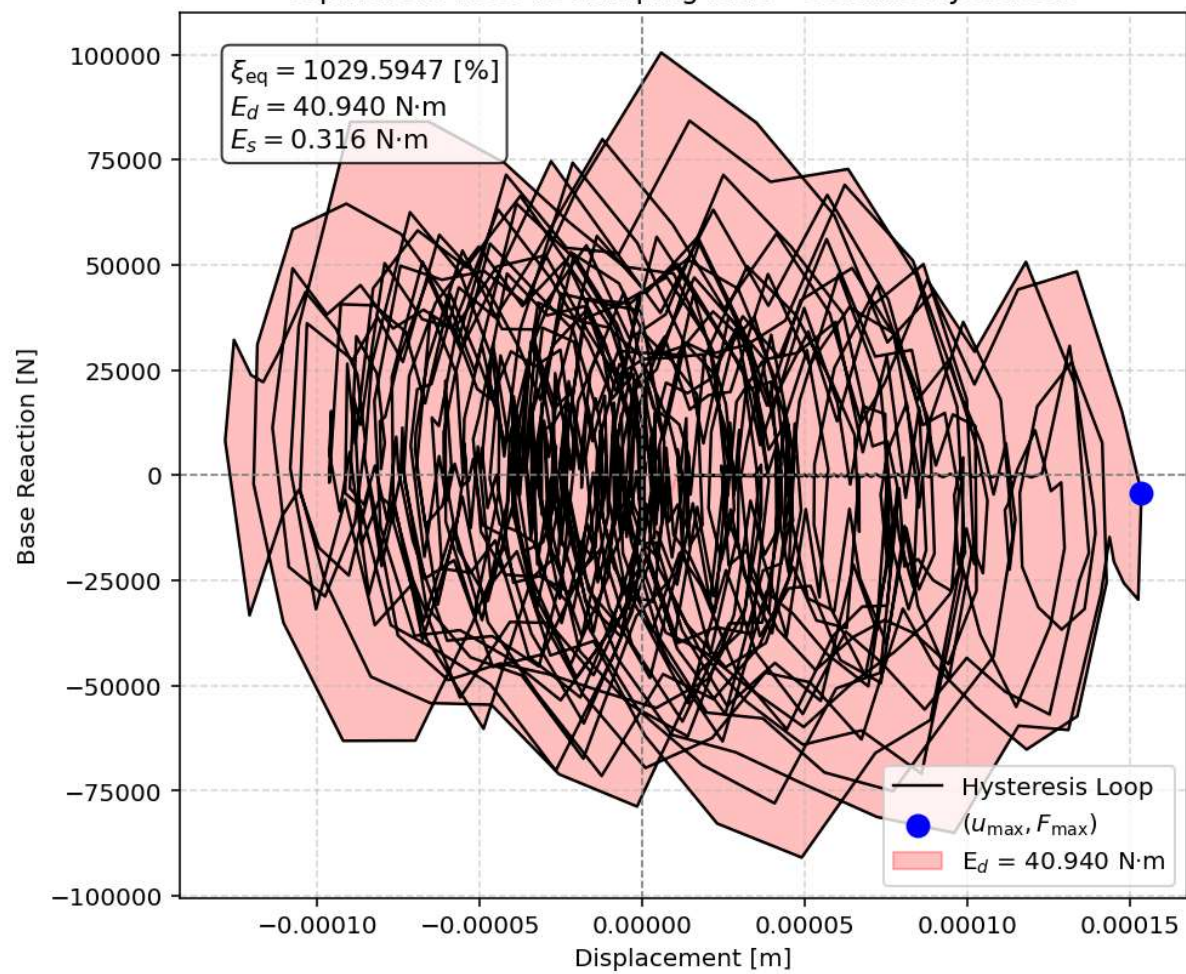




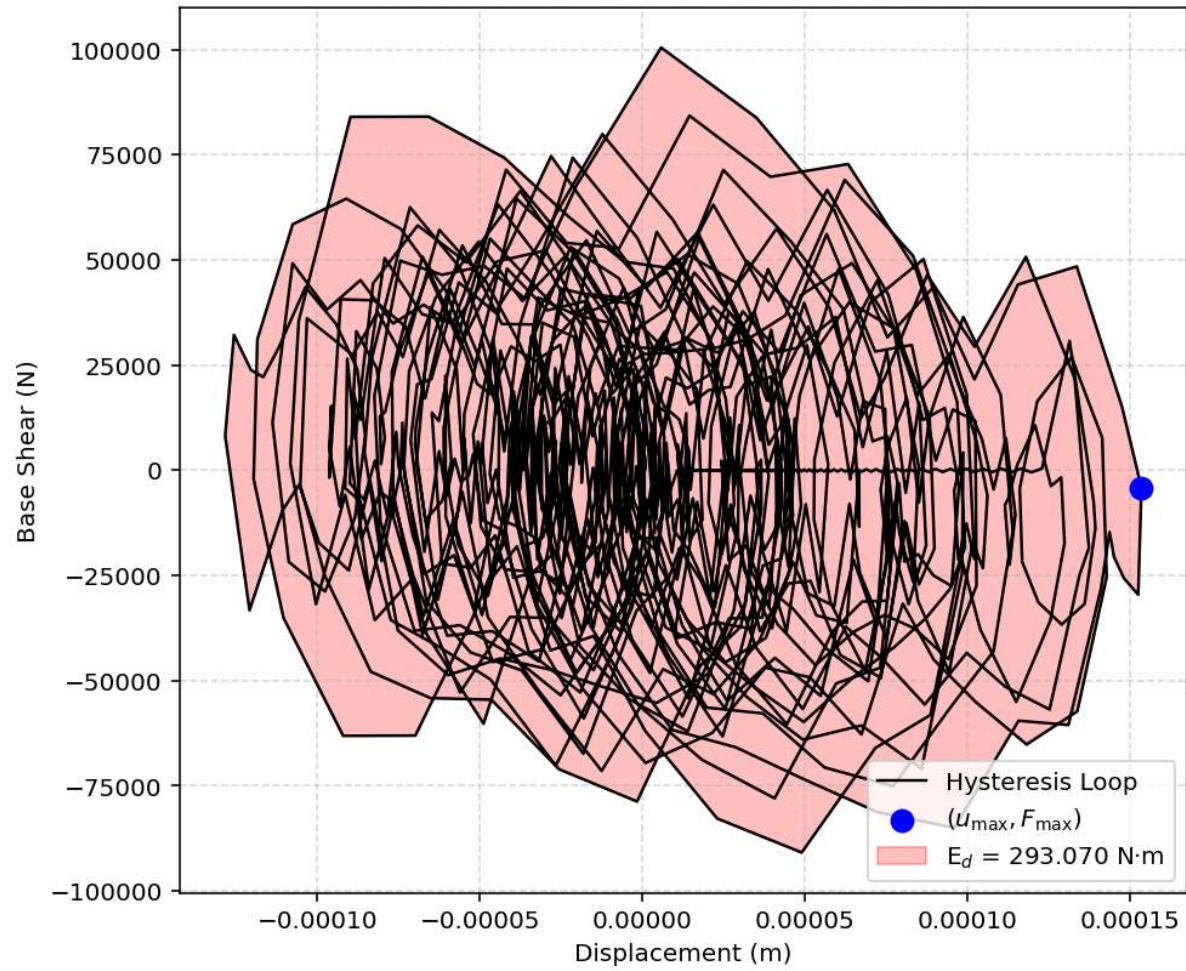


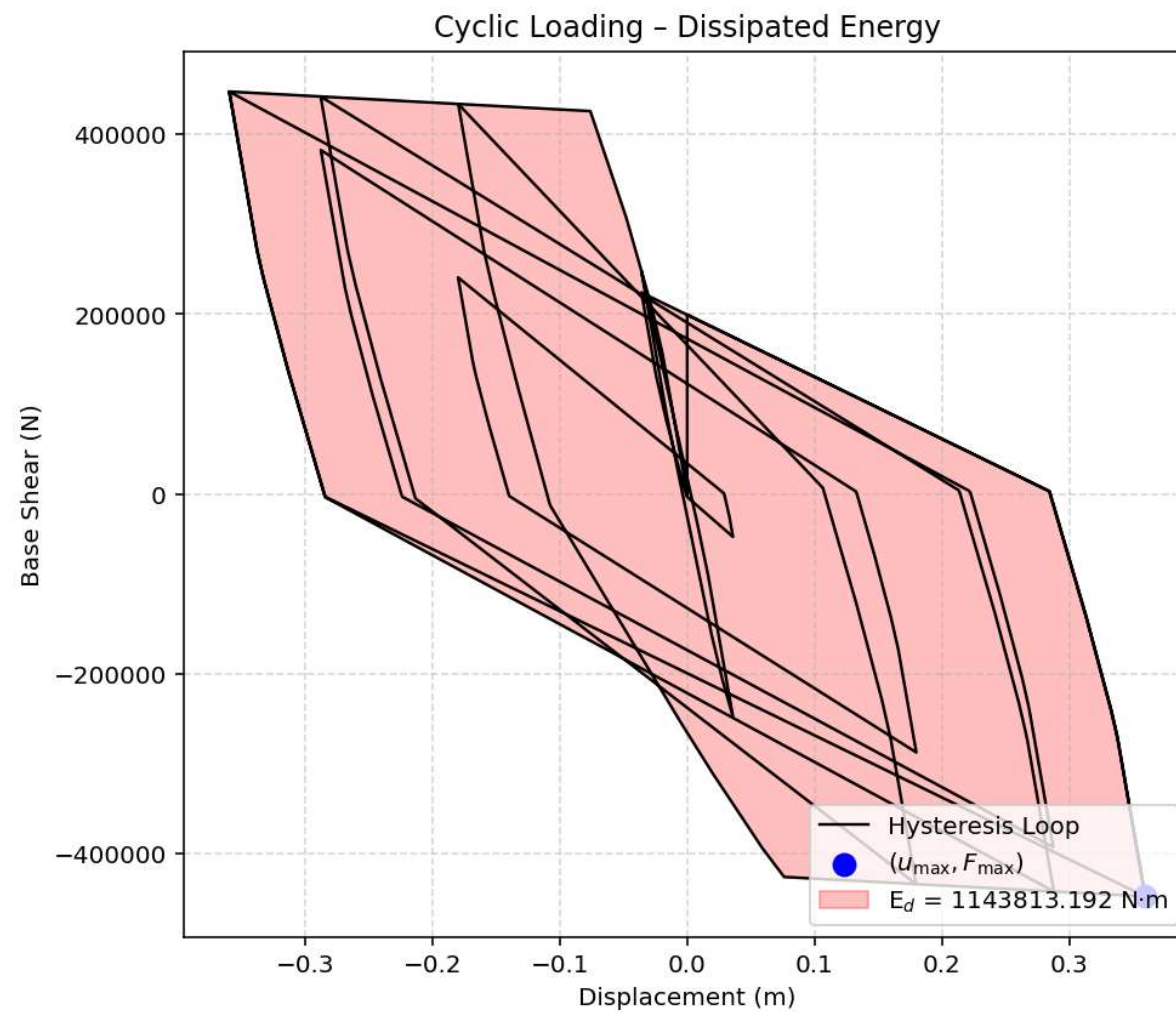


Equivalent viscous damping ratio - Seismic Hysteresis



Earthquake Response - Dissipated Energy





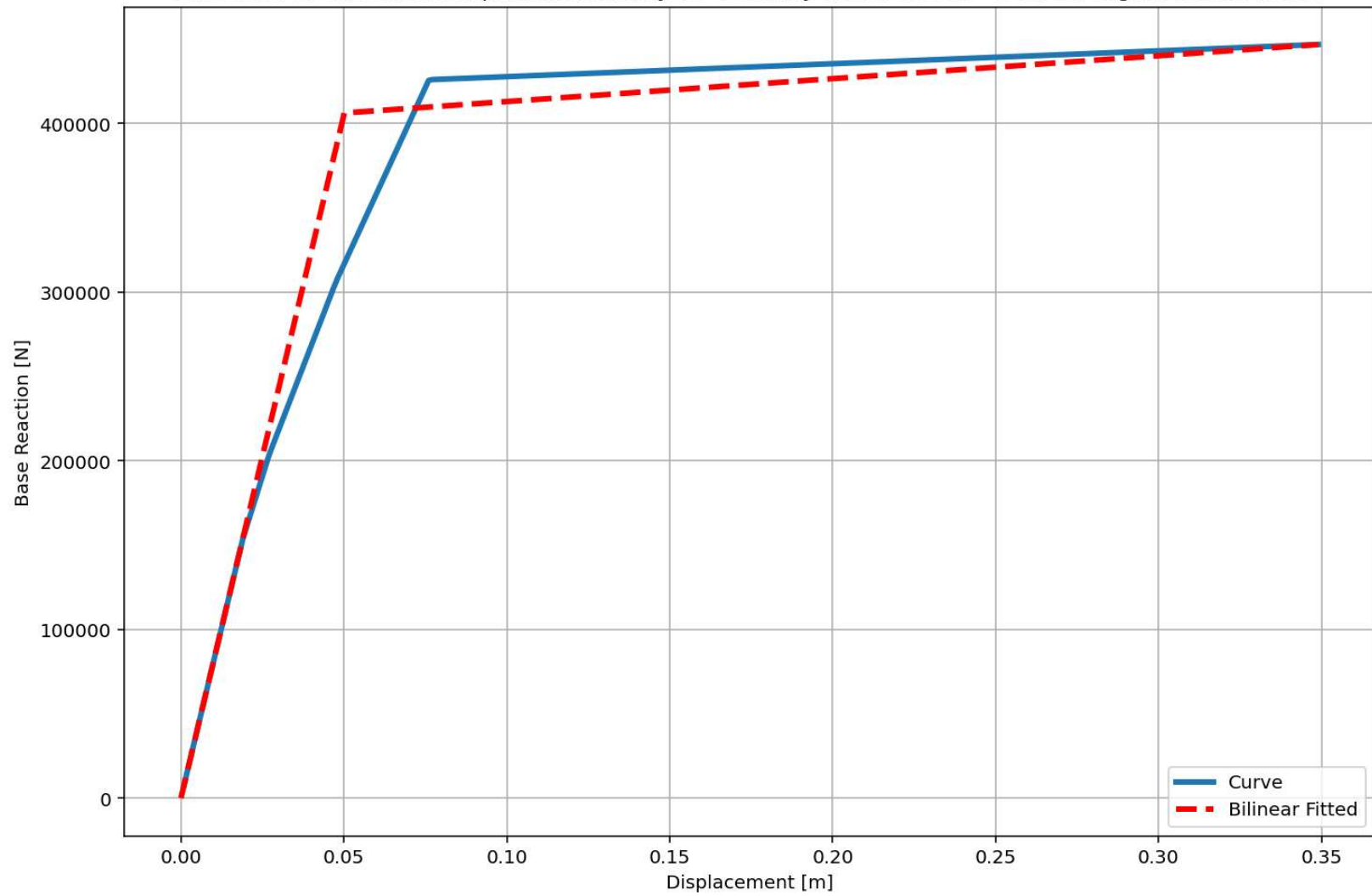
Dissipated Energy from Earthquake= 293.07 N·m

Dissipated Energy from Cyclic Displacement= 1143813.19 N·m

DISSIPATED ENERGY CAPACITY INDEX: 0.026 [%]

ZONE 1: VERY MINOR DAMAGE

Last Data of BaseShear-Displacement Analysis - Ductility Ratio: 6.9609 - Over Strength Factor: 1.0999



+=====+

= Analysis curve fitted =

Disp Base Shear

[[0.00000000e+00 0.00000000e+00]

[5.01374697e-02 4.06113504e+05]

[3.49000000e-01 4.46693631e+05]]

+=====+

+-----+

Structure Elastic Stiffness : 8100000.00

Structure Plastic Stiffness : 1279924.45

Structure Tangent Stiffness : 135781.92

Structure Ductility Ratio : 6.96

Structure Over Strength Factor: 1.10

+-----+

Over Strength Coefficient (Ω_0): 1.0999

Displacement Ductility Ratio (μ): 6.9609

Ductility Coefficient ($R\mu$): 6.9609

Structural Behavior Coefficient (R): 7.6564

Structural Ductility Damage Index in X Direction: -16.7248 (%)

Seismic Fragility Curves by Damage State from DYN. TIME-HISTORY ANA. DUCTILITY DAMAGE INDEX [%]

